

第48章 MDK 的编译过程及文件类型全解

本章参考资料：MDK 的帮助手册《ARM Development Tools》，点击 MDK 界面的“help->uVision Help”菜单可打开该文件。关于 ELF 文件格式，参考配套资料里的《ELF 文件格式》文件。

在本章中讲解了非常多的文件类型，学习时请跟着教程的节奏，打开实际工程中的文件来了解。

相信您已经非常熟练地使用 MDK 创建应用程序了，平时使用 MDK 编写源代码，然后编译生成机器码，再把机器码下载到 STM32 芯片上运行，但是这个编译、下载的过程 MDK 究竟做了什么工作？它编译后生成的各种文件又有什么作用？本章节将对这些过程进行讲解，了解编译及下载过程有助于理解芯片的工作原理，这些知识对制作 IAP(bootloader) 以及读写控制器内部 FLASH 的应用时非常重要。

48.1 编译过程

48.1.1 编译过程简介

首先我们简单了解下 MDK 的编译过程，它与其它编译器的工作过程是类似的，该过程见图 48-1。

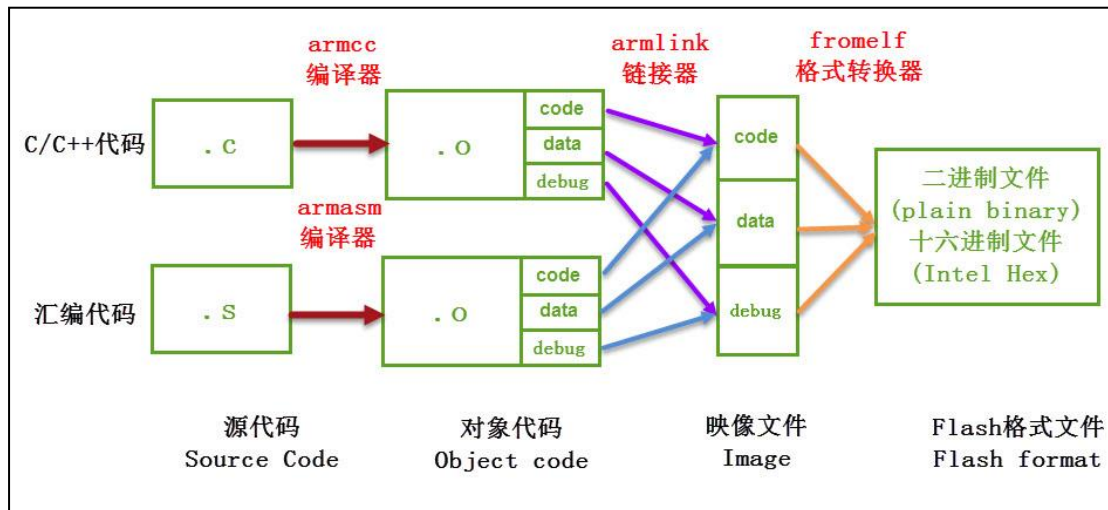


图 48-1 MDK 编译过程

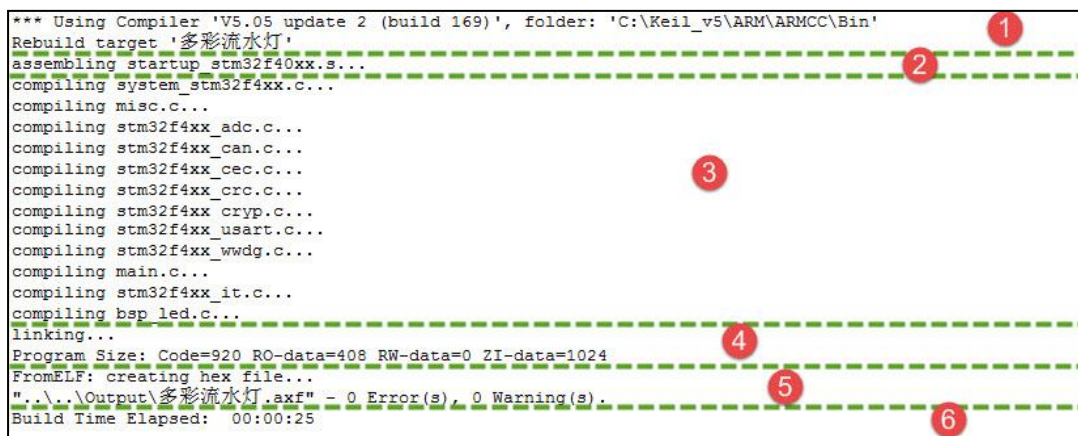
编译过程生成的不同文件将在后面的小节详细说明，此处先抓住主要流程来理解。

- (1) 编译，MDK 软件使用的编译器是 armcc 和 armasm，它们根据每个 c/c++ 和汇编源文件编译成对应的以“.o”为后缀名的对象文件(Object Code，也称目标文件)，其内容主要是从源文件编译得到的机器码，包含了代码、数据以及调试使用的信息；
- (2) 链接，链接器 armlink 把各个.o 文件及库文件链接成一个映像文件“.axf”或“.elf”；

- (3) 格式转换，一般来说 Windows 或 Linux 系统使用链接器直接生成可执行映像文件 elf 后，内核根据该文件的信息加载后，就可以运行程序了，但在单片机平台上，需要把该文件的内容加载到芯片上，所以还需要对链接器生成的 elf 映像文件利用格式转换器 fromelf 转换成“.bin”或“.hex”文件，交给下载器下载到芯片的 FLASH 或 ROM 中。

48.1.2 具体工程中的编译过程

下面我们打开“多彩流水灯”的工程，以它为例进行讲解，其它工程的编译过程也是一样的，只是文件有差异。打开工程后，点击 MDK 的“rebuild”按钮，它会重新构建整个工程，构建的过程会在 MDK 下方的“Build Output”窗口输出提示信息，见图 48-2。



```
*** Using Compiler 'V5.05 update 2 (build 169)', folder: 'C:\Keil_v5\ARM\ARMCC\Bin'
Rebuild target '多彩流水灯'
assembling startup_stm32f40xx.s...
compiling system_stm32f4xx.c...
compiling misc.c...
compiling stm32f4xx_adc.c...
compiling stm32f4xx_can.c...
compiling stm32f4xx_cec.c...
compiling stm32f4xx_crc.c...
compiling stm32f4xx_cryp.c...
compiling stm32f4xx_usart.c...
compiling stm32f4xx_wwdg.c...
compiling main.c...
compiling stm32f4xx_it.c...
compiling bsp_led.c...
linking...
Program Size: Code=920 RO-data=408 RW-data=0 ZI-data=1024
FromELF: creating hex file...
"..\\..\\Output\\多彩流水灯.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:25
```

图 48-2 编译工程时的编译提示

构建工程的提示输出主要分 6 个部分，说明如下：

- (1) 提示信息的第一部分说明构建过程调用的编译器。图中的编译器名字是“V5.06(build 20)”，后面附带了该编译器所在的文件夹。在电脑上打开该路径，可看到该编译器包含图 48-3 中的各个编译工具，如 armar、armasm、armcc、armlink 及 fromelf，后面四个工具已在图 48-1 中已讲解，而 armar 是用于把.o 文件打包成 lib 文件的。

armar.exe	2015/7/27 18:03	应用程序	1,549 KB
armasm.exe	2015/7/27 18:03	应用程序	5,871 KB
armcc.exe	2015/7/27 18:03	应用程序	15,571 KB
armcompiler_libFNP.dll	2015/7/27 18:03	DLL 文件	4,656 KB
armlink.exe	2015/7/27 18:03	应用程序	6,379 KB
fromelf.exe	2015/7/27 18:03	应用程序	5,307 KB

图 48-3 编译工具

- (2) 使用 armasm 编译汇编文件。图中列出了编译 startup 启动文件时的提示，编译后每个汇编源文件都对应有一个独立的.o 文件。
- (3) 使用 armcc 编译 c/c++ 文件。图中列出了工程中所有的 c/c++ 文件的提示，同样地，编译后每个 c/c++ 源文件都对应有一个独立的.o 文件。
- (4) 使用 armlink 链接对象文件，根据程序的调用把各个.o 文件的内容链接起来，最后生成程序的 axf 映像文件，并附带程序各个域大小的说明，包括 Code、RO-data、RW-data 及 ZI-data 的大小。

- (5) 使用 fromelf 生成下载格式文件，它根据 axf 映像文件转化成 hex 文件，并列出编译过程出现的错误(Error)和警告(Warning)数量。
- (6) 最后一段提示给出了整个构建过程消耗的时间。
- 构建完成后，可在工程的“Output”及“Listing”目录下找到由以上过程生成的各种文件，见图 48-4。



图 48-4 编译后 Output 及 Listing 文件夹中的内容

可以看到，每个 C 源文件都对应生成了.o、.d 及.crf 后缀的文件，还有一些额外的.dep、.hex、.axf、.htm、.lnp、.sct、.lst 及.map 文件。

48.2 程序的组成、存储与运行

48.2.1 CODE、RO、RW、ZI Data 域及堆栈空间

在工程的编译提示输出信息中有一个语句“Program Size: Code=xx RO-data=xx RW-data=xx ZI-data=xx”，它说明了程序各个域的大小，编译后，应用程序中所有具有同一性质的数据(包括代码)被归到一个域，程序在存储或运行的时候，不同的域会呈现不同的状态，这些域的意义如下：

- ❑ Code：即代码域，它指的是编译器生成的机器指令，这些内容被存储到 ROM 区。
- ❑ RO-data: Read Only data，即只读数据域，它指程序中用到的只读数据，这些数据被存储在 ROM 区，因而程序不能修改其内容。例如 C 语言中 const 关键字定义的变量就是典型的 RO-data。

- ❑ **RW-data: Read Write data**, 即可读写数据域, 它指初始化为“非 0 值”的可读写数据, 程序刚运行时, 这些数据具有非 0 的初始值, 且运行的时候它们会常驻在 RAM 区, 因而应用程序可以修改其内容。例如 C 语言中使用定义的全局变量, 且定义时赋予“非 0 值”给该变量进行初始化。
 - ❑ **ZI-data: Zero Initialie data**, 即 0 初始化数据, 它指初始化为“0 值”的可读写数据域, 它与 RW-data 的区别是程序刚运行时这些数据初始值全都为 0, 而后续运行过程与 RW-data 的性质一样, 它们也常驻在 RAM 区, 因而应用程序可以更改其内容。例如 C 语言中使用定义的全局变量, 且定义时赋予“0 值”给该变量进行初始化(若定义该变量时没有赋予初始值, 编译器会把它当 ZI-data 来对待, 初始化为 0);
 - ❑ **ZI-data 的栈空间(Stack)及堆空间(Heap)**: 在 C 语言中, 函数内部定义的局部变量属于栈空间, 进入函数的时候从向栈空间申请内存给局部变量, 退出时释放局部变量, 归还内存空间。而使用 malloc 动态分配的变量属于堆空间。在程序中的栈空间和堆空间都是属于 ZI-data 区域的, 这些空间都会被初始化为 0 值。编译器给出的 ZI-data 占用的空间值中包含了堆栈的大小(经实际测试, 若程序中完全没有使用 malloc 动态申请堆空间, 编译器会优化, 不把堆空间计算在内)。
- 综上所述, 以程序的组成构件为例, 它们所属的区域类别见表 48-1。

表 48-1 程序组件所属的区域

程序组件	所属类别
机器代码指令	Code
常量	RO-data
初值非 0 的全局变量	RW-data
初值为 0 的全局变量	ZI-data
局部变量	ZI-data 栈空间
使用 malloc 动态分配的空间	ZI-data 堆空间

48.2.2 程序的存储与运行

RW-data 和 ZI-data 它们仅仅是初始值不一样而已, 为什么编译器非要把它们区分开? 这就涉及到程序的存储状态了, 应用程序具有静止状态和运行状态。静止态的程序被存储在非易失存储器中, 如 STM32 的内部 FLASH, 因而系统掉电后也能正常保存。但是当程序在运行状态的时候, 程序常常需要修改一些暂存数据, 由于运行速度的要求, 这些数据往往存放在内存中(RAM), 掉电后这些数据会丢失。因此, 程序在静止与运行的时候它在存储器中的表现是不一样的, 见图 48-5。

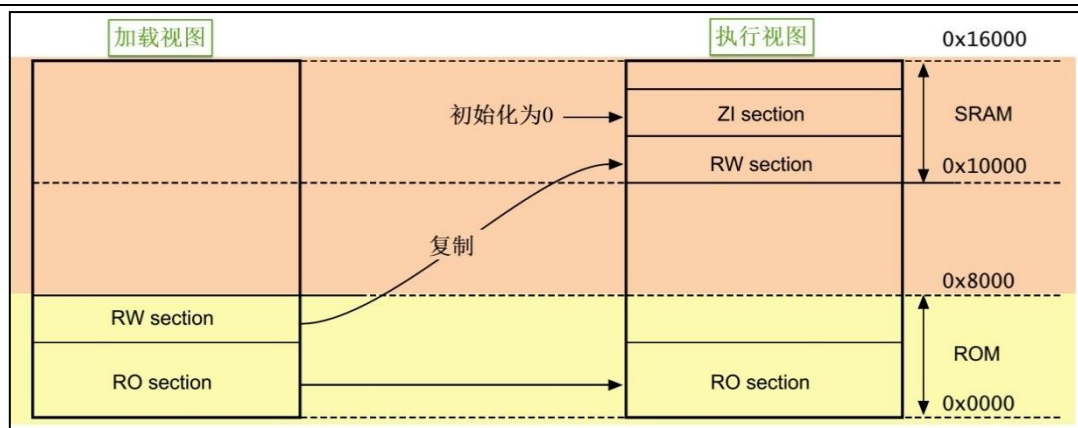


图 48-5 应用程序的加载视图与执行视图

图中的左侧是应用程序的存储状态，右侧是运行状态，而上方是 RAM 存储器区域，下方是 ROM 存储器区域。

程序在存储状态时，RO 节(RO section)及 RW 节都被保存在 ROM 区。当程序开始运行时，内核直接从 ROM 中读取代码，并且在执行主体代码前，会先执行一段加载代码，它把 RW 节数据从 ROM 复制到 RAM，并且在 RAM 加入 ZI 节，ZI 节的数据都被初始化为 0。加载完后 RAM 区准备完毕，正式开始执行主体程序。

编译生成的 RW-data 的数据属于图中的 RW 节，ZI-data 的数据属于图中的 ZI 节。是否需要掉电保存，这就是把 RW-data 与 ZI-data 区别开来的原因，因为在 RAM 创建数据的时候，默认值为 0，但如果有的数据要求初值非 0，那就需要使用 ROM 记录该初始值，运行时再复制到 RAM。

STM32 的 RO 区域不需要加载到 SRAM，内核直接从 FLASH 读取指令运行。计算机系统的应用程序运行过程很类似，不过计算机系统的程序在存储状态时位于硬盘，执行的时候甚至会把上述的 RO 区域(代码、只读数据)加载到内存，加快运行速度，还有虚拟内存管理单元(MMU)辅助加载数据，使得可以运行比物理内存还大的应用程序。而 STM32 没有 MMU，所以无法支持 Linux 和 Windows 系统。

当程序存储到 STM32 芯片的内部 FLASH 时(即 ROM 区)，它占用的空间是 Code、RO-data 及 RW-data 的总和，所以如果这些内容比 STM32 芯片的 FLASH 空间大，程序就无法被正常保存了。当程序在执行的时候，需要占用内部 SRAM 空间(即 RAM 区)，占用的空间包括 RW-data 和 ZI-data。应用程序在各个状态时各区域的组成见表 48-2。

表 48-2 程序状态区域的组成

程序状态与区域	组成
程序执行时的只读区域(RO)	Code + RO data
程序执行时的可读写区域(RW)	RW data + ZI data
程序存储时占用的 ROM 区	Code + RO data + RW data

在 MDK 中，我们建立的工程一般会选择芯片型号，选择后就有确定的 FLASH 及 SRAM 大小，若代码超出了芯片的存储器的极限，编译器会提示错误，这时就需要裁剪程序了，裁剪时可针对超出的区域来优化。

48.3 编译工具链

在前面编译过程中，MDK 调用了各种编译工具，平时我们直接配置 MDK，不需要学习如何使用它们，但了解它们是非常有好处的。例如，若希望使用 MDK 编译生成 bin 文件的，需要在 MDK 中输入指令控制 fromelf 工具；在本章后面讲解 AXF 及 O 文件的时候，需要利用 fromelf 工具查看其文件信息，这都是无法直接通过 MDK 做到的。关于这些工具链的说明，在 MDK 的帮助手册《ARM Development Tools》都有详细讲解，点击 MDK 界面的“help->uVision Help”菜单可打开该文件。

48.3.1 设置环境变量

调用这些编译工具，需要用到 Windows 的“命令行提示符工具”，为了让命令行方便地找到这些工具，我们先把工具链的目录添加到系统的环境变量中。查看本机工具链所在的具体目录可根据上一小节讲解的工程编译提示输出信息中找到，如本机的路径为“D:\work\keil5\ARM\ARMCC\bin”。

1. 添加路径到 PATH 环境变量

本文以 Win7 系统为例添加工具链的路径到 PATH 环境变量，其它系统是类似的。

(1) 右键电脑系统的“计算机图标”，在弹出的菜单中选择“属性”，见图 48-6；

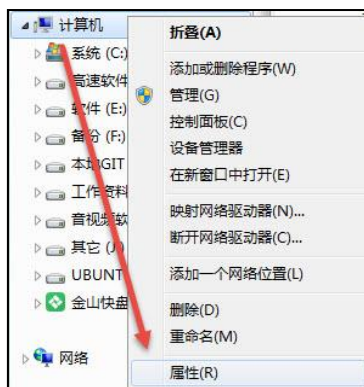


图 48-6 计算机属性页面

(2) 在弹出的属性页面依次点击“高级系统设置”->“环境变量”，在用户变量一栏中找到名为“PATH”的变量，若没有该变量，则新建一个。编辑“PATH”变量，在它的变量值中输入工具链的路径，如本机的是

“;D:\work\keil5\ARM\ARMCC\bin”，注意要使用“分号;”让它与其它路径分隔开，输入完毕后依次点确定，见图 48-7；

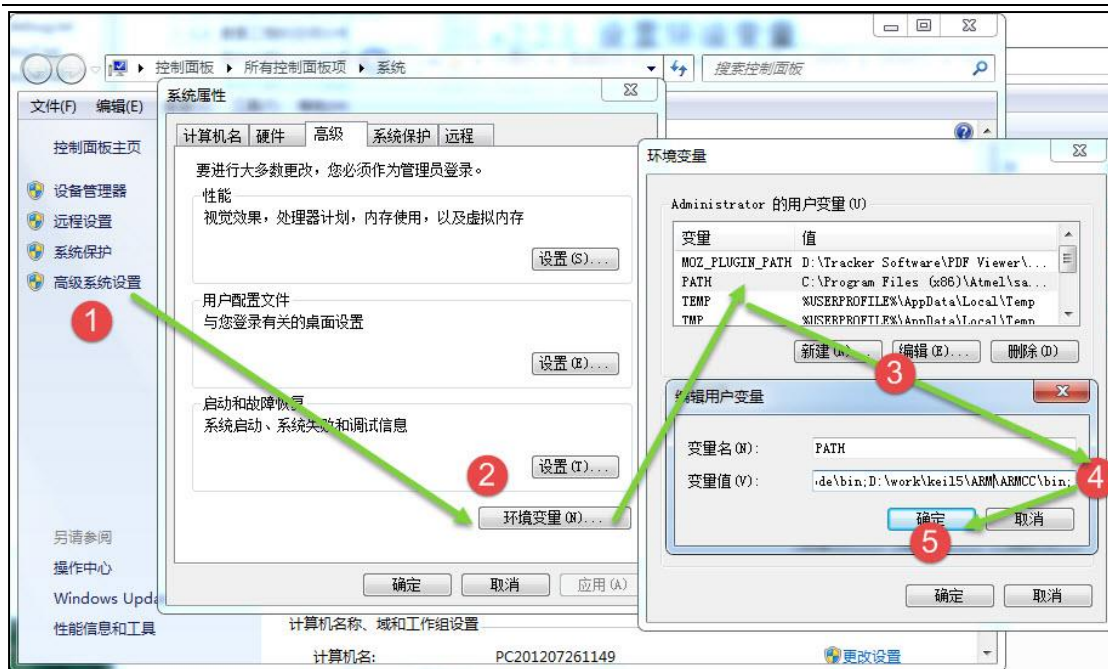


图 48-7 添加工具链路径到 PATH 变量

- (3) 打开 Windows 的命令行，点击系统的“开始菜单”，在搜索框输入“cmd”，在搜索结果中点击“cmd.exe”即可打开命令行，见图 48-8；

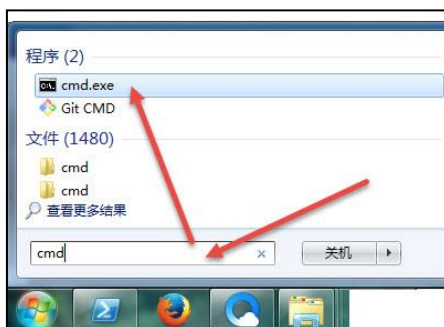


图 48-8 打开命令行

- (4) 在弹出的命令行窗口中输入“fromelf”回车，若窗口打印出 fromelf 的帮助说明，那么路径正常，就可以开始后面的工作了；若提示“不是内部或外部命令，也不是可运行的程序...”信息，说明路径不对，请重新配置环境变量，并确认该工作目录下有编译工具链。

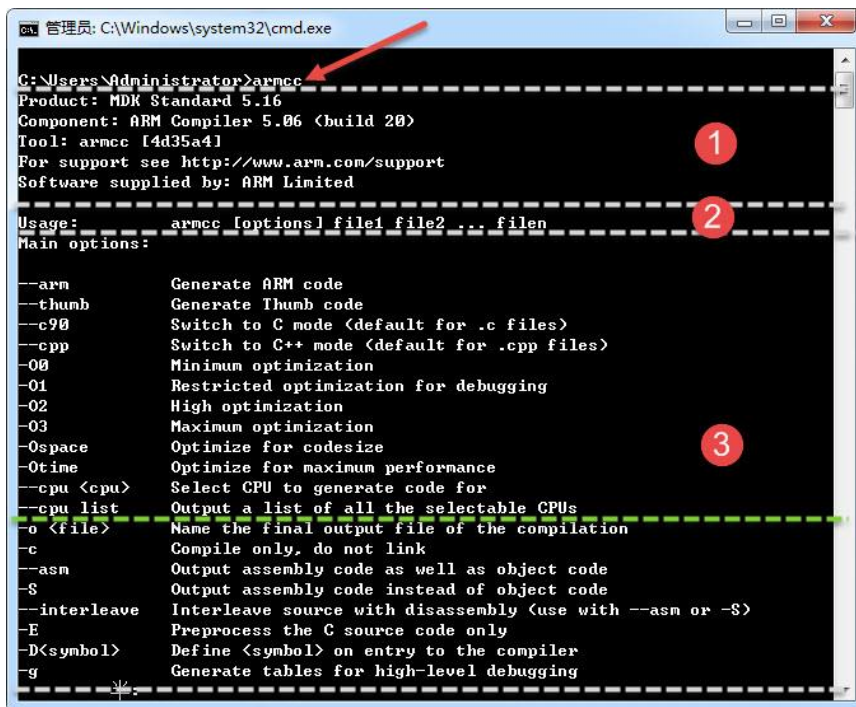
这个过程本质就是让命令行通过“PATH”路径找到“fromelf.exe”程序运行，默认运行“fromelf.exe”时它会输出自己的帮助信息，这就是工具链的调用过程，MDK 本质上也是如此调用工具链的，只是它集成为 GUI，相对于命令行对用户更友好，毕竟上述配置环境变量的过程已经让新手烦躁了。

48.3.2 armcc、armasm 及 armlink

接下来我们看看各个工具链的具体用法，主要以 armcc 为例。

1. armcc

armcc 用于把 c/c++ 文件编译成 ARM 指令代码，编译后会输出 ELF 格式的 O 文件(对象、目标文件)，在命令行中输入“armcc”回车可调用该工具，它会打印帮助说明，见图 48-9



```
C:\Users\Administrator>armcc
Product: MDK Standard 5.16
Component: ARM Compiler 5.06 <build 20>
Tool: armcc [4d35a4]
For support see http://www.arm.com/support
Software supplied by: ARM Limited

Usage: armcc [options] file1 file2 ... fileN
Main options:
--arm          Generate ARM code
--thumb       Generate Thumb code
--c90         Switch to C mode <default for .c files>
--cpp         Switch to C++ mode <default for .cpp files>
-O0           Minimum optimization
-O1           Restricted optimization for debugging
-O2           High optimization
-O3           Maximum optimization
-Ospace       Optimize for codesize
-Otime        Optimize for maximum performance
--cpu <cpu>   Select CPU to generate code for
--cpu list    Output a list of all the selectable CPUs
-o <file>     Name the final output file of the compilation
-c           Compile only, do not link
--asm         Output assembly code as well as object code
-S           Output assembly code instead of object code
--interleave  Interleave source with disassembly <use with --asm or -S>
-E           Preprocess the C source code only
-D<symbol>    Define <symbol> on entry to the compiler
-g           Generate tables for high-level debugging
```

图 48-9 armcc 的帮助提示

帮助提示中分三部分，第一部分是 armcc 版本信息，第二部分是命令的用法，第三部分是主要命令选项。

根据命令用法：armcc [options] file1 file2 ... fileN，在[option]位置可输入下面的“--arm”、“--cpu list”选项，若选项带文件输入，则把文件名填充在 file1 file2...的位置，这些文件一般是 c/c++ 文件。

例如根据它的帮助说明，“--cpu list”可列出编译器支持的所有 cpu，我们在命令行中输入“armcc --cpu list”，可查看图 48-10 中的 cpu 列表。

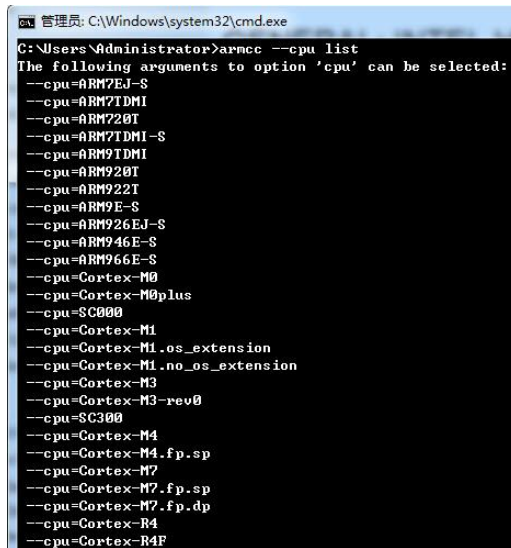


图 48-10 cpulist

打开 MDK 的 Options for Target->c/c++菜单, 可看到 MDK 对编译器的控制命令, 见图 48-11。

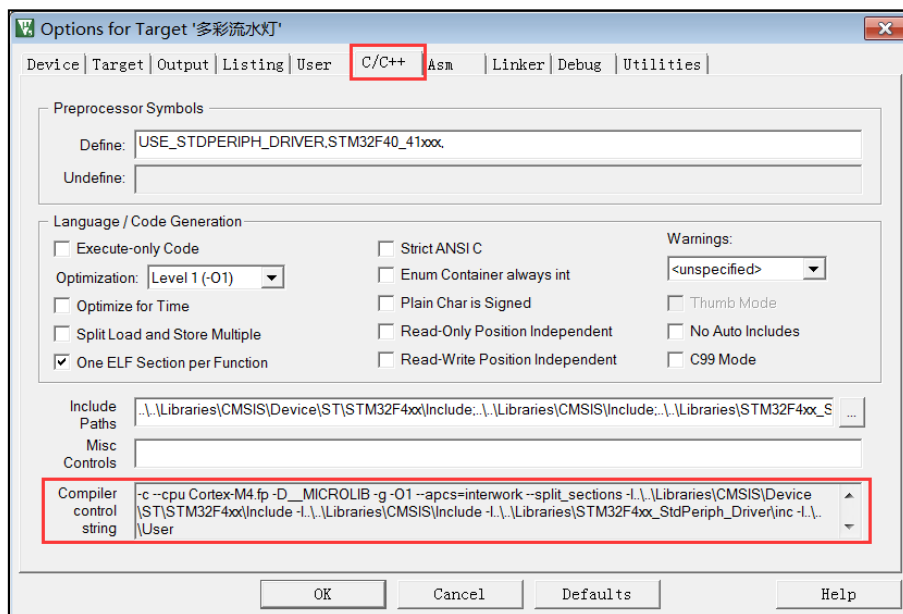


图 48-11 MDK 的 ARMCC 编译选项

从该图中的命令可看到, 它调用了 `-c`、`-cpu` `-D` `-g` `-O1` 等编译选项, 当我们修改 MDK 的编译配置时, 可看到该控制命令也会有相应的变化。然而我们无法在该编译选项框中输入命令, 只能通过 MDK 提供的选项修改。

了解这些, 我们就可以查询具体的 MDK 编译选项的具体信息了, 如 c/c++选项中的“Optimization: Leve 1 (-O1)”是什么功能呢? 首先可了解到它是“-O”命令, 命令后还带个数字, 查看 MDK 的帮助手册, 在 armcc 编译器说明章节, 可详细了解, 如图 48-9。

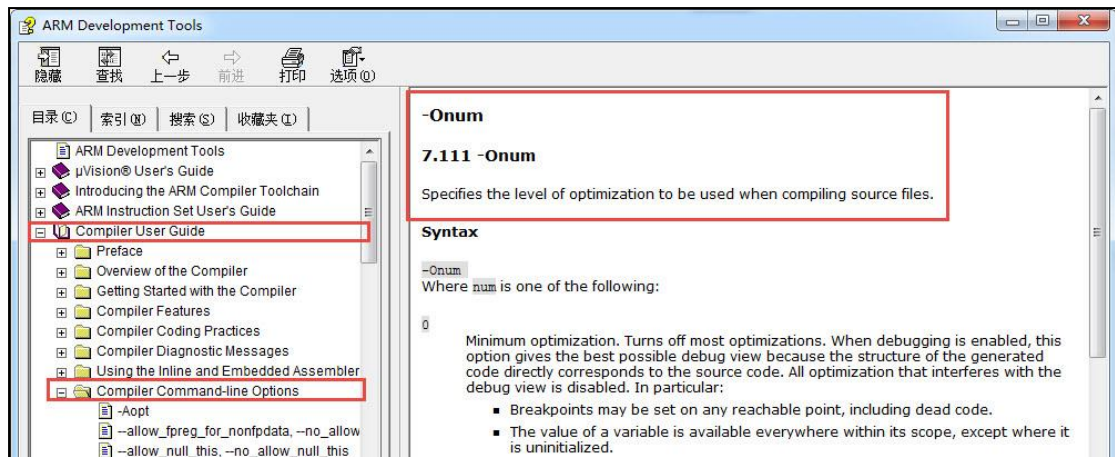


图 48-12 编译器选项说明

利用 MDK，我们一般不需要自己调用 armcc 工具，但经过这样的过程我们就会对 MDK 有更深入的认识，面对它的各种编译选项，就不会那么头疼了。

2. armasm

armasm 是汇编器，它把汇编文件编译成 O 文件。与 armcc 类似，MDK 对 armasm 的调用选项可在“Option for Target->Asm”页面进行配置，见图 48-13。

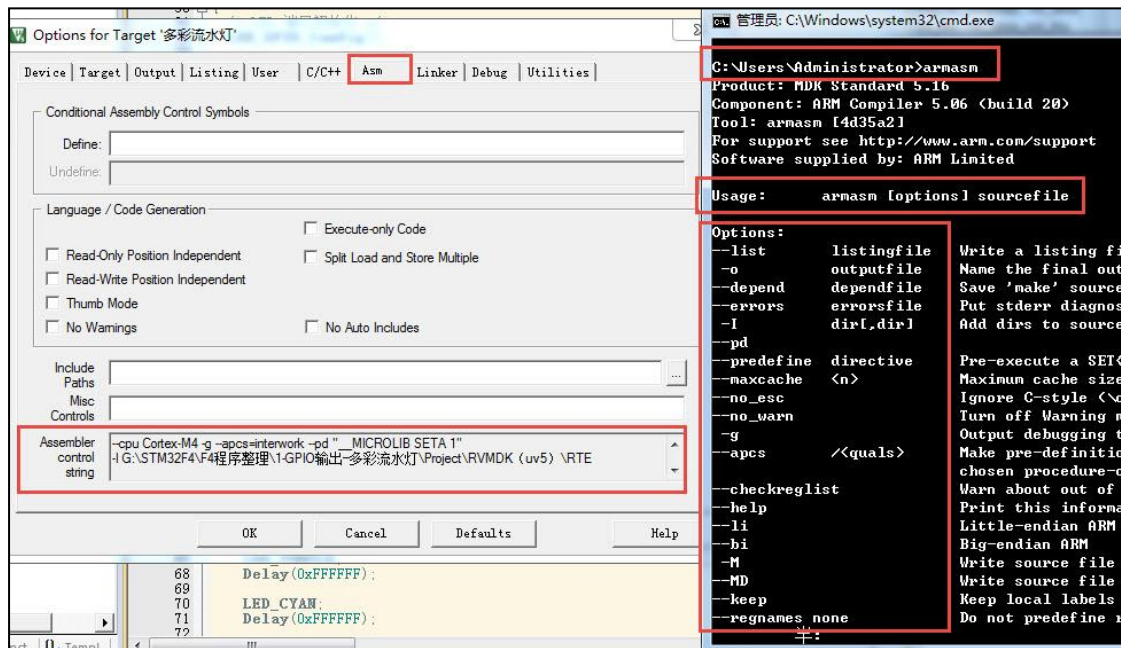


图 48-13 armasm 与 MDK 的编译选项

3. armlink

armlink 是链接器，它把各个 O 文件链接组合在一起生成 ELF 格式的 AXF 文件，AXF 文件是可执行的，下载器把该文件中的指令代码下载到芯片后，该芯片就能运行程序了；利用 armlink 还可以控制程序存储到指定的 ROM 或 RAM 地址。在 MDK 中可在“Option for Target->Linker”页面配置 armlink 选项，见图 48-14。

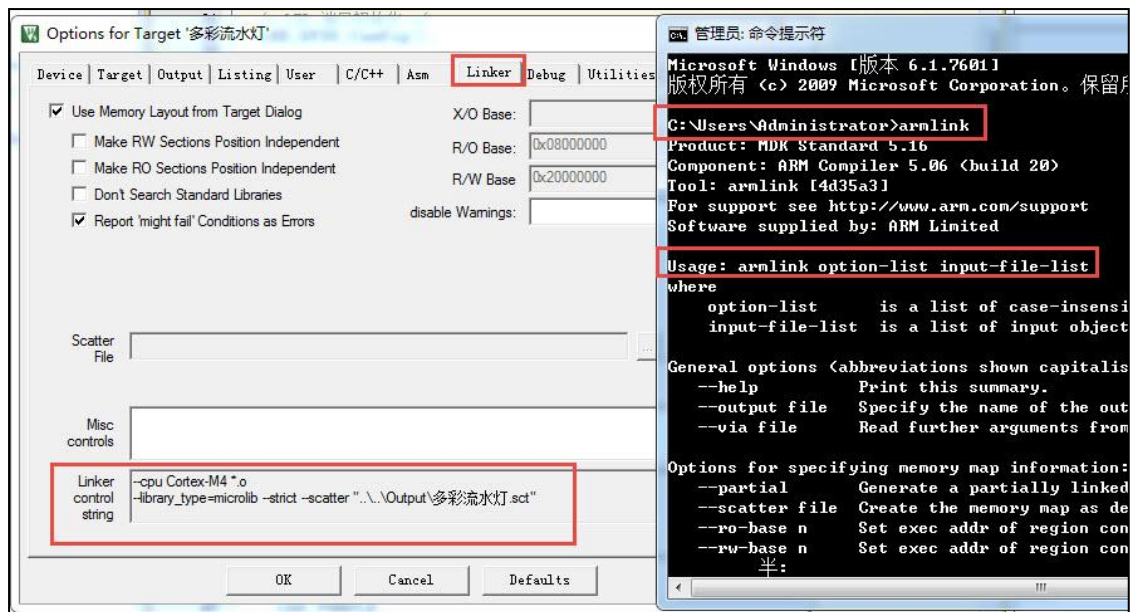


图 48-14 armlink 与 MDK 的配置选项

链接器默认是根据芯片类型的存储器分布来生成程序的，该存储器分布被记录在工程里的 sct 后缀的文件中，有特殊需要的话可自行编辑该文件，改变链接器的链接方式，具体后面我们会详细讲解。

48.3.3 armar、fromelf 及用户指令

armar 工具用于把工程打包成库文件，fromelf 可根据 axf 文件生成 hex、bin 文件，hex 和 bin 文件是大多数下载器支持的下载文件格式。

在 MDK 中，针对 armar 和 fromelf 工具的选项几乎没有，仅集成了生成 HEX 或 Lib 的选项，见图 48-15。

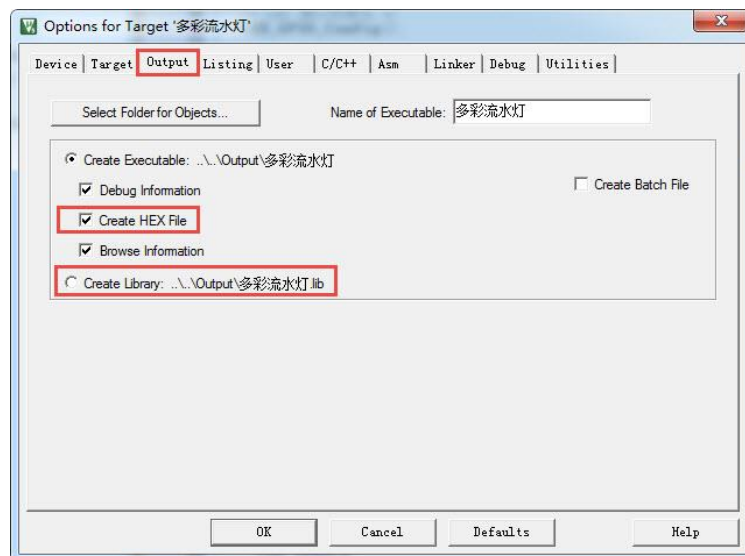


图 48-15 MDK 中，控制 fromelf 生成 hex 及控制 armar 生成 lib 的配置

例如如果我们想利用 fromelf 生成 bin 文件，可以在 MDK 的“Option for Target->User”页中添加调用 fromelf 的指令，见图 48-16。

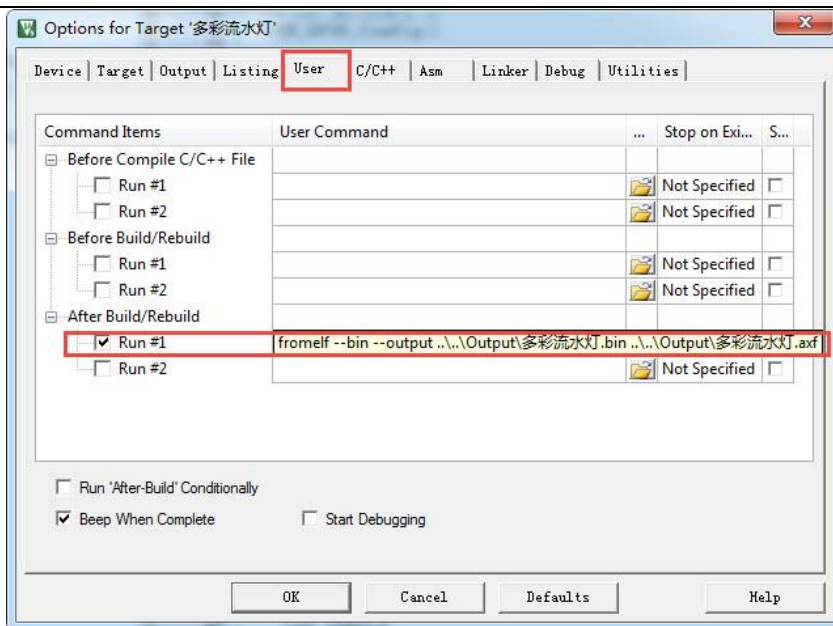


图 48-16 在 MDK 中添加指令

在 User 配置页面中，提供了三种类型的用户指令输入框，在不同组的框输入指令，可控制指令的执行时间，分别是编译前(Before Compile c/c++ file)、构建前(Before Build/Rebuild)及构建后(After Build/Rebuild)执行。这些指令并没有限制必须是 arm 的编译工具链，例如如果您自己编写了 python 脚本，也可以在这里输入用户指令执行该脚本。

图中的生成 bin 文件指令调用了 fromelf 工具，紧跟后面的是工具的选项及输出文件名、输入文件名。由于 fromelf 是根据 axf 文件生成 bin 的，而 axf 文件又是构建(build)工程后才生成，所以我们把该指令放到“After Build/Rebuild”一栏。

48.4 MDK 工程的文件类型

除了上述编译过程生成的文件，MDK 工程中还包含了各种各样的文件，下面我们统一介绍，MDK 工程的常见文件类型见表 48-3。

表 48-3 MDK 常见的文件类型(不分大小写)

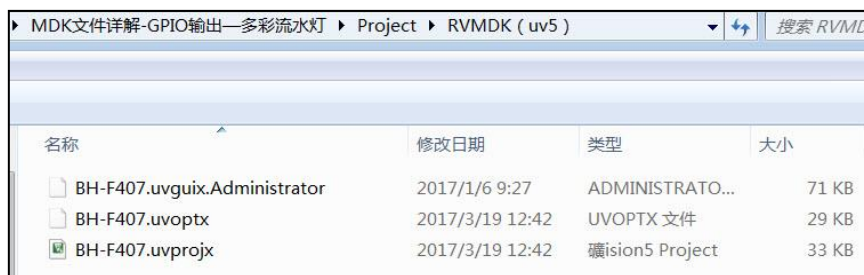
后缀	说明
Project 目录下的工程文件	
.uvguix	MDK5 工程的窗口布局文件，在 MDK4 中.UVGUI 后缀的文件功能相同
.uvprojx	MDK5 的工程文件，它使用了 XML 格式记录了工程结构，双击它可以打开整个工程，在 MDK4 中.UVPROJ 后缀的文件功能相同
.uvoptx	MDK5 的工程配置选项，包含 debugger、trace configuration、breakpoints 以及当前打开的文件，在 MDK4 中.UVOPT 后缀的文件功能相同
*.ini	某些下载器的配置记录文件
源文件	
*.c	C 语言源文件
*.cpp	C++ 语言源文件
*.h	C/C++ 的头文件
*.s	汇编语言的源文件

*.inc	汇编语言的头文件(使用“\$include”来包含)
Output 目录下的文件	
*.lib	库文件
*.dep	整个工程的依赖文件
*.d	描述了对应.o 的依赖的文件
*.crf	交叉引用文件, 包含了浏览信息(定义、引用及标识符)
*.o	可重定位的对象文件(目标文件)
*.bin	二进制格式的映像文件, 是纯粹的 FLASH 映像, 不含任何额外信息
*.hex	Intel Hex 格式的映像文件, 可理解为带存储地址描述格式的 bin 文件
*.elf	由 GCC 编译生成的文件, 功能跟 axf 文件一样, 该文件不可重定位
*.axf	由 ARMCC 编译生成的可执行对象文件, 可用于调试, 该文件不可重定位
*.sct	链接器控制文件(分散加载)
*.scr	链接器产生的分散加载文件
*.lnp	MDK 生成的链接输入文件, 用于调用链接器时的命令输入
*.htm	链接器生成的静态调用图文件
*.build_log.htm	构建工程的日志记录文件
Listing 目录下的文件	
*.lst	C 及汇编编译器产生的列表文件
*.map	链接器生成的列表文件, 包含存储器映像分布
其它	
*.ini	仿真、下载器的脚本文件

这些文件主要分为 MDK 相关文件、源文件以及编译、链接器生成的文件。我们以“多彩流水灯”工程为例讲解各种文件的功能。

48.4.1 uvprojx、uvoptx 及 uvguix 工程文件

在工程的“Project”目录下主要是 MDK 工程相关的文件, 见图 48-17。



名称	修改日期	类型	大小
BH-F407.uvguix.Administrator	2017/1/6 9:27	ADMINISTRATO...	71 KB
BH-F407.uvoptx	2017/3/19 12:42	UVOPTX 文件	29 KB
BH-F407.uvprojx	2017/3/19 12:42	Microvision5 Project	33 KB

图 48-17 Project 目录下的 uvprojx、uvoptx、uvguix 及 ini 文件

1. uvprojx 文件

uvprojx 文件就是我们平时双击打开的工程文件, 它记录了整个工程的结构, 如芯片类型、工程包含了哪些源文件等内容, 见图 48-18。

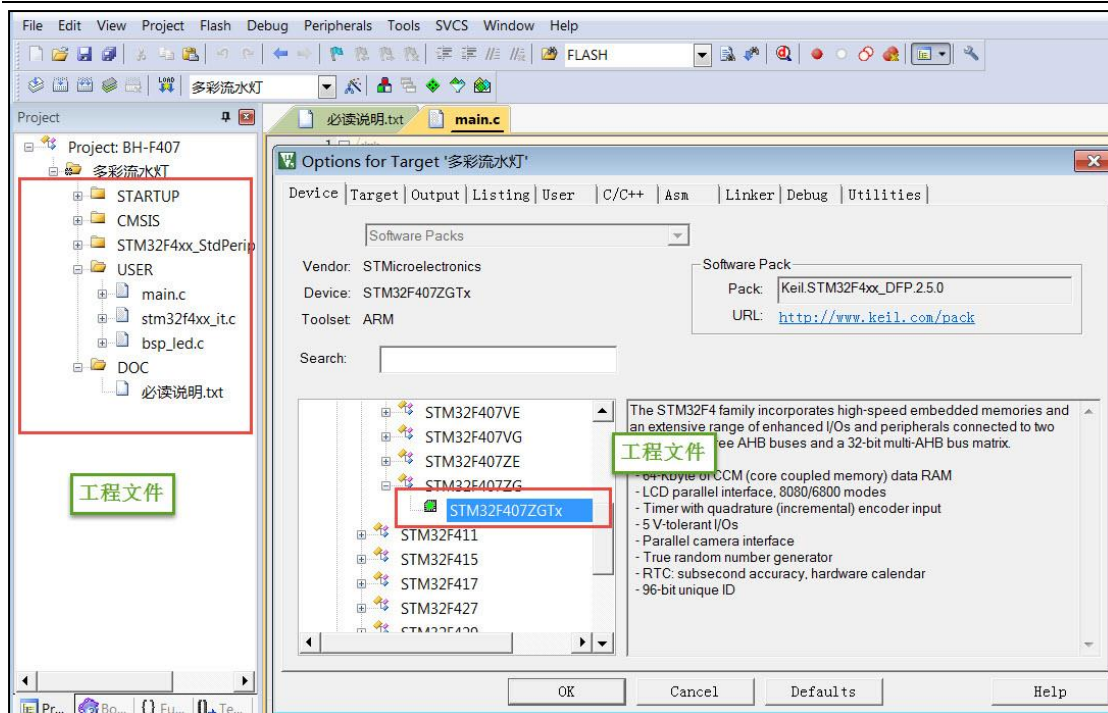


图 48-18 工程包含的文件、芯片类型等内容

2. uvoptx 文件

uvoptx 文件记录了工程的配置选项，如下载器的类型、变量跟踪配置、断点位置以及当前已打开的文件等等，见图 48-19。

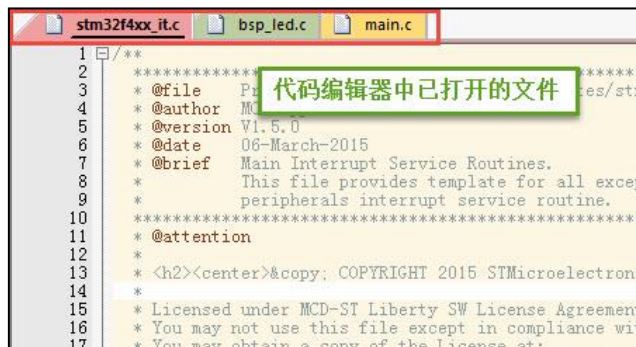


图 48-19 代码编辑器中已打开的文件

3. uvguix 文件

uvguix 文件记录了 MDK 软件的 GUI 布局，如代码编辑区窗口的大小、编译输出提示窗口的位置等等。

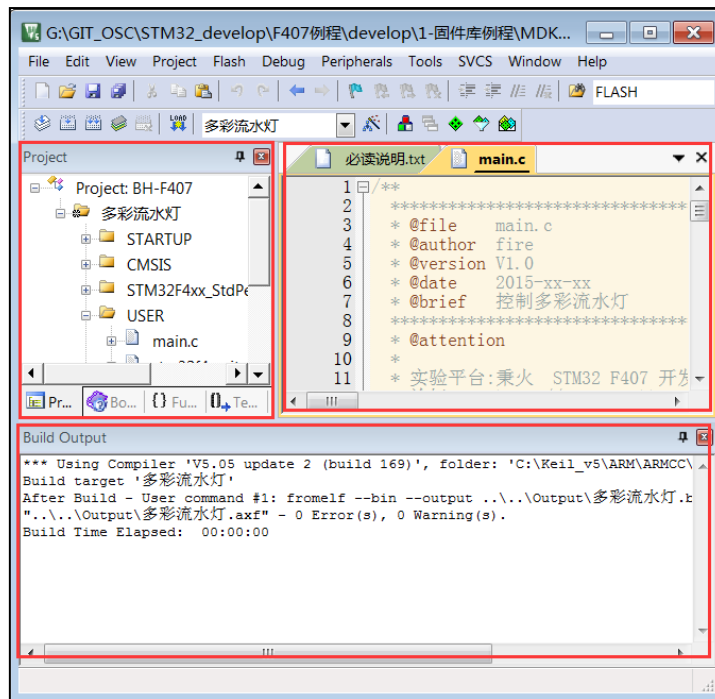


图 48-20 记录 MDK 工作环境中各个窗口的大小

uvprojx、uvoptx 及 uvguix 都是使用 XML 格式记录的文件，若使用记事本打开可以看到 XML 代码，见图 48-17。而当使用 MDK 软件打开时，它根据这些文件的 XML 记录加载工程的各种参数，使得我们每次重新打开工程时，都能恢复上一次的工作环境。

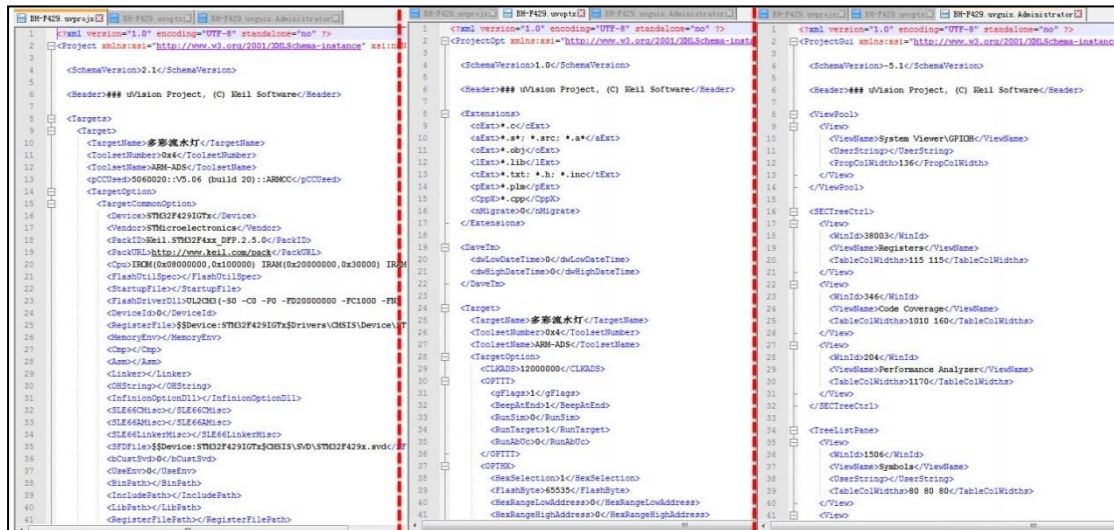


图 48-21 使用记事本打开 uvprojx、uvoptx 及 uvguix 文件可看到 XML 格式的记录

这些工程参数都是当 MDK 正常退出时才会被写入保存，所以若 MDK 错误退出时(如使用 Windows 的任务管理器强制关闭)，工程配置参数的最新更改是会被记录的，重新打开工程时要再次配置。根据这几个文件的记录类型，可以知道 uvprojx 文件是最重要的，删掉我们就无法再正常打开工程了，而 uvoptx 及 uvguix 文件并不是必须的，可以删除，重新使用 MDK 打开 uvprojx 工程文件后，会以默认参数重新创建 uvoptx 及 uvguix 文件。(所以当使用 Git/SVN 等代码管理的时候，往往只保留 uvprojx 文件)

48.4.2 源文件

源文件是工程中我们最熟悉的内容了，它们就是我们编写的各种源代码，MDK 支持 c、cpp、h、s、inc 类型的源代码文件，其中 c、cpp 分别是 c/c++ 语言的源代码，h 是它们的头文件，s 是汇编文件，inc 是汇编文件的头文件，可使用“\$include”语法包含。编译器根据工程中的源文件最终生成机器码。

48.4.3 Output 目录下生成的文件

点击 MDK 中的编译按钮，它会根据工程的配置及工程中的源文件输出各种对象和列表文件，在工程的“Options for Target->Output->Select Folder for Objects”和“Options for Target->Listing->Select Folder for Listings”选项配置它们的输出路径，见图 48-22 和图 48-23。

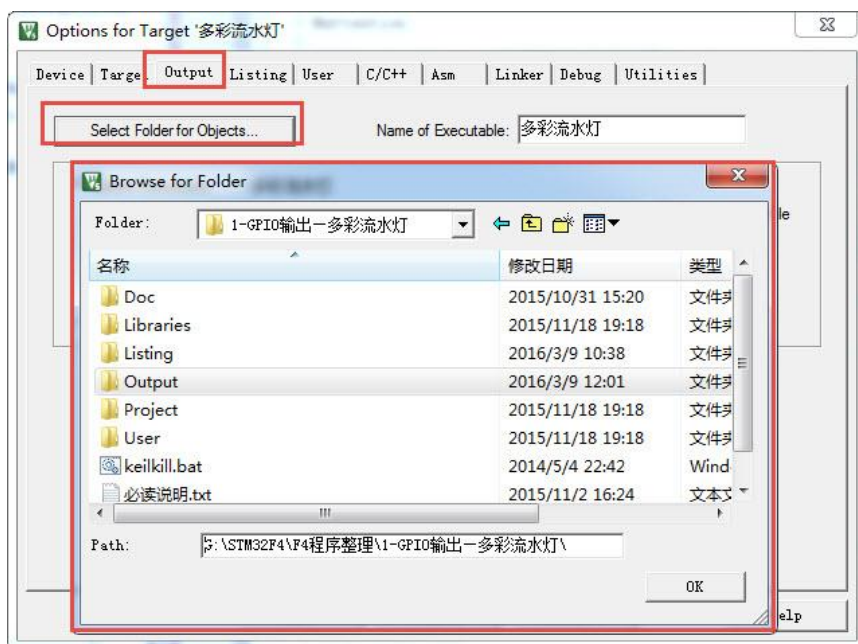


图 48-22 设置 Output 输出路径

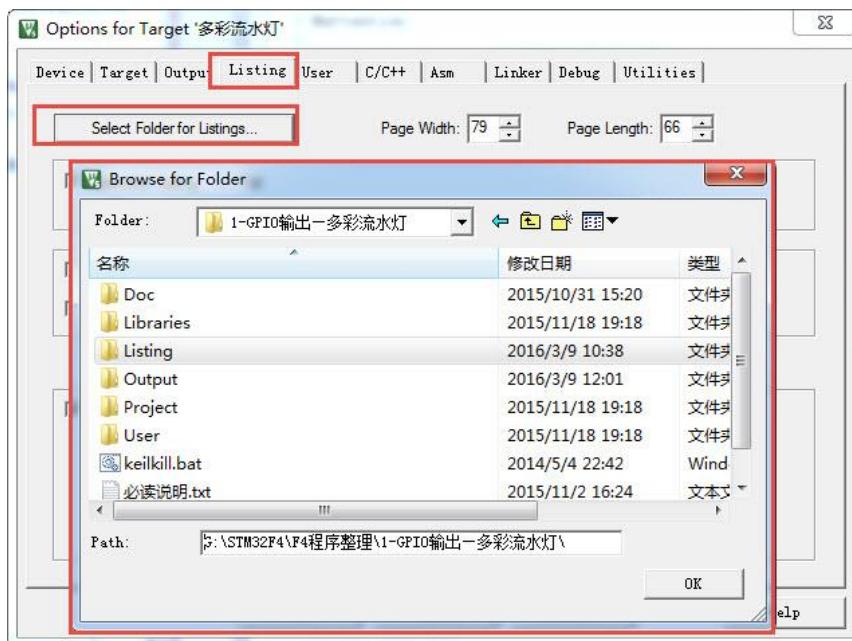


图 48-23 设置 Listing 输出路径

编译后 Output 和 Listing 目录下生成的文件见图 48-24。



图 48-24 编译后 Output 及 Listing 文件夹中的内容

接下来我们讲解 Output 路径下的文件。

1. lib 库文件

在某些场合下我们希望提供给第三方一个可用的代码库，但不希望对方看到源码，这个时候我们就可以把工程生成 lib 文件(Library file)提供给对方，在 MDK 中可配置

“Options for Target->Create Library” 选项把工程编译成库文件，见图 48-25。

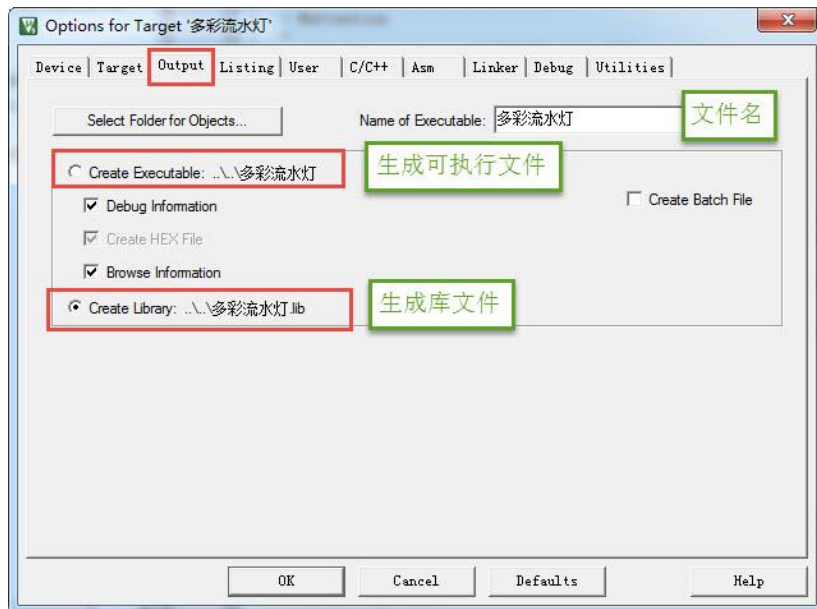


图 48-25 生成库文件或可执行文件

工程中生成可执行文件或库文件只能二选一，默认编译是生成可执行文件的，可执行文件即我们下载到芯片上直接运行的机器码。

得到生成的*.lib 文件后，可把它像 C 文件一样添加到其它工程中，并在该工程调用 lib 提供的函数接口，除了不能看到*.lib 文件的源码，在应用方面它跟 C 源文件没有区别。

2. dep、d 依赖文件

*.dep 和 *.d 文件(Dependency file)记录的是工程或其它文件的依赖，主要记录了引用的头文件路径，其中*.dep 是整个工程的依赖，它以工程名命名，而*.d 是单个源文件的依赖，它们以对应的源文件名命名。这些记录使用文本格式存储，我们可直接使用记事本打开，见图 48-26 和图 48-27。



图 48-26 工程的 dep 文件内容

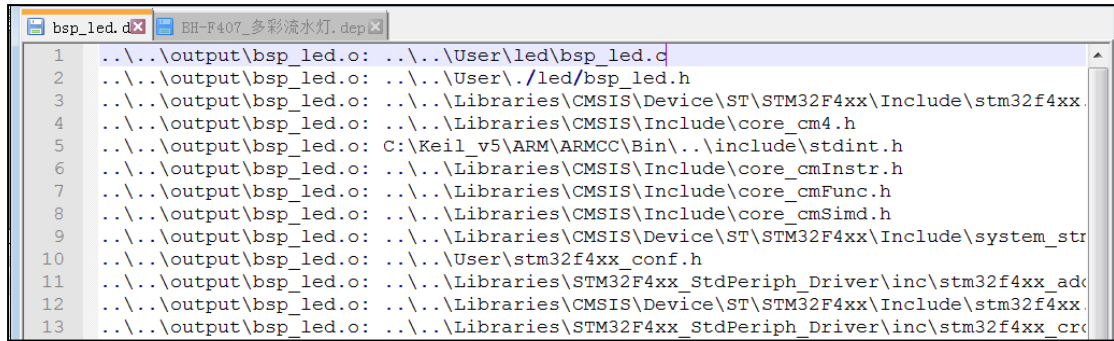


图 48-27 bsp_led.d 文件的内容

3. crf 交叉引用文件

*.crf 是交叉引用文件(Cross-Reference file)，它主要包含了浏览信息(browse information)，即源代码中的宏定义、变量及函数的定义和声明的位置。

我们在代码编辑器中点击“Go To Definition Of 'xxxx'”可实现浏览跳转，见图 48-28，跳转的时候，MDK 就是通过*.crf 文件查找出跳转位置的。

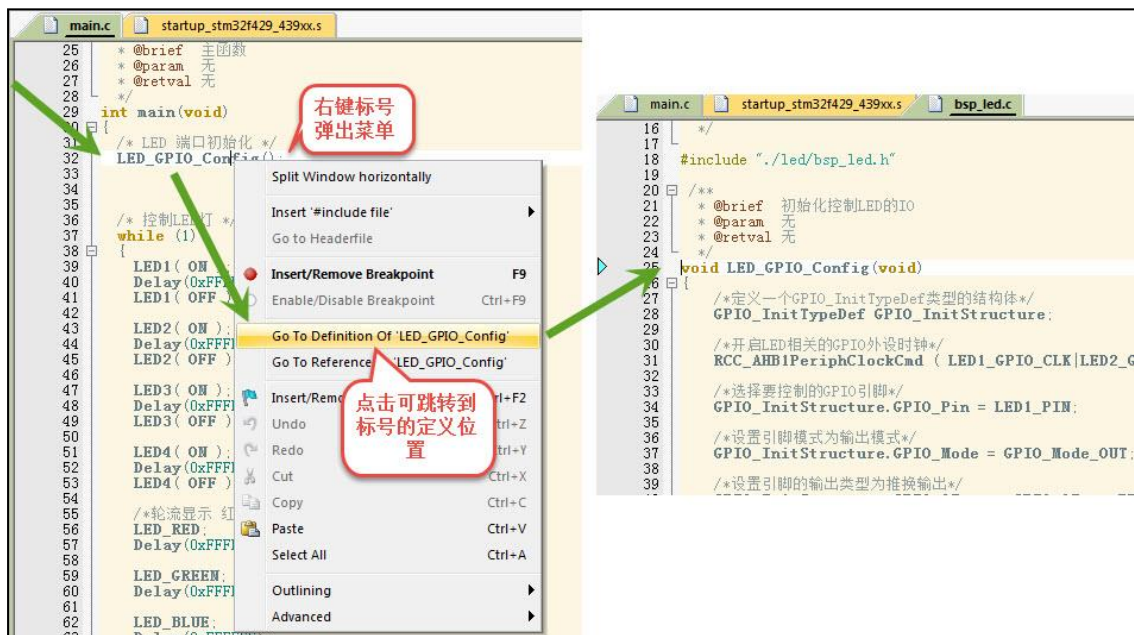


图 48-28 浏览信息

通过配置 MDK 中的“Option for Target->Output->Browse Information”选项可以设置编译时是否生成浏览信息，见图 48-29。只有勾选该选项并编译后，才能实现上面的浏览跳转功能。

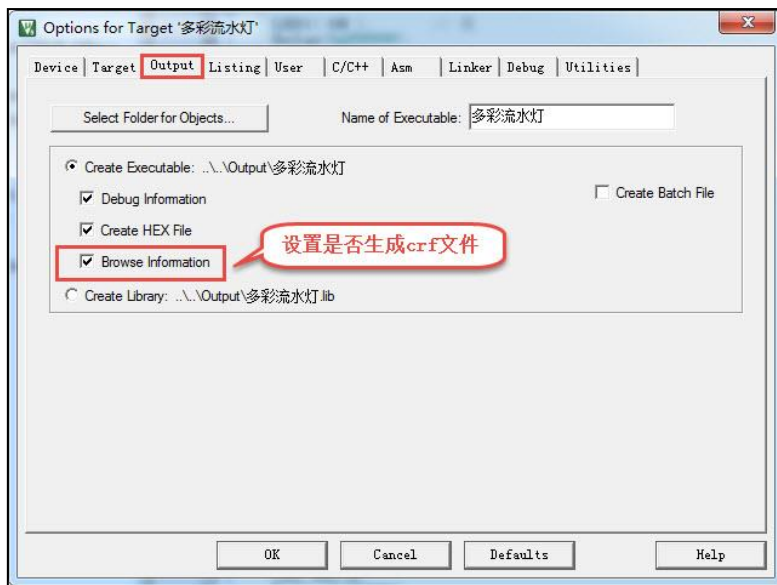


图 48-29 在 Options for Target 中设置是否生成浏览信息

*.crf 文件使用了特定的格式表示，直接用文本编辑器打开会看到大部分乱码，见图 48-30，我们不作深入研究。

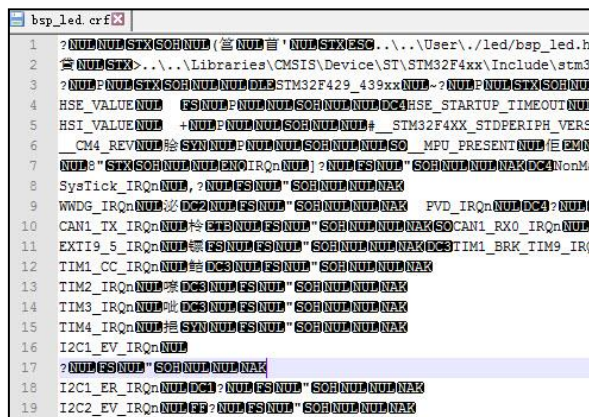


图 48-30 crf 文件内容

4. o、axf 及 elf 文件

.o、.elf、*.axf、*.bin 及*.hex 文件都存储了编译器根据源代码生成的机器码，根据应用场合的不同，它们又有所区别。

ELF 文件说明

.o、.elf、*.axf 以及前面提到的 lib 文件都是属于目标文件，它们都是使用 ELF 格式来存储的，关于 ELF 格式的详细内容请参考配套资料里的《ELF 文件格式》文档了解，它讲解的是 Linux 下的 ELF 格式，与 MDK 使用的格式有小区别，但大致相同。在本教程中，仅讲解 ELF 文件的核心概念。

ELF 是 Executable and Linking Format 的缩写，译为可执行链接格式，该格式用于记录目标文件的内容。在 Linux 及 Windows 系统下都有使用该格式的文件(或类似格式)用于记录应用程序的内容，告诉操作系统如何链接、加载及执行该应用程序。

目标文件主要有如下三种类型：

- (1) 可重定位的文件(Relocatable File)，包含基础代码和数据，但它的代码及数据都没有指定绝对地址，因此它适合于与其他目标文件链接来创建可执行文件或者共享目标文件。这种文件一般由编译器根据源代码生成。

例如 MDK 的 armcc 和 armasm 生成的*.o 文件就是这一类，另外还有 Linux 的*.o 文件，Windows 的 *.obj 文件。

- (2) 可执行文件(Executable File)，它包含适合于执行的程序，它内部组织的代码数据都有固定的地址(或相对于基地址的偏移)，系统可根据这些地址信息把程序加载到内存执行。这种文件一般由链接器根据可重定位文件链接而成，它主要是组织各个可重定位文件，给它们的代码及数据一一打上地址标号，固定其在程序内部的位置，链接后，程序内部各种代码及数据段不可再重定位(即不能再参与链接器的链接)。

例如 MDK 的 armlink 生成的*.elf 及*.axf 文件，(使用 gcc 编译工具可生成 *.elf 文件，用 armlink 生成的是*.axf 文件，*.axf 文件在*.elf 之外，增加了调试使用的信息，其余区别不大，后面我们仅讲解*.axf 文件)，另外还有 Linux 的 /bin/bash 文件，Windows 的*.exe 文件。

- (3) 共享目标文件(Shared Object File)，它的定义比较难理解，我们直接举例，MDK 生成的*.lib 文件就属于共享目标文件，它可以继续参与链接，加入到可执行文件之中。另外，Linux 的.so，如/lib/ glibc-2.5.so，Windows 的 DLL 都属于这一类。

o 文件与 axf 文件的关系

根据上面的分类，我们了解到，*.axf 文件是由多个*.o 文件链接而成的，而*.o 文件由相应的源文件编译而成，一个源文件对应一个*.o 文件。它们的关系见图 48-31。

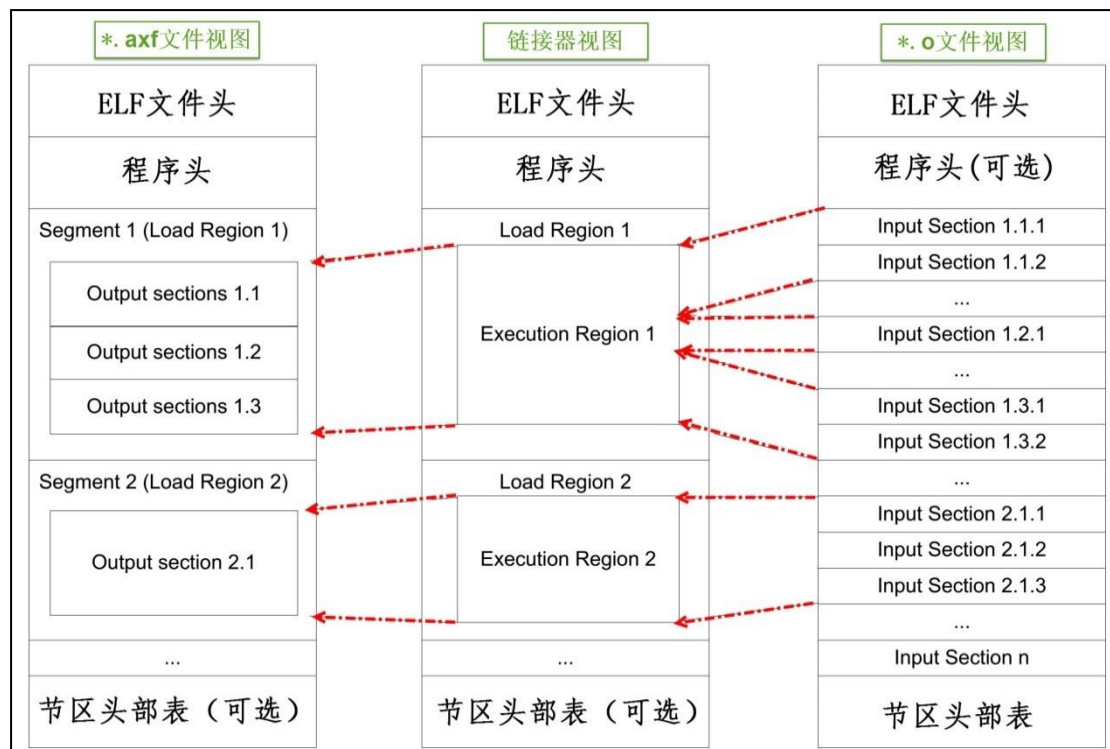


图 48-31*.axf 文件与*.o 文件的关系

图中的中间代表的是 armlink 链接器，在它的右侧是输入链接器的*.o 文件，左侧是它输出的*.axf 文件。

可以看到，由于都使用 ELF 文件格式，*.o 与*.axf 文件的结构是类似的，它们包含 ELF 文件头、程序头、节区(section)以及节区头部表。各个部分的功能说明如下：

- ❑ ELF 文件头用来描述整个文件的组织，例如数据的大小端格式，程序头、节区头在文件中的位置等。
- ❑ 程序头告诉系统如何加载程序，例如程序主体存储在本文件的哪个位置，程序的大小，程序要加载到内存什么地址等等。MDK 的可重定位文件*.o 不包含这部分内容，因为它还不是可执行文件，而 armlink 输出的*.axf 文件就包含该内容了。
- ❑ 节区是*.o 文件的独立数据区域，它包含提供给链接视图使用的大量信息，如指令(Code)、数据(RO、RW、ZI-data)、符号表(函数、变量名等)、重定位信息等，例如每个由 C 语言定义的函数在*.o 文件中都会有一个独立的节区；
- ❑ 存储在最后的节区头则包含了本文件节区的信息，如节区名称、大小等等。

总的来说，链接器把各个*.o 文件的节区归类、排列，根据目标器件的情况编排地址生成输出，汇总到*.axf 文件。例如，见图 48-32，“多彩流水灯”工程中在“bsp_led.c”文件中有一个 LED_GPIO_Config 函数，而它内部调用了“stm32f4xx_gpio.c”的 GPIO_Init 函数，经过 armcc 编译后，LED_GPIO_Config 及 GPIO_Init 函数都成了指令代码，分别存储在 bsp_led.o 及 stm32f4xx_gpio.o 文件中，这些指令在*.o 文件都没有指定地址，仅包含了内容、大小以及调用的链接信息，而经过链接器后，链接器给它们都分配了特定的地址，并且把地址根据调用指向链接起来。

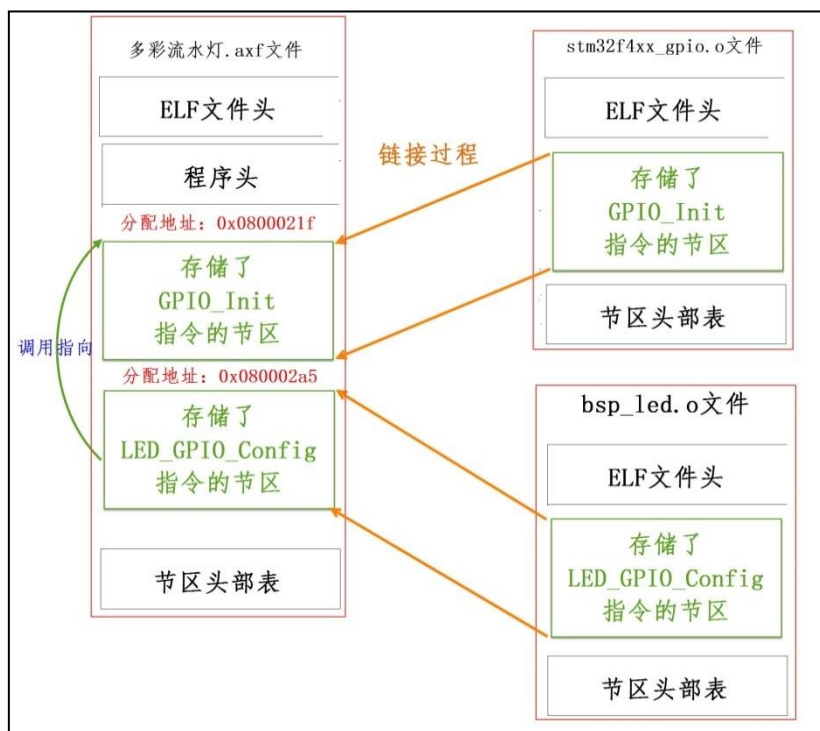


图 48-32 具体的链接过程

ELF 文件头

接下来我们看看具体文件的内容，使用 fromelf 文件可以查看*.o、*.axf 及*.lib 文件的 ELF 信息。

使用命令行，切换到文件所在的目录，输入“fromelf -text -v bsp_led.o”命令，可控制输出 bsp_led.o 的详细信息，见图 48-33。利用“-c、-z”等选项还可输出反汇编指令文件、代码及数据文件等信息，请亲手尝试一下。

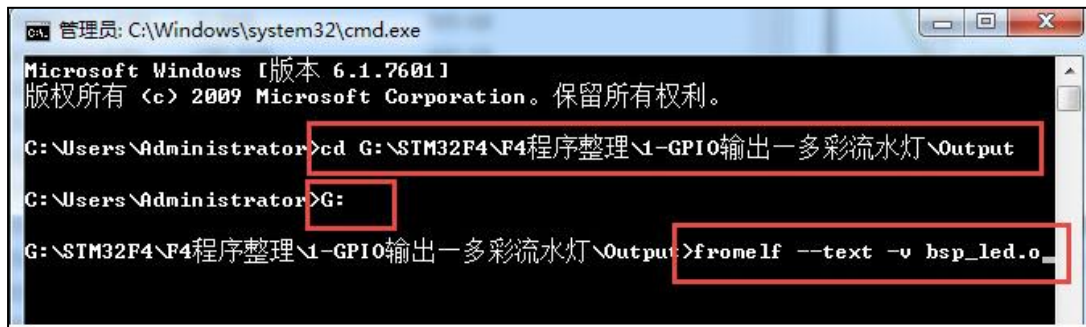


图 48-33 使用 fromelf 查看 o 文件信息

为了便于阅读，我已使用 fromelf 指令生成了“多彩流水灯.axf”、“bsp_led”及“多彩流水灯.lib”的 ELF 信息，并已把这些信息保存在独立的文件中，在配套资料的“elf 信息输出”文件夹下可查看，见表 48-4。

表 48-4 配套资料里使用 fromelf 生成的文件

fromelf 选项	可查看的信息	生成到配套资料里相应的文件
-v	详细信息	bsp_led_o_elfInfo_v.txt/多彩流水灯_axf_elfInfo_v.txt
-a	数据的地址	bsp_led_o_elfInfo_a.txt/多彩流水灯_axf_elfInfo_a.txt
-c	反汇编代码	bsp_led_o_elfInfo_c.txt/多彩流水灯_axf_elfInfo_c.txt
-d	data section 的内容	bsp_led_o_elfInfo_d.txt/多彩流水灯_axf_elfInfo_d.txt
-e	异常表	bsp_led_o_elfInfo_e.txt/多彩流水灯_axf_elfInfo_e.txt
-g	调试表	bsp_led_o_elfInfo_g.txt/多彩流水灯_axf_elfInfo_g.txt
-r	重定位信息	bsp_led_o_elfInfo_r.txt/多彩流水灯_axf_elfInfo_r.txt
-s	符号表	bsp_led_o_elfInfo_s.txt/多彩流水灯_axf_elfInfo_s.txt
-t	字符串表	bsp_led_o_elfInfo_t.txt/多彩流水灯_axf_elfInfo_t.txt
-y	动态段内容	bsp_led_o_elfInfo_y.txt/多彩流水灯_axf_elfInfo_y.txt
-z	代码及数据的大小信息	bsp_led_o_elfInfo_z.txt/多彩流水灯_axf_elfInfo_z.txt

直接打开“elf 信息输出”目录下的 bsp_led_o_elfInfo_v.txt 文件，可看到代码清单 48-1 中的内容。

代码清单 48-1 bsp_led.o 文件的 ELF 文件头(可到“bsp_led_o_elfInfo_v.txt”文件查看)

```
1 =====
2
3 ** ELF Header Information
4
5 File Name:
6 .\bsp_led.o //bsp_led.o 文件
7
8 Machine class: ELFCLASS32 (32-bit) //32 位机
9 Data encoding: ELFDATA2LSB (Little endian) //小端格式
```

```
10 Header version: EV_CURRENT (Current version)
11 Operating System ABI: none
12 ABI Version: 0
13 File Type: ET_REL (Relocatable object) (1) //可重定位文件类型
14 Machine: EM_ARM (ARM)
15
16 Entry offset (in SHF ENTRYSECT section): 0x00000000
17 Flags: None (0x05000000)
18
19 ARM ELF revision: 5 (ABI version 2)
20
21 Built with
22 Component: ARM Compiler 5.05 update 2 (build 169) Tool: armasm [4d0f2f]
23 Component: ARM Compiler 5.05 update 2 (build 169) Tool: armlink [4d0f33]
24
25 Header size: 52 bytes (0x34)
26 Program header entry size: 0 bytes (0x0) //程序头大小
27 Section header entry size: 40 bytes (0x28)
28
29 Program header entries: 0
30 Section header entries: 231
31
32 Program header offset: 0 (0x00000000) //程序头在文件中的位置(没有程序头)
33 Section header offset: 478264 (0x00074c38) //节区头在文件中的位置
34
35 Section header string table index: 228
36
37 =====
```

在上述代码中已加入了部分注释，解释了相应项的意义，值得一提的是在这个*.o文件中，它的 ELF 文件头中告诉我们它的程序头(Program header)大小为“0 bytes”，且程序头所在的文件位置偏移也为“0”，这说明它是没有程序头的。

程序头

接下来打开“多彩流水灯_axf_elfInfo_v.txt”文件，查看工程的*.axf 文件的详细信息，见代码清单 48-2。

代码清单 48-2 *.axf 文件中的 elf 文件头及程序头(可到“多彩流水灯_axf_elfInfo_v.txt”文件查看)

```
1 =====
2
3 ** ELF Header Information
4
5 File Name:
6 .\多彩流水灯.axf //多彩流水灯.axf 文件
7
8 Machine class: ELFCLASS32 (32-bit) //32 位机
9 Data encoding: ELFDATA2LSB (Little endian) //小端格式
10 Header version: EV_CURRENT (Current version)
11 Operating System ABI: none
12 ABI Version: 0
13 File Type: ET_EXEC (Executable) (2) //可执行文件类型
14 Machine: EM_ARM (ARM)
15
16 Image Entry point: 0x08000189
17 Flags: EF_ARM_HASENTRY (0x05000002)
18
19 ARM ELF revision: 5 (ABI version 2)
20
21 Conforms to Soft float procedure-call standard
22
23 Built with
```

```

24 Component: ARM Compiler 5.05 update 2 (build 169) Tool: armasm [4d0f2f]
25 Component: ARM Compiler 5.05 update 2 (build 169) Tool: armlink [4d0f33]
26
27 Header size: 52 bytes (0x34)
28 Program header entry size: 32 bytes (0x20)
29 Section header entry size: 40 bytes (0x28)
30
31 Program header entries: 1
32 Section header entries: 15
33
34 Program header offset: 334956 (0x00051c6c) //程序头在文件中的位置
35 Section header offset: 334988 (0x00051c8c) //节区头在文件中的位置
36
37 Section header string table index: 14
38
39 =====
40
41 ** Program header #0
42
43 Type : PT_LOAD (1) //表示这是可加载的内容
44 File Offset : 52 (0x34) //在文件中的偏移
45 Virtual Addr : 0x08000000 //虚拟地址(此处等于物理地址)
46 Physical Addr : 0x08000000 //物理地址
47 Size in file : 1316 bytes (0x524) //程序在文件中占据的大小
48 Size in memory: 2340 bytes (0x924) //若程序加载到内存, 占据的内存空间
49 Flags : PF_X + PF_W + PF_R + PF_ARM_ENTRY (0x80000007)
50 Alignment : 8 //地址对齐
51
52
53 =====

```

对比之下, 可发现*.axf 文件的 ELF 文件头对程序头的大小说明为非 0 值, 且给出了它在文件的偏移地址, 在输出信息之中, 包含了程序头的详细信息。可看到, 程序头的

“Physical Addr” 描述了本程序要加载到的内存地址 “0x0800 0000”, 正好是 STM32 内部 FLASH 的首地址; “size in file” 描述了本程序占据的空间大小为 “1316 bytes”, 它正是程序烧录到 FLASH 中需要占据的空间。

节区头

在 ELF 的原文件中, 紧接着程序头的一般是节区的主体信息, 在节区主体信息之后是描述节区主体信息的节区头, 我们先来看看节区头中的信息了解概况。通过对比*.o 文件及*.axf 文件的节区头部信息, 可以清楚地看出这两种文件的区别, 见代码清单 48-3。

代码清单 48-3 *.o 文件的节区信息(“bsp_led_o_elfInfo_v.txt” 文件)

```

1 =====
2 ** Section #4
3
4 Name : i.LED_GPIO_Config //节区名
5
6
7 //此节区包含程序定义的信息, 其格式和含义都由程序来解释。
8 Type : SHT_PROGBITS (0x00000001)
9
10
11 //此节区在进程执行过程中占用内存。 节区包含可执行的机器指令。
12 Flags : SHF_ALLOC + SHF_EXECINSTR (0x00000006)
13
14 Addr : 0x00000000 //地址
15 File Offset : 68 (0x44) //在文件中的偏移
16 Size : 88 bytes (0x58) //大小
17 Link : SHN_UNDEF
18 Info : 0
19
20 Alignment : 4 //字节对齐

```

```
21 Entry Size : 0
```

```
22 =====
```

这个节区的名称为 LED_GPIO_Config，它正好是我们在 bsp_led.c 文件中定义的函数名。

注意：编译时要勾选“Options for Target ->C/C++ -> One ELF Section per Function”中的选项，生成的*.o 文件内部的代码区域才会与 C 文件中定义的函数名一致，否则它会把多个函数合成一个代码段，名字会不同。见图 48-34。

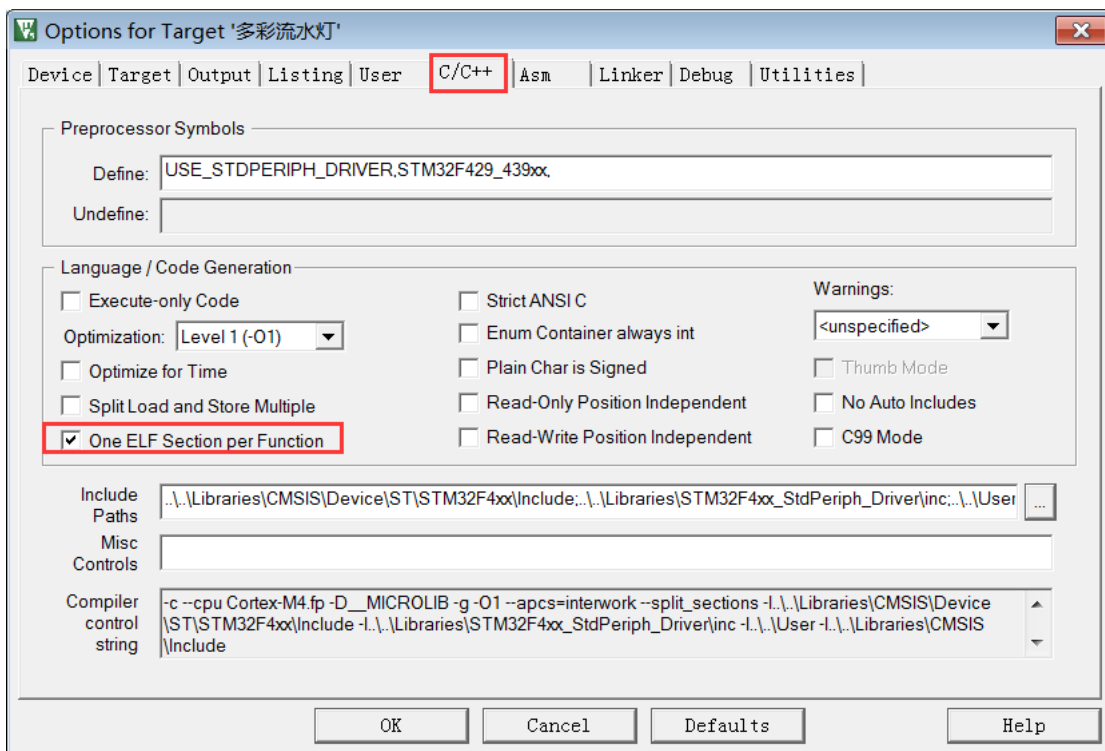


图 48-34 勾选 One ELF Section per Function

这个节区头描述的是该函数被编译后的节区信息，其中包含了节区的类型(指令类型)、节区应存储到的地址(0x00000000)、它主体信息在文件位置中的偏移(68)以及节区的大小(88 bytes)。

由于*.o 文件是可重定位文件，所以它的地址并没有被分配，是 0x00000000（假如文件中还有其它函数，该函数生成的节区中，对应的地址描述也都是 0）。当链接器链接时，根据这个节区头信息，在文件中找到它的主体内容，并根据它的类型，把它加入到主程序中，并分配实际地址，链接后生成的*.axf 文件，我们再来看看它的内容，见代码清单 48-4。代码清单 48-4 *.axf 文件的节区信息(“多彩流水灯_axf_elfInfo_v.txt”文件)

```
1 =====
2 ** Section #1
3
4 Name      : ER_IROM1                      //节区名
5
6 //此节区包含程序定义的信息，其格式和含义都由程序来解释。
7 Type      : SHT_PROGBITS (0x00000001)
8
9 //此节区在进程执行过程中占用内存。 节区包含可执行的机器指令
10 Flags     : SHF_ALLOC + SHF_EXECINSTR (0x00000006)
11 Addr      : 0x08000000                    //地址
```



```

12      File Offset : 52 (0x34)
13      Size       : 1316 bytes (0x524)           //大小
14      Link       : SHN_UNDEF
15      Info       : 0
16      Alignment  : 4
17      Entry Size : 0
18
19 =====
20 ** Section #2
21
22      Name       : RW_IRAM1                     //节区名
23
24      //包含将出现在程序的内存映像中的为初始
25      //化数据。 根据定义， 当程序开始执行， 系统
26      //将把这些数据初始化为 0。
27      Type       : SHT_NOBITS (0x00000008)
28
29      //此节区在进程执行过程中占用内存。 节区包含进程执行过程中将可写的。
30      Flags      : SHF_ALLOC + SHF_WRITE (0x00000003)
31      Addr       : 0x20000000                     //地址
32      File Offset : 1368 (0x558)
33      Size       : 1024 bytes (0x400)           //大小
34      Link       : SHN_UNDEF
35      Info       : 0
36      Alignment  : 8
37      Entry Size : 0
38 =====

```

在*.axf 文件中，主要包含了两个节区，一个名为 ER_IROM1，一个名为 RW_IRAM1，这些节区头信息中除了具有*.o 文件中节区头描述的节区类型、文件位置偏移、大小之外，更重要的是它们都有具体的地址描述，其中 ER_IROM1 的地址为 0x08000000，而 RW_IRAM1 的地址为 0x20000000，它们正好是内部 FLASH 及 SRAM 的首地址，对应节区的大小就是程序需要占用 FLASH 及 SRAM 空间的实际大小。

也就是说，经过链接器后，它生成的*.axf 文件已经汇总了其它*.o 文件的所有内容，生成的 ER_IROM1 节区内容可直接写入到 STM32 内部 FLASH 的具体位置。例如，前面 *.o 文件中的 i.LED_GPIO_Config 节区已经被加入到*.axf 文件的 ER_IROM1 节区的某地址。

节区主体及反汇编代码

使用 fromelf 的 -c 选项可以查看部分节区的主体信息，对于指令节区，可根据其内容查看相应的反汇编代码，打开“bsp_led_o_elfInfo_c.txt”文件可查看这些信息，见代码清单 48-5。

代码清单 48-5 *.o 文件的 LED_GPIO_Config 节区及反汇编代码(bsp_led_o_elfInfo_c.txt 文件)

```

1 ** Section #4 'i.LED_GPIO_Config' (SHT_PROGBITS) [SHF_ALLOC + SHF_EXECINSTR]
2      Size       : 88 bytes (alignment 4)
3      Address: 0x00000000
4
5      $t
6      i.LED_GPIO_Config
7      LED_GPIO_Config
8      // 地址          内容          (ASCII 码)      内容对应的代码
9      //              (无意义)
10      0x00000000:      e92d41fc      -..A      PUSH      {r2-r8,lr}
11      0x00000004:      2101          .!          MOVS      r1,#1
12      0x00000006:      2020          MOVS      r0,#0x20
13 0x00000008:      f7fffffe      ....      BL          RCC_AHB1PeriphClockCmd
14      0x0000000c:      2440          @$          MOVS      r4,#0x40
15      0x0000000e:      9400          ..          STR          r4,[sp,#0]

```

```

16      0x00000010:    2001      .      MOVS      r0,#1
17      0x00000012:    f88d0004    ....    STRB      r0,[sp,#4]
18      0x00000016:    2100      .!     MOVS      r1,#0
19      0x00000018:    f88d1006    ....    STRB      r1,[sp,#6]
20      0x0000001c:    f88d0007    ....    STRB      r0,[sp,#7]
21      0x00000022:    f88d0005    ....    STRB      r0,[sp,#5]
22      0x00000024:    4f0b      .O     LDR      r7,[pc,#44] ; ;
23      0x00000026:    4669      iF     MOV      r1,sp
24      0x00000028:    4638      8F     MOV      r0,r7
25      0x0000002a:    f7ffffffe    ....    BL        GPIO_Init
26      0x0000002e:    2680      .&     MOVS      r6,#0x80
27      /*....以下省略*/

```

可看到，由于这是*.o 文件，它的节区地址还是没有分配的，基地址为 0x00000000，接着在 LED_GPIO_Config 标号之后，列出了一个表，表中包含了地址偏移、相应地址中的内容以及根据内容反汇编得到的指令。细看汇编指令，还可看到它包含了跳转到 RCC_AHB1PeriphClockCmd 及 GPIO_Init 标号的语句，而且这两个跳转语句原来的内容都是“f7ffffffe”，这是因为还*.o 文件中并没有 RCC_AHB1PeriphClockCmd 及 GPIO_Init 标号的具体地址索引，在*.axf 文件中，这是不一样的。

接下来我们打开“多彩流水灯_axf_elfInfo.c.txt”文件，查看*.axf 文件中，ER_IROM1 节区中对应 LED_GPIO_Config 的内容，见代码清单 48-6。

代码清单 48-6*.axf 文件的 LED_GPIO_Config 反汇编代码(多彩流水灯_axf_elfInfo.c.txt 文件)

```

1  LED_GPIO_Config
2      0x08000280:    e92d41fc    -..A    PUSH      {r2-r8,lr}
3      0x08000284:    2101      .!     MOVS      r1,#1
4      0x08000286:    2020      .      MOVS      r0,#0x20
5      0x08000288:    f000f82a    ...*    BL        RCC_AHB1PeriphClockCmd ; 0x80002e0
6      0x0800028c:    2440      @$     MOVS      r4,#0x40
7      0x0800028e:    9400      ..     STR      r4,[sp,#0]
8      0x08000290:    2001      .      MOVS      r0,#1
9      0x08000292:    f88d0004    ....    STRB      r0,[sp,#4]
10     0x08000296:    2100      .!     MOVS      r1,#0
11     0x08000298:    f88d1006    ....    STRB      r1,[sp,#6]
12     0x0800029c:    f88d0007    ....    STRB      r0,[sp,#7]
13     0x080002a0:    f88d1005    ....    STRB      r1,[sp,#5]
14     0x080002a4:    4f0b      .O     LDR      r7,[pc,#44] ; [0x80002d4] = 0x40021400
15     0x080002a6:    4669      iF     MOV      r1,sp
16     0x080002a8:    4638      8F     MOV      r0,r7
17     0x080002aa:    f7ffffa6    ....    BL        GPIO_Init ; 0x80001fa
18     0x080002ae:    2680      .&     MOVS      r6,#0x80
19     /*....以下省略*/

```

可看到，除了基地址以及跳转地址不同之外，LED_GPIO_Config 中的内容跟*.o 文件中的一样。另外，由于*.o 是独立的文件，而*.axf 是整个工程汇总的文件，所以在*.axf 中包含了所有调用到*.o 文件节区的内容。例如，在“bsp_led_o_elfInfo.c.txt”(bsp_led.o 文件的反汇编信息)中不包含 RCC_AHB1PeriphClockCmd 及 GPIO_Init 的内容，而在“多彩流水灯_axf_elfInfo.c.txt”(多彩流水灯.axf 文件的反汇编信息)中则可找到它们的具体信息，且它们也有具体的地址空间。

在*.axf 文件中，跳转到 RCC_AHB1PeriphClockCmd 及 GPIO_Init 标号的这两个指令后都有注释，分别是“; 0x80002e0”及“; 0x80001fa”，它们是这两个标号所在的具体地址，而且这两个跳转语句的跟*.o 中的也有区别，内容分别为“f000f82a”及“f7ffffa6”(*.o 中的均为 f7ffffffe)。这就是链接器链接的含义，它把不同*.o 中的内容链接起来了。

分散加载代码

学习至此，还有一个疑问，前面提到程序有存储态及运行态，它们之间应有一个转化过程，把存储在 FLASH 中的 RW-data 数据拷贝至 SRAM。然而我们的工程中并没有编写这样的代码，在汇编文件中也查不到该过程，芯片是如何知道 FLASH 的哪些数据应拷贝到 SRAM 的哪些区域呢？

通过查看“多彩流水灯_axf_elfInfo.c.txt”的反汇编信息，了解到程序中具有一段名为“__scatterload”的分散加载代码，见代码清单 48-7，它是由 armlink 链接器自动生成的。

代码清单 48-7 分散加载代码(多彩流水灯_axf_elfInfo.c.txt 文件)

```

1      $t
2      .text
3      __scatterload
4      __scatterload_rt2
5      0x080001c0: 4c06      .L      LDR      r4,[pc,#24] ; [0x80001dc] = 0x8000514
6      0x080001c2: 4d07      .M      LDR      r5,[pc,#28] ; [0x80001e0] = 0x8000524
7      0x080001c4: e006      ..      B        0x80001d4 ; __scatterload + 20
8      0x080001c6: 68e0      .h      LDR      r0,[r4,#0xc]
9      0x080001c8: f0400301 @...    ORR      r3,r0,#1
10     0x080001cc: e8940007 ....    LDM      r4,{r0-r2}
11     0x080001d0: 4798      .G      BLX      r3
12     0x080001d2: 3410      .4      ADDS     r4,r4,#0x10
13     0x080001d4: 42ac      .B      CMP      r4,r5
14     0x080001d6: d3f6      ..      BCC      0x80001c6 ; __scatterload + 6
15     0x080001d8: f7ffffda ....    BL        __main_after_scatterload ; 0x8000190
16     $d
17     0x080001dc: 08000514 ....    DCD      134219028
18     0x080001e0: 08000524 $....    DCD      134219044

```

这段分散加载代码包含了拷贝过程(LDM 复制指令)，而 LDM 指令的操作数中包含了加载的源地址，这些地址中包含了内部 FLASH 存储的 RW-data 数据。而“__scatterload”的代码会被“__main”函数调用，见代码清单 48-8，__main 在启动文件中的

“Reset_Handler”会被调用，因而，在主体程序执行前，已经完成了分散加载过程。

代码清单 48-8 __main 的反汇编代码（部分，多彩流水灯_axf_elfInfo.c.txt 文件）

```

1      __main
2      main stk
3      0x08000188: f8dfd00c ....    LDR      sp,__lit__00000000 ; [0x8000198] = 0x20000400
4      .ARM.Collect$__$00000004
5      __main_scatterload
6      0x0800018c: f000f818 ....    BL        __scatterload ; 0x80001c0

```

5. hex 文件及 bin 文件

若编译过程无误，即可把工程生成前面对应的*.axf 文件，而在 MDK 中使用下载器(DAP/JLINK/ULINK 等)下载程序或仿真的时候，MDK 调用的就是*.axf 文件，它解释该文件，然后控制下载器把*.axf 中的代码内容下载到 STM32 芯片对应的存储空间，然后复位后芯片就开始执行代码了。

然而，脱离了 MDK 或 IAR 等工具，下载器就无法直接使用*.axf 文件下载代码了，它们一般仅支持 hex 和 bin 格式的代码数据文件。默认情况下 MDK 都不会生成 hex 及 bin 文件，需要配置工程选项或使用 fromelf 命令。

生成 hex 文件

生成 hex 文件的配置比较简单，在“Options for Target->Output->Create Hex File”中勾选该选项，然后编译工程即可，见图 48-35。



图 48-35 生成 hex 文件的配置

生成 bin 文件

使用 MDK 生成 bin 文件需要使用 fromelf 命令，在 MDK 的“Options For Target->Users”中加入图 48-36 中的命令。

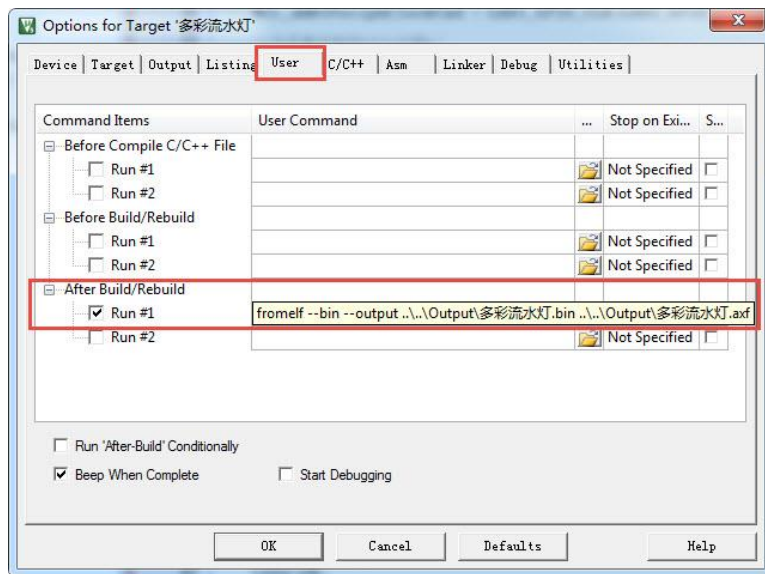


图 48-36 使用 fromelf 指令生成 bin 文件

图中的指令内容为：

“fromelf --bin --output ..\..\Output\多彩流水灯.bin ..\..\Output\多彩流水灯.axf”

该指令是根据本机及工程的配置而写的，在不同的系统环境或不同的工程中，指令内容都不一样，我们需要理解它，才能为自己的工程定制指令，首先看看 fromelf 的帮助，见图 48-37。

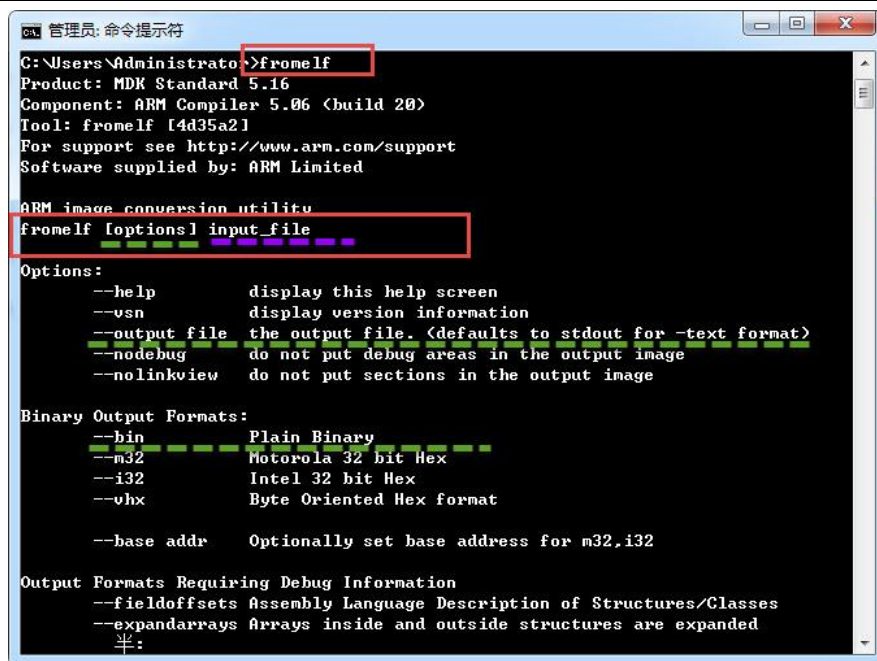


图 48-37 fromelf 的帮助

我们在 MDK 输入的指令格式是遵守 fromelf 帮助里的指令格式说明的，其格式为：

“fromelf [options] input_file”

其中 optinos 是指令选项，一个指令支持输入多个选项，每个选项之间使用空格隔开，我们的实例中使用 “--bin” 选项设置输出 bin 文件，使用 “--output file” 选项设置输出文件的名为 “..\..\Output\多彩流水灯.bin”，这个名字是一个相对路径格式，如果不了解如何使用 “..” 表示路径，可使用 MDK 命令输入框后面的文件夹图标打开文件浏览器选择文件，在命令的最后使用 “..\..\Output\多彩流水灯.axf” 作为命令的输入文件。具体的格式分解见图 48-38。

fromelf	--bin	--output	..\..\Output\多彩流水灯.bin	..\..\Output\多彩流水灯.axf
	输出文件名(含路径)			输入文件名(含路径)
工具路径	选项1	选项2	输入文件	
fromelf	[options]			inputfile

图 48-38 fromelf 命令格式分解

fromelf 需要根据工程的*.axf 文件输入来转换得到 bin 文件，所以在命令的输入文件参数中要选择本工程对应的*.axf 文件，在 MDK 命令输入栏中，我们把 fromelf 指令放置在 “After Build/Rebuild” (工程构建完成后执行)一栏也是基于这个考虑，这样设置后，工程构建完成生成了最新的*.axf 文件，MDK 再执行 fromelf 指令，从而得到最新的 bin 文件。

设置完成生成 hex 的选项或添加了生成 bin 的用户指令后，点击工程的编译(build)按钮，重新编译工程，成功后可看到图 48-39 中的输出。打开相应的目录即可找到文件，若找不到 bin 文件，请查看提示输出栏执行指令的信息，根据信息改正 fromelf 指令。


```
Build Output
*** Using Compiler 'V5.06 (build 20)', folder: 'D:\work\keil5\ARM\ARMCC\Bin'
Build target '多彩流水灯'
linking...
Program Size: Code=1012 RO-data=444 RW-data=0 ZI-data=1024
FromELF: creating hex file...
After Build - User command #1: fromelf --bin --output ..\..\Output\多彩流水灯.bin ..\..\Output\多彩流水灯.axf
"..\..\Output\多彩流水灯.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:02
```

图 48-39 fromelf 生成 hxe 及 bin 文件的提示

其中 bin 文件是纯二进制数据，无特殊格式，接下来我们了解一下 hex 文件格式。

hex 文件格式

hex 是 Intel 公司制定的一种使用 ASCII 文本记录机器码或常量数据的文件格式，这种文件常常用来记录将要存储到 ROM 中的数据，绝大多数下载器支持该格式。

一个 hex 文件由多条记录组成，而每条记录由五个部分组成，格式形如

“:l~~aaa~~att[dd...]~~cc~~”，例如本“多彩流水灯”工程生成的 hex 文件前几条记录见代码清单 48-9。

代码清单 48-9 Hex 文件实例(多彩流水灯.hex 文件，可直接用记事本打开)

```
1 :020000040800F2
2 :10000000000400209D010008DB0200087F020008B8
3 :10001000D9020008E50100080D04000800000000F6
4 :10002000000000000000000000000000FD020008C9
5 :10003000E701000800000000DD020008B903000825
6 :10004000B7010008B7010008B7010008B7010008B0
```

记录的各个部分介绍如下：

- “:”：每条记录的开头都使用冒号来表示一条记录的开始；
- 11：以 16 进制数表示这条记录的主体数据区的长度(即后面[dd...]的长度)；
- aaaa:表示这条记录中的内容应存放到 FLASH 中的起始地址；
- tt: 表示这条记录的类型，它包含中的各种类型；

表 48-5 tt 值所代表的类型说明

tt 的值	代表的类型
00	数据记录
01	本文件结束记录
02	扩展地址记录
04	扩展线性地址记录(表示后面的记录按个这地址递增)
05	表示一个线性地址记录的起始(只适用于 ARM)

- dd: 表示一个字节的数，一条记录中可以有多个字节数据，ll 区表示了它有多少个字节的数据；
- cc: 表示本条记录的校验和，它是前面所有 16 进制数据 (除冒号外，两个为一组) 的对 256 取模运算的结果的补码。

例如，代码清单 48-9 中的第一条记录解释如下：

- (1) 02: 表示这条记录数据区的长度为 2 字节；
- (2) 0000: 表示这条记录要存储到的地址；
- (3) 04: 表示这是一条扩展线性地址记录；

- (4) 0800: 由于这是一条扩展线性地址记录, 所以这部分表示地址的高 16 位, 与前面的“0000”结合在一起, 表示要扩展的线性地址为“0x0800 0000”, 这正好是 STM32 内部 FLASH 的首地址;
- (5) F2: 表示校验和, 它的值为 $(0x02+0x00+0x00+0x04+0x08+0x00)\%256$ 的值再取补码。

再来看第二条记录:

- (1) 10: 表示这条记录数据区的长度为 2 字节;
- (2) 0000: 表示这条记录所在的地址, 与前面的扩展记录结合, 表示这条记录要存储的 FLASH 首地址为 $(0x0800\ 0000+0x0000)$;
- (3) 00: 表示这是一条数据记录, 数据区的是地址;
- (4) 000400209D010008DB0200087F020008: 这是要按地址存储的数据;
- (5) B8: 校验和

为了更清楚地对比 bin、hex 及 axf 文件的差异, 我们来查看这些文件内部记录的信息来进行对比。

hex、bin 及 axf 文件的区别与联系

bin、hex 及 axf 文件都包含了指令代码, 但它们的信息丰富程度是不一样的。

- ❑ bin 文件是最直接的代码映像, 它记录的内容就是要存储到 FLASH 的二进制数据(机器码本质上就是二进制数据), 在 FLASH 中是什么形式它就是什么形式, 没有任何辅助信息, 包括大小端格式也没有, 因此下载器需要有针对芯片 FLASH 平台的辅助文件才能正常下载(一般下载器程序会有匹配的这些信息);
- ❑ hex 文件是一种使用十六进制符号表示的代码记录, 记录了代码应该存储到 FLASH 的哪个地址, 下载器可以根据这些信息辅助下载;
- ❑ axf 文件在前文已经解释, 它不仅包含代码数据, 还包含了工程的各种信息, 因此它也是三个文件中最大的。

同一个工程生成的 bin、hex 及 axf 文件的大小见图 48-40。

名称	大小
多彩流水灯.axf	328 KB
多彩流水灯.bin	2 KB
多彩流水灯.hex	4 KB

图 48-40 同一个工程的 bin、hex 及 axf 文件大小

实际上, 这个工程要烧写到 FLASH 的内容总大小为 1316 字节, 然而在 Windows 中查看的 bin 文件却比它大(bin 文件是 FLASH 的代码映像, 大小应一致), 这是因为 Windows 文件显示单位的原因, 使用右键查看文件的属性, 可以查看它实际记录内容的大小, 见图 48-41。

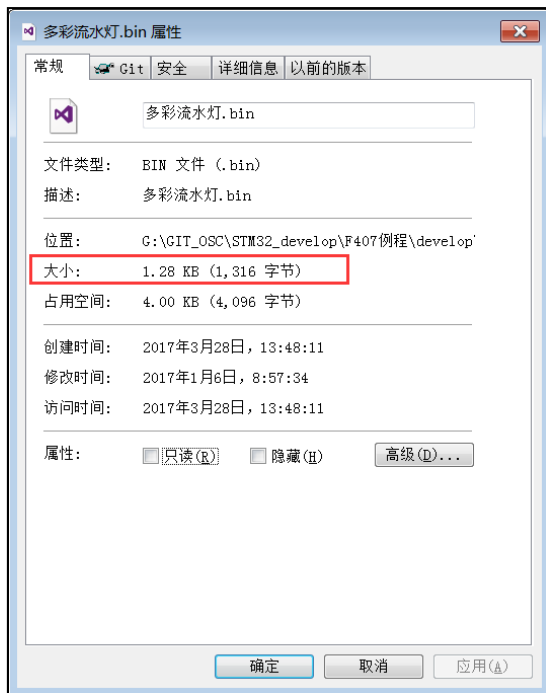


图 48-41 bin 文件大小

接下来我们打开本工程的“多彩流水灯.bin”、“多彩流水灯.hex”及由“多彩流水灯.axf”使用 fromelf 工具输出的反汇编文件“多彩流水灯_axf_elfInfo_c.txt”文件，清晰地对比它们的差异，见图 48-42。如果您想要亲自阅读自己电脑上的 bin 文件，推荐使用 sublime 软件打开，它可以把二进制数以 ASCII 码呈现出来，便于阅读。

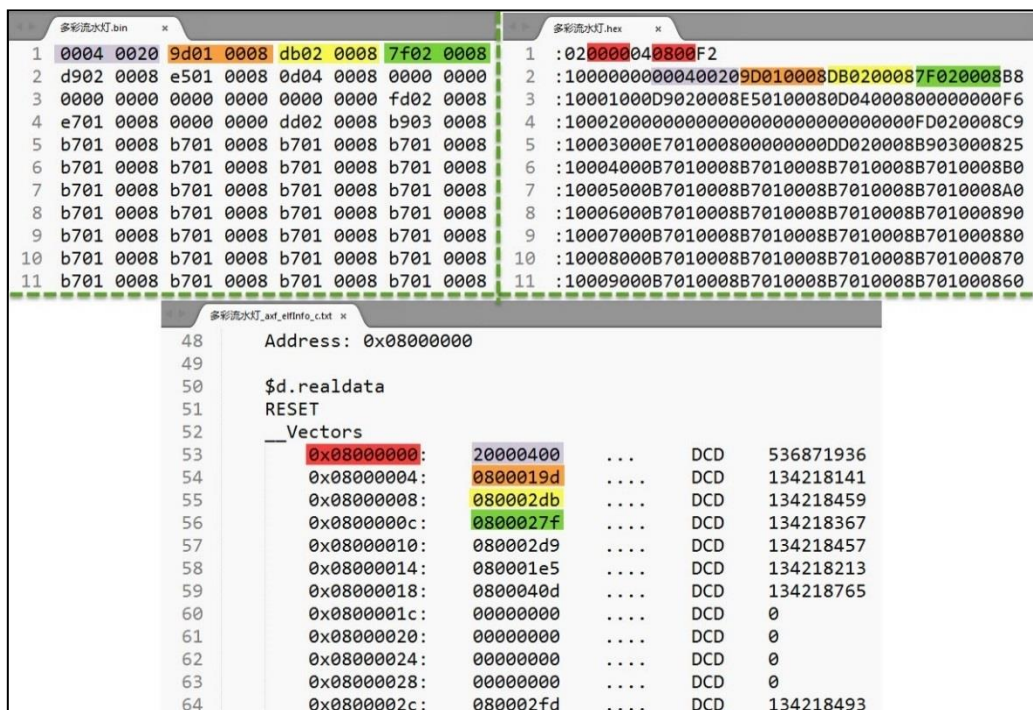


图 48-42 同一个工程的 bin、hex 及 axf 文件对代码的记录

在“多彩流水灯_axf_elfInfo_c.txt”文件中不仅可以看到代码数据，还有具体的标号、地址以及反汇编得到的代码，虽然它不是*.axf 文件的原始内容，但因为它是通过*.axf 文件

fromelf 工具生成的，我们可以认为*.axf 文件本身记录了大量这些信息，它的内容非常丰富，熟悉汇编语言的人可轻松阅读。

在 hex 文件中包含了地址信息以及地址中的内容，而在 bin 文件中仅包含了内容，连存储的地址信息都没有。观察可知，bin、hex 及 axf 文件中的数据内容都是相同的，它们存储的都是机器码。这就是它们三都之间的区别与联系。

由于文件中存储的都是机器码，见图 48-43，该图是我根据 axf 文件的 GPIO_Init 函数的机器码，在 bin 及 hex 中找到的对应位置。所以经验丰富的人是有可能从 bin 或 hex 文件中恢复出汇编代码的，只是成本较高，但不是不可能。

多彩流水灯.bin	多彩流水灯.hex
31 ad01 0008 064c 074d 06e0 e068 40f0 0103	33 :1001F00094E8070098471034AC42F6D3FFF7DAFFD3
32 94e8 0700 9847 1034 ac42 f6d3 fff7 daff	34 :10020000A0050008B0050008FEE7704701B502E050
33 a005 0008 b005 0008 fee7 7047 01b5 02e0	35 :10021000098401E009000980028F9D108BD2DE9F3
34 0098 401e 0090 0098 0028 f9d1 08bd 2de9	36 :10022000F041002201274FF0030C07FA02F50B689A
35 f041 0022 0127 4ff0 030c 07fa 02f5 0b68	37 :100230002B40AB422CD106685300CFA03F4A643C2
36 2b40 ab42 2cd1 0668 5300 0cfa 03f4 a643	38 :1002400006600E79D0F800809E4046EA08060660F7
37 0660 0e79 d0f8 0080 9e40 46ea 0806 0660	39 :10025000E79012E01D0022E12D18668A643866047
38 0e79 012e 01d0 022e 12d1 8668 a643 8660	40 :100260004E79D0F800809E4046EA080606604668C7
39 4e79 d0f8 0880 9e40 46ea 0806 8660 4668	41 :10027000AE43466045688E799640B68235434560D8
40 ae43 4660 4568 8e79 9640 b6b2 3543 4560	42 :10028000C568A543C560CC799C40C3681C43C46065
41 c568 a543 c560 cc79 9c40 c368 1c43 c460	43 :10029000521C102AC9D38DE8F08141837047018305

多彩流水灯.axf_elfInfo.c.txt
336 0x0800021c: bd08 .. POP {r3,pc}
337 i.GPIO_Init
338 GPIO_Init
339 0x0800021e: e92d41f0 -. .A PUSH {r4-r8,lr}
340 0x08000222: 2200 . " MOVs r2,#0
341 0x08000224: 2701 . ' MOVs r7,#1
342 0x08000226: f04f0c03 0... MOV r12,#3
343 0x0800022a: fa07f502 LSL r5,r7,r2
344 0x0800022e: 680b .h LDR r3,[r1,#0]
345 0x08000230: 402b +@ ANDS r3,r3,r5
346 0x08000232: 42ab .B CMP r3,r5

图 48-43 GPIO_Init 函数的代码数据在三个文件中的表示

如果芯片没有做任何加密措施，使用下载器可以直接从芯片读回它存储在 FLASH 中的数据，从而得到 bin 映像文件，根据芯片型号还原出部分代码即可进行修改，甚至不用修改代码，直接根据目标产品的硬件 PCB，抄出一样的板子，再把 bin 映像下载芯片，直接山寨出目标产品，所以在实际的生产中，一定要注意做好加密措施。由于 axf 文件中含有大量的信息，且直接使用 fromelf 即可反汇编代码，所以更不要随便泄露 axf 文件。lib 文件也能反使用 fromelf 文件反汇编代码，不过它不能还原出 C 代码，由于 lib 文件的主要目的是为了为了保护 C 源代码，也算是达到了它的要求。

6. htm 静态调用图文件

在 Output 目录下，有一个以工程文件命名的后缀为*.bulid_log.htm 及*.htm 文件，如“多彩流水灯.bulid_log.htm”及“多彩流水灯.htm”，它们都可以使用浏览器打开。其中*.build_log.htm 是工程的构建过程日志，而*.htm 是链接器生成的静态调用图文件。

在静态调用图文件中包含了整个工程各种函数之间互相调用的关系图，而且它还给出了静态占用最深的栈空间数量以及它对应的调用关系链。

例如图 48-44 是“多彩流水灯.htm”文件顶部的说明。

Static Call Graph for image ..\..\Output\多彩流水灯.axf

#<CALLGRAPH># ARM Linker, 5050169: Last Updated: Mon Mar 27 16:30:14 2017

Maximum Stack Usage = 56 bytes + Unknown(Cycles, Untraceable Function Pointers)

Call chain for Maximum Stack Depth:

main ⇒ LED_GPIO_Config ⇒ GPIO_Init

图 48-44 “多彩流水灯.htm”中的静态占用最深的栈空间说明

该文件说明了本工程的静态栈空间最大占用 56 字节(Maximum Stack Usage:56bytes)，这个占用最深的静态调用为“main->LED_GPIO_Config->GPIO_Init”。注意这里给出的空间只是静态的栈使用统计，链接器无法统计动态使用情况，例如链接器无法知道递归函数的递归深度。在本文件的后面还可查询到其它函数的调用情况及其它细节。

利用这些信息，我们可以大致了解工程中应该分配多少空间给栈，有空间余量的情况下，一般会设置比这个静态最深栈使用量大一倍，在 STM32 中可修改启动文件改变堆栈的大小；如果空间不足，可从该文件中了解到调用深度的信息，然后优化该代码。

注意：

查看了各个工程的静态调用图文件统计后，我们发现本书提供的一些比较大规模的工程例子，静态栈调用最大深度都已超出 STM32 启动文件默认的栈空间大小 0x00000400，即 1024 字节，但在当时的调试过程中却没有发现错误，因此我们也没有修改栈的默认大小(有一些工程调试时已发现问题，它们的栈空间就已经被我们改大了)，虽然这些工程实际运行并没有错误，但这可能只是因为它使用的栈溢出 RAM 空间恰好没被程序其它部分修改而已。所以，建议您在实际的大型工程应用中(特别是使用了各种外部库时，如 Lwip/emWin/Fatfs 等)，要查看本静态调用图文件，了解程序的栈使用情况，给程序分配合适的栈空间。

48.4.4 Listing 目录下的文件

在 Listing 目录下包含了*.map 及*.lst 文件，它们都是文本格式的，可使用 Windows 的记事本软件打开。其中 lst 文件仅包含了一些汇编符号的链接信息，我们重点分析 map 文件。

1. map 文件说明

map 文件是由链接器生成的，它主要包含交叉链接信息，查看该文件可以了解工程中各种符号之间的引用以及整个工程的 Code、RO-data、RW-data 以及 ZI-data 的详细及汇总信息。它的内容中主要包含了“节区的跨文件引用”、“删除无用节区”、“符号映像表”、“存储器映像索引”以及“映像组件大小”，各部分介绍如下：

节区的跨文件引用

打开“多彩流水灯.map”文件，可看到它的第一部分——节区的跨文件引用(Section Cross References)，见代码清单 48-10。

代码清单 48-10 节区的跨文件引用(部分，多彩流水灯.map 文件)

```
1 =====
2 Section Cross References
3
4 startup_stm32f40xx.o(RESET) refers to startup_stm32f40xx.o(STACK) for __initial_sp
5 startup_stm32f40xx.o(RESET) refers to startup_stm32f40xx.o(.text) for Reset_Handler
6 startup_stm32f40xx.o(RESET) refers to stm32f4xx_it.o(i.NMI_Handler) for NMI_Handler
7 startup_stm32f40xx.o(RESET) refers to stm32f4xx_it.o(i.HardFault_Handler) for HardFault_Handler
8 /**...以下部分省略****/
9 main.o(i.main) refers to bsp_led.o(i.LED_GPIO_Config) for LED_GPIO_Config
10 main.o(i.main) refers to stm32f4xx_gpio.o(i.GPIO_ResetBits) for GPIO_ResetBits
11 main.o(i.main) refers to main.o(i.Delay) for Delay
12 main.o(i.main) refers to stm32f4xx_gpio.o(i.GPIO_SetBits) for GPIO_SetBits
13 bsp_led.o(i.LED_GPIO_Config) refers to stm32f4xx_rcc.o(i.RCC_AHB1PeriphClockCmd) for RCC_AHB1PeriphClockCmd
14 bsp_led.o(i.LED_GPIO_Config) refers to stm32f4xx_gpio.o(i.GPIO_Init) for GPIO_Init
15 /**...以下部分省略****/
16 =====
```

在这部分中，详细列出了各个*.o 文件之间的符号引用。由于*.o 文件是由 asm 或 c/c++ 源文件编译后生成的，各个文件及文件内的节区间互相独立，链接器根据它们之间的互相引用链接起来，链接的详细信息在这个“Section Cross References”一一列出。

例如，开头部分说明的是 startup_stm32f429_439xx.o 文件中的“RESET”节区分为它使用的“__initial_sp”符号引用了同文件“STACK”节区。

也许我们对启动文件不熟悉，不清楚这究竟是什么，那我们继续浏览，可看到 main.o 文件的引用说明，如说明 main.o 文件的 i.main 节区为它使用的 LED_GPIO_Config 符号引用了 bsp_led.o 文件的 i.LED_GPIO_Config 节区。

同样地，下面还有 bsp_led.o 文件的引用说明，如说明了 bsp_led.o 文件的 i.LED_GPIO_Config 节区为它使用的 GPIO_Init 符号引用了 stm32f4xx_gpio.o 文件的 i.GPIO_Init 节区。

可以了解到，这些跨文件引用的符号其实就是源文件中的函数名、变量名。有时在构建工程的时候，编译器会输出“Undefined symbol xxx(referred from xxx.o)”这样的提示，该提示的原因就是在链接过程中，某个文件无法在外部找到它引用的标号，因而产生链接错误。例如，见图 48-45，我们把 bsp_led.c 文件中定义的函数 LED_GPIO_Config 改名为 LED_GPIO_ConfigABCD，而不修改 main.c 文件中的调用，就会出现 main 文件无法找到 LED_GPIO_Config 符号的提示。

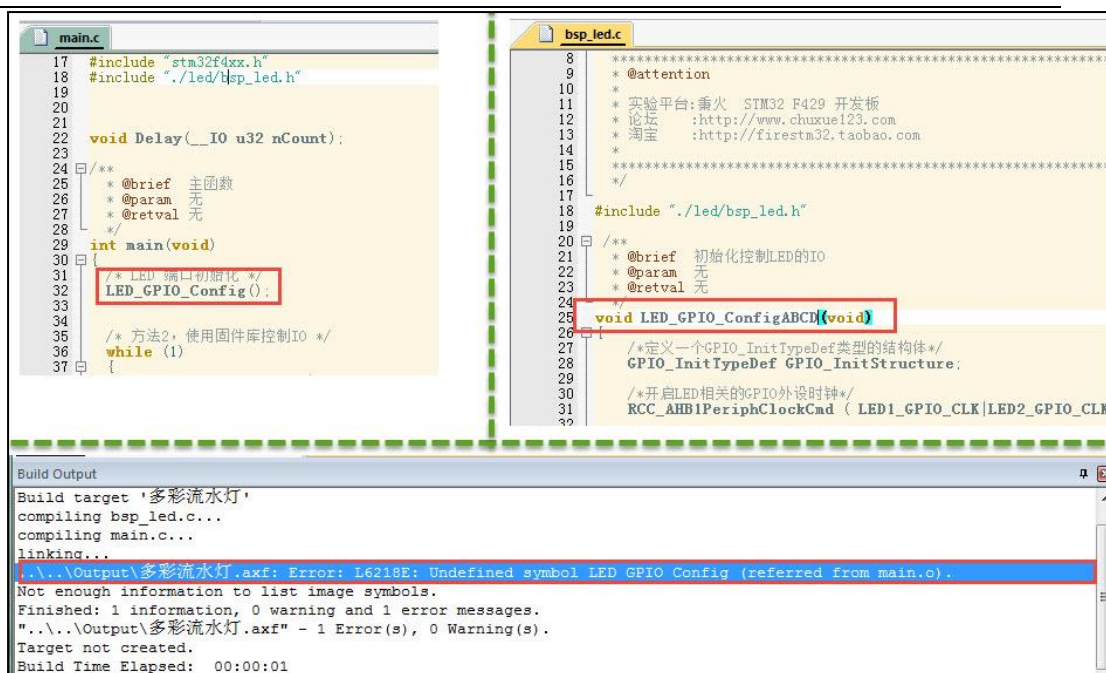


图 48-45 找不到符号的错误提示

删除无用节区

map 文件的第二部分是删除无用节区的说明(Removing Unused input sections from the image.), 见代码清单 48-11。

代码清单 48-11 删除无用节区(部分, 多彩流水灯.map 文件)

```
1 =====
2
3 Removing Unused input sections from the image.
4
5 Removing startup_stm32f40xx.o(HEAP), (512 bytes).
6 Removing system_stm32f4xx.o(.rev16_text), (4 bytes).
7 Removing system_stm32f4xx.o(.revsh_text), (4 bytes).
8 Removing system_stm32f4xx.o(.rrx_text), (6 bytes).
9 Removing system_stm32f4xx.o(i.SystemCoreClockUpdate), (136 bytes).
10 Removing system_stm32f4xx.o(.data), (20 bytes).
11 Removing misc.o(.rev16_text), (4 bytes).
12 Removing misc.o(.revsh_text), (4 bytes).
13 Removing misc.o(.rrx_text), (6 bytes).
14 Removing misc.o(i.NVIC_Init), (104 bytes).
15 Removing misc.o(i.NVIC_PriorityGroupConfig), (20 bytes).
16 Removing misc.o(i.NVIC_SetVectorTable), (20 bytes).
17 Removing misc.o(i.NVIC_SystemLPConfig), (28 bytes).
18 Removing misc.o(i.SysTick_CLKSourceConfig), (28 bytes).
19 Removing stm32f4xx_adc.o(.rev16_text), (4 bytes).
20 Removing stm32f4xx_adc.o(.revsh_text), (4 bytes).
21 Removing stm32f4xx_adc.o(.rrx_text), (6 bytes).
22 Removing stm32f4xx_adc.o(i.ADC_AnalogWatchdogCmd), (16 bytes).
23 Removing stm32f4xx_adc.o(i.ADC_AnalogWatchdogSingleChannelConfig), (12 bytes).
24 Removing stm32f4xx_adc.o(i.ADC_AnalogWatchdogThresholdsConfig), (6 bytes).
25 Removing stm32f4xx_adc.o(i.ADC_AutoInjectedConvCmd), (24 bytes).
26 Removing stm32f4xx_adc.o(i.ADC_ClearFlag), (6 bytes).
27 Removing stm32f4xx_adc.o(i.ADC_ClearITPendingBit), (8 bytes).
28 Removing stm32f4xx_adc.o(i.ADC_Cmd), (24 bytes).
29 /**...以下部分省略****/
30 =====
```

这部分列出了在链接过程它发现工程中未被引用的节区, 这些未被引用的节区将会被删除(指不加入到*.axf 文件, 不是指在*.o 文件删除), 这样可以防止这些无用数据占用程序空间。

例如, 上面的信息中说明 startup_stm32f429_439xx.o 中的 HEAP(在启动文件中定义的用于动态分配的“堆”区)以及 stm32f4xx_adc.o 的各个节区都被删除了, 因为在我们这个工程中没有使用动态内存分配, 也没有引用任何 stm32f4xx_adc.c 中的内容。由此也可以知道, 虽然我们把 STM32 标准库的各个外设对应的 c 库文件都添加到了工程, 但不必担心这会使工程变得臃肿, 因为未被引用的节区内容不会被加入到最终的机器码文件中。

符号映像表

map 文件的第三部分是符号映像表(Image Symbol Table), 见代码清单 48-12。

代码清单 48-12 符号映像表(部分, 多彩流水灯.map 文件)

```
1 =====
2
3 Image Symbol Table
4
5 Local Symbols
6
7 Symbol Name                                Value      Ov Type      Size  Object(Section)
8
9 ../clib/microlib/init/entry.s              0x00000000  Number       0      entry8a.o ABSOLUTE
10 ../clib/microlib/init/entry.s              0x00000000  Number       0      entry7b.o ABSOLUTE
11 ../clib/microlib/init/entry.s              0x00000000  Number       0      entry7a.o ABSOLUTE
12 /*...省略部分*/
13 LED_GPIO_Config                            0x08000281  Thumb Code   84      bsp_led.o(i.LED_GPIO_Config)
14 MemManage_Handler                         0x080002d9  Thumb Code   2      stm32f4xx_it.o(i.MemManage_Handler)
15 NMI_Handler                               0x080002db  Thumb Code   2      stm32f4xx_it.o(i.NMI_Handler)
16 PendSV_Handler                            0x080002dd  Thumb Code   2      stm32f4xx_it.o(i.PendSV_Handler)
17 RCC_AHB1PeriphClockCmd                    0x080002e1  Thumb Code   22      stm32f4xx_rcc.o(i.RCC_AHB1PeriphClockCmd)
18 SVC_Handler                               0x080002fd  Thumb Code   2      stm32f4xx_it.o(i.SVC_Handler)
19 SysTick_Handler                           0x080003b9  Thumb Code   2      stm32f4xx_it.o(i.SysTick_Handler)
20 SystemInit                                0x080003bd  Thumb Code   74      system_stm32f4xx.o(i.SystemInit)
21 UsageFault_Handler                        0x08000419  Thumb Code   2      stm32f4xx_it.o(i.UsageFault_Handler)
22 __scatterload_copy                         0x0800041b  Thumb Code   14      handlers.o(i.__scatterload_copy)
23 __scatterload_null                         0x08000429  Thumb Code   2      handlers.o(i.__scatterload_null)
24 __scatterload_zeroinit                     0x0800042b  Thumb Code   14      handlers.o(i.__scatterload_zeroinit)
25 main                                        0x08000439  Thumb Code   226     main.o(i.main)
26 Region$$Table$$Base                        0x08000520  Number       0      anon$$obj.o(Region$$Table)
27 Region$$Table$$Limit                       0x08000530  Number       0      anon$$obj.o(Region$$Table)
28 __initial_sp                               0x20000400  Data         0      startup_stm32f40xx.o(STACK)
29
30 /*...省略部分*/
31
32 =====
```

这个表列出了被引用的各个符号在存储器中的具体地址、占据的空间大小等信息。如我们可以查到 LED_GPIO_Config 符号存储在 0x08000281 地址, 它属于 Thumb Code 类型, 大小为 84 字节, 它所在的节区为 bsp_led.o 文件的 i.LED_GPIO_Config 节区。

存储器映像索引

map 文件的第四部分是存储器映像索引(Memory Map of the image), 见代码清单 48-13。

代码清单 48-13 存储器映像索引(部分, 多彩流水灯.map 文件)

```
1 =====
2 Memory Map of the image
3
4 Image Entry point : 0x080001ad
5 Load Region LR_IROM1 (Base: 0x08000000, Size: 0x000005b0, Max: 0x00100000, ABSOLUTE)
6
7 Execution Region ER_IROM1 (Base: 0x08000000, Size: 0x000005b0, Max: 0x00100000, ABSOLUTE)
8
9 Base Addr      Size      Type  Attr      Idx  E Section Name      Object
10
11 0x08000000     0x000001ac  Data  RO        3    RESET               startup_stm32f429_439xx.o
12 /*...省略部分*/
13 0x08000020c     0x00000012  Code  RO       5161   i.Delay              main.o
14 0x08000021e     0x00000007c  Code  RO       2046   i.GPIO_Init          stm32f4xx_gpio.o
15 0x08000029a     0x000000004  Code  RO       2053   i.GPIO_ResetBits     stm32f4xx_gpio.o
16 0x08000029e     0x000000004  Code  RO       2054   i.GPIO_SetBits       stm32f4xx_gpio.o
17 0x0800002a2     0x000000002  Code  RO       5196   i.HardFault_Handler  stm32f4xx_it.o
18 0x0800002a4     0x000000074  Code  RO       5269   i.LED_GPIO_Config    bsp_led.o
19 0x080000318     0x000000002  Code  RO       5197   i.MemManage_Handler  stm32f4xx_it.o
20 /*...省略部分*/
21 0x080000488     0x000000118  Code  RO       5162   i.main               main.o
22 0x0800005a0     0x000000010  Data  RO       5309   Region$$Table        anon$$obj.o
23
24 Execution Region RW_IRAM1 (Base: 0x20000000, Size: 0x00000400, Max: 0x00030000, ABSOLUTE)
25
26 Base Addr      Size      Type  Attr      Idx  E Section Name      Object
27
28 0x20000000     0x00000400  Zero  RW        1    STACK               startup_stm32f429_439xx.o
```

```
1 =====
2
3 Memory Map of the image
4
5 Image Entry point : 0x08000189
6
7 Load Region LR_IROM1 (Base: 0x08000000, Size: 0x00000530, Max: 0x00100000, ABSOLUTE)
8
9 Execution Region ER_IROM1 (Base: 0x08000000, Size: 0x00000530, Max: 0x00100000, ABSOLUTE)
10
11 Base Addr      Size      Type  Attr      Idx  E Section Name      Object
12
13 0x08000000     0x00000188    Data  RO          3   RESET               startup_stm32f40xx.o
14 /*..省略部分*/
15 0x080001e8     0x00000012    Code  RO        4765   i.Delay              main.o
16 0x080001fa     0x0000007c    Code  RO        1857   i.GPIO_Init          stm32f4xx_gpio.o
17 0x08000276     0x00000004    Code  RO        1864   i.GPIO_ResetBits     stm32f4xx_gpio.o
18 0x0800027a     0x00000004    Code  RO        1865   i.GPIO_SetBits       stm32f4xx_gpio.o
19 0x0800027e     0x00000002    Code  RO        4800   i.HardFault_Handler  stm32f4xx_it.o
20 0x08000280     0x00000058    Code  RO        4873   i.LED_GPIO_Config    bsp_led.o
21 0x080002d8     0x00000002    Code  RO        4801   i.MemManage_Handler  stm32f4xx_it.o
22 /*..省略部分*/
23 0x08000438     0x000000e8    Code  RO        4766   i.main                main.o
24 0x08000520     0x00000010    Data  RO        4913   Region$$Table         anon$$obj.o
25
26
27 Execution Region RW_IRAM1 (Base: 0x20000000, Size: 0x00000400, Max: 0x00020000, ABSOLUTE)
28
29 Base Addr      Size      Type  Attr      Idx  E Section Name      Object
30
31 0x20000000     0x00000400    Zero  RW          1   STACK                startup_stm32f40xx.o
32
33
34 =====
```

本工程的存储器映像索引分为 ER_IROM1 及 RW_IRAM1 部分，它们分别对应 STM32 内部 FLASH 及 SRAM 的空间。相对于符号映像表，这个索引表描述的单位是节区，而且它描述的主要信息中包含了节区的类型及属性，由此可以区分 Code、RO-data、RW-data 及 ZI-data。

例如，从上面的表中我们可以看到 i.LED_GPIO_Config 节区存储在内部 FLASH 的 0x08000280 地址，大小为 0x00000058，类型为 Code，属性为 RO。而程序的 STACK 节区(栈空间)存储在 SRAM 的 0x20000000 地址，大小为 0x00000400，类型为 Zero，属性为 RW（即 RW-data）。

映像组件大小

map 文件的最后一部分是包含映像组件大小的信息(Image component sizes)，这也是最常查询的内容，见代码清单 48-14。

代码清单 48-14 映像组件大小(部分，多彩流水灯.map 文件)

```
1 =====
2
3 Image component sizes
4
5
6 Code (inc. data)  RO Data  RW Data  ZI Data  Debug  Object Name
7
8 88              4          0          0          0       702  bsp_led.o
9 250             6          0          0          0      1787  main.o
10 36              8        392          0        1024     1048  startup_stm32f40xx.o
11 132             0          0          0          0     2940  stm32f4xx_gpio.o
12 18              0          0          0          0     5198  stm32f4xx_it.o
13 28              6          0          0          0      785  stm32f4xx_rcc.o
14 276            34          0          0          0    250921  system_stm32f4xx.o
15
16 -----
17 834             58        408          0        1024    263381  Object Totals
18 0              0         16          0          0         0  (incl. Generated)
19 6              0          0          0          0         0  (incl. Padding)
20
```

```
21 /*...省略部分*/
22 =====
23
24
25 Code (inc. data)   RO Data   RW Data   ZI Data   Debug
26
27 920                74         408        0        1024    262453   Grand Totals
28 920                74         408        0        1024    262453   ELF Image Totals
29 920                74         408        0         0         0       ROM Totals
30
31 =====
32
33 Total RO   Size (Code + RO Data)           1328 (   1.30kB)
34 Total RW   Size (RW Data + ZI Data)         1024 (   1.00kB)
35 Total ROM  Size (Code + RO Data + RW Data)   1328 (   1.30kB)
36
37 =====
```

这部分包含了各个使用到的*.o 文件的空间汇总信息、整个工程的空间汇总信息以及占用不同类型存储器的空间汇总信息，它们分类描述了具体占据的 Code、RO-data、RW-data 及 ZI-data 的大小，并根据这些大小统计出占据的 ROM 总空间。

我们仅分析最后两部分信息，如 Grand Totals 一项，它表示整个代码占据的所有空间信息，其中 Code 类型的数据大小为 920 字节，这部分包含了 74 字节的指令数据(inc .data)已算在内，另外 RO-data 占 408 字节，RW-data 占 0 字节，ZI-data 占 1024 字节。在它的下面两行有一项 ROM Totals 信息，它列出了各个段所占据的 ROM 空间，除了 ZI-data 不占 ROM 空间外，其余项都与 Grand Totals 中相等(RW-data 也占据 ROM 空间，只是本工程中没有 RW-data 类型的数据而已)。

最后一部分列出了只读数据(RO)、可读写数据(RW)及占据的 ROM 大小。其中只读数据大小为 1328 字节，它包含 Code 段及 RO-data 段；可读写数据大小为 1024 字节，它包含 RW-data 及 ZI-data 段；占据的 ROM 大小为 1328 字节，它除了 Code 段和 RO-data 段，还包含了运行时需要从 ROM 加载到 RAM 的 RW-data 数据。

综合整个 map 文件的信息，可以分析出，当程序下载到 STM32 的内部 FLASH 时，需要使用的内部 FLASH 是从 0x0800 0000 地址开始的大小为 1328 字节的空间；当程序运行时，需要使用的内部 SRAM 是从 0x20000000 地址开始的大小为 1024 字节的空间。

粗略一看，发现这个小程序竟然需要 1024 字节的 SRAM，实在说不过去，但仔细分析 map 文件后，可了解到这 1024 字节都是 STACK 节区的空间(即栈空间)，栈空间大小是在启动文件中定义的，这 1024 字节是默认值(0x00000400)。它是提供给 C 语言程序局部变量申请使用的空间，若我们确认自己的应用程序不需要这么大的栈，完全可以修改启动文件，把它改小一点，查看前面讲解的 htm 静态调用图文件可了解静态的栈调用情况，可以用它作为参考。

48.4.5 sct 分散加载文件的格式与应用

1. sct 分散加载文件简介

当工程按默认配置构建时，MDK 会根据我们选择的芯片型号，获知芯片的内部 FLASH 及内部 SRAM 存储器概况，生成一个以工程名命名的后缀为*.sct 的分散加载文件(Linker Control File, scatter loading)，链接器根据该文件的配置分配各个节区地址，生成分散加载代码，因此我们通过修改该文件可以定制具体节区的存储位置。

例如可以设置源文件中定义的所有变量自动按地址分配到外部 SRAM，这样就不需要再使用关键字“__attribute__”按具体地址来指定了；利用它还可以控制代码的加载区与执行区的位置，例如可以把程序代码存储到单位容量价格便宜的 NAND-FLASH 中，但在 NAND-FLASH 中的代码是不能像内部 FLASH 的代码那样直接提供给内核运行的，这时可通过修改分散加载文件，把代码加载区设定为 NAND-FLASH 的程序位置，而程序的执行区设定为外部 SRAM 中的位置，这样链接器就会生成一个配套的分散加载代码，该代码会把 NAND-FLASH 中的代码加载到外部 SRAM 中，内核再从外部 SRAM 中运行主体代码，大部分运行 Linux 系统的代码都是这样加载的。

2. 分散加载文件的格式

下面先来看看 MDK 默认使用的 sct 文件，在 Output 目录下可找到“多彩流水灯.sct”，该文件记录的内容见代码清单 48-15。

代码清单 48-15 默认的分散加载文件内容(“多彩流水灯.sct”)

```
1 ; *****
2 ; *** Scatter-Loading Description File generated by uVision ***
3 ; *****
4
5 LR_IROM1 0x08000000 0x00100000 { ; 注释:加载域, 基地址 空间大小
6   ER_IROM1 0x08000000 0x00100000 { ; 注释:加载地址 = 执行地址
7     *.o (RESET, +First)
8     *(InRoot$$Sections)
9     .ANY (+RO)
10  }
11  RW_IRAM1 0x20000000 0x00020000 { ; 注释:可读写数据
12    .ANY (+RW +ZI)
13  }
14 }
15
```

在默认的 sct 文件配置中仅分配了 Code、RO-data、RW-data 及 ZI-data 这些大区域的地址，链接时各个节区(函数、变量等)直接根据属性排列到具体的地址空间。

sct 文件中主要包含描述加载域及执行域的部分，一个文件中可包含有多个加载域，而一个加载域可由多个部分的执行域组成。同等级的域之间使用花括号“{}”分隔开，最外层的是加载域，第二层“{}”内的是执行域，其整体结构见图 48-46。

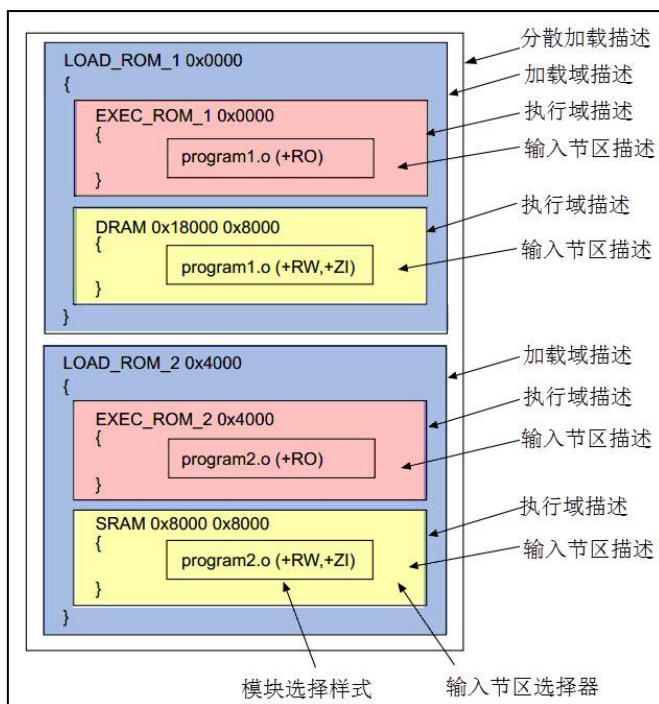


图 48-46 分散加载文件的整体结构

加载域

sct 文件的加载域格式见代码清单 48-16。

代码清单 48-16 加载域格式

```

1 //方括号中的为选填内容
2 加载域名 (基地址 | ("+" 地址偏移)) [属性列表] [最大容量]
3 "{"
4     执行区域描述+
5 "}"

```

配合前面代码清单 48-15 中的分散加载文件内容，各部分介绍如下：

- ❑ 加载域名：名称，在 map 文件中的描述会使用该名称来标识空间。如本例中只有一个加载域，该域名为 LR_IROM1。
- ❑ 基地址+地址偏移：这部分说明了本加载域的基地址，可以使用+号连接一个地址偏移，算进基地址中，整个加载域以它们的结果为基地址。如本例中的加载域基地址为 0x08000000，刚好是 STM32 内部 FLASH 的基地址。
- ❑ 属性列表：属性列表说明了加载域的是否为绝对地址、N 字节对齐等属性，该配置是可选的。本例中没有描述加载域的属性。
- ❑ 最大容量：最大容量说明了这个加载域可使用的最大空间，该配置也是可选的，如果加上这个配置后，当链接器发现工程要分配到该区域的空间比容量还大，它会在工程构建过程给出提示。本例中的加载域最大容量为 0x00100000，即 1MB，正是本型号 STM32 内部 FLASH 的空间大小。

执行域

sct 文件的执行域格式见代码清单 48-17。

代码清单 48-17 执行域格式

```

1 //方括号中的为选填内容
2 执行域名（基地址 | "+" 地址偏移）[属性列表] [最大容量 ]
3 "{"
4     输入节区描述
5 "}"

```

执行域的格式与加载域是类似的，区别只是输入节区的描述有所不同，在代码清单 48-15 的例子中包含了 ER_IROM1 及 RW_IRAM 两个执行域，它们分别对应描述了 STM32 的内部 FLASH 及内部 SRAM 的基地址及空间大小。而它们内部的“输入节区描述”说明了哪些节区要存储到这些空间，链接器会根据它来处理编排这些节区。

输入节区描述

配合加载域及执行域的配置，在相应的域配置“输入节区描述”即可控制该节区存储到域中，其格式见代码清单 48-18。

代码清单 48-18 输入节区描述的几种格式

```

1 //除模块选择样式部分外，其余部分都可选选填
2 模块选择样式"("输入节区样式","+"输入节区属性")"
3 模块选择样式"("输入节区样式","+"节区特性")"
4
5 模块选择样式"("输入符号样式","+"节区特性")"
6 模块选择样式"("输入符号样式","+"输入节区属性")"

```

配合前面代码清单 48-15 中的分散加载文件内容，各部分介绍如下：

- ❑ 模块选择样式：模块选择样式可用于选择 o 及 lib 目标文件作为输入节区，它可以直接使用目标文件名或“*”通配符，也可以使用“.ANY”。例如，使用语句“bsp_led.o”可以选择 bsp_led.o 文件，使用语句“*.o”可以选择所有 o 文件，使用“*.lib”可以选择所有 lib 文件，使用“*”或“.ANY”可以选择所有的 o 文件及 lib 文件。其中“.ANY”选择语句的优先级是最低的，所有其它选择语句选择完剩下的数据才会被“.ANY”语句选中。
- ❑ 输入节区样式：我们知道在目标文件中会包含多个节区或符号，通过输入节区样式可以选择要控制的节区。
示例文件中“(RESET, +First)”语句的 RESET 就是输入节区样式，它选择了名为 RESET 的节区，并使用后面介绍的节区特性控制字“+First”表示它要存储到本区域的第一个地址。示例文件中的“*(InRoot\$\$Sections)”是一个链接器支持的特殊选择符号，它可以选择所有标准库里要求存储到 root 区域的节区，如 __main.o、__scatter*.o 等内容。
- ❑ 输入符号样式：同样地，使用输入符号样式可以选择要控制的符号，符号样式需要使用“:gdef:”来修饰。例如可以使用“*(gdef:Value_Test)”来控制选择符号“Value_Test”。
- ❑ 输入节区属性：通过在模块选择样式后面加入输入节区属性，可以选择样式中不同的内容，每个节区属性描述符前要写一个“+”号，使用空格或“，”号分隔开，可以使用的节区属性描述符见表 48-6。

表 48-6 属性描述符及其意义

节区属性描述符	说明
RO-CODE 及 CODE	只读代码段
RO-DATA 及 CONST	只读数据段

RO 及 TEXT	包括 RO-CODE 及 RO-DATA
RW-DATA	可读写数据段
RW-CODE	可读写代码段
RW 及 DATA	包括 RW-DATA 及 RW-CODE
ZI 及 BSS	初始化为 0 的可读写数据段
XO	只可执行的区域
ENTRY	节区的入口点

例如，示例文件中使用“.ANY(+RO)”选择剩余所有节区 RO 属性的内容都分配到执行域 ER_IROM1 中，使用“.ANY(+RW +ZI)”选择剩余所有节区 RW 及 ZI 属性的内容都分配到执行域 RW_IRAM1 中。

- ❑ 节区特性：节区特性可以使用“+FIRST”或“+LAST”选项配置它要存储到的位置，FIRST 存储到区域的头部，LAST 存储到尾部。通常重要的节区会放在头部，而 CheckSum(校验和)之类的数据会放在尾部。

例如示例文件中使用“(RESET,+First)”选择了 RESET 节区，并要求把它放置到本区域第一个位置，而 RESET 是工程启动代码中定义的向量表，见代码清单 48-19，该向量表中定义的堆栈顶和复位向量指针必须要存储在内部 FLASH 的前两个地址，这样 STM32 才能正常启动，所以必须使用 FIRST 控制它们存储到首地址。

代码清单 48-19 startup_stm32f429_439xx.s 文件中定义的 RESET 区(部分)

```

1 ; Vector Table Mapped to Address 0 at Reset
2      AREA    RESET, DATA, READONLY
3      EXPORT  __Vectors
4      EXPORT  __Vectors_End
5      EXPORT  __Vectors_Size
6
7 __Vectors    DCD      __initial_sp          ; Top of Stack
8              DCD      Reset_Handler        ; Reset Handler
9              DCD      NMI_Handler          ; NMI Handler

```

总的来说，我们的 sct 示例文件配置如下：程序的加载域为内部 FLASH 的 0x08000000，最大空间为 0x00100000；程序的执行基地址与加载基地址相同，其中 RESET 节区定义的向量表要存储在内部 FLASH 的首地址，且所有 o 文件及 lib 文件的 RO 属性内容都存储在内部 FLASH 中；程序执行时 RW 及 ZI 区域都存储在以 0x20000000 为基地址，大小为 0x00020000 的空间(128KB)，这部分正好是 STM32 内部主 SRAM 的大小。

链接器根据 sct 文件链接，链接后各个节区、符号的具体地址信息可以在 map 文件中查看。

3. 通过 MDK 配置选项来修改 sct 文件

了解 sct 文件的格式后，可以手动编辑该文件控制整个工程的分散加载配置，但 sct 文件格式比较复杂，所以 MDK 提供了相应的配置选项可以方便地修改该文件，这些选项配置能满足基本的使用需求，本小节将对这些选项进行说明。

选择 sct 文件的产生方式

首先需要选择 sct 文件产生的方式，选择使用 MDK 生成还是使用用户自定义的 sct 文件。在 MDK 的“Options for Target->Linker->Use Memory Layout from Target Dialog”选项即可配置该选择，见图 48-47。

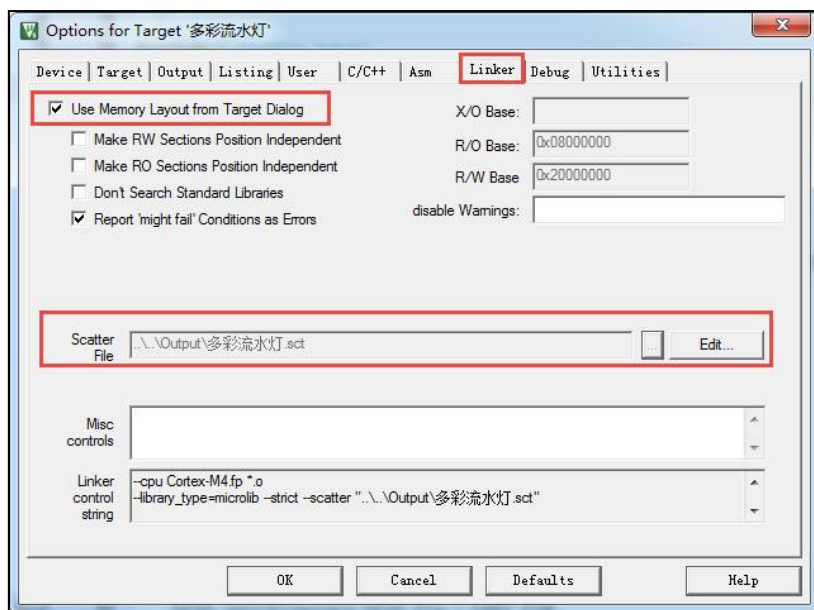


图 48-47 选择使用 MDK 生成的 sct 文件

该选项的译文为“是否使用 Target 对话框中的存储器分布配置”，勾选后，它会根据“Options for Target”对话框中的选项生成 sct 文件，这种情况下，即使我们手动打开它生成的 sct 文件编辑也是无效的，因为每次构建工程的时候，MDK 都会生成新的 sct 文件覆盖旧文件。该选项在 MDK 中是默认勾选的，若希望 MDK 使用我们手动编辑的 sct 文件构建工程，需要取消勾选，并通过 Scatter File 框中指定 sct 文件的路径，见图 48-48。

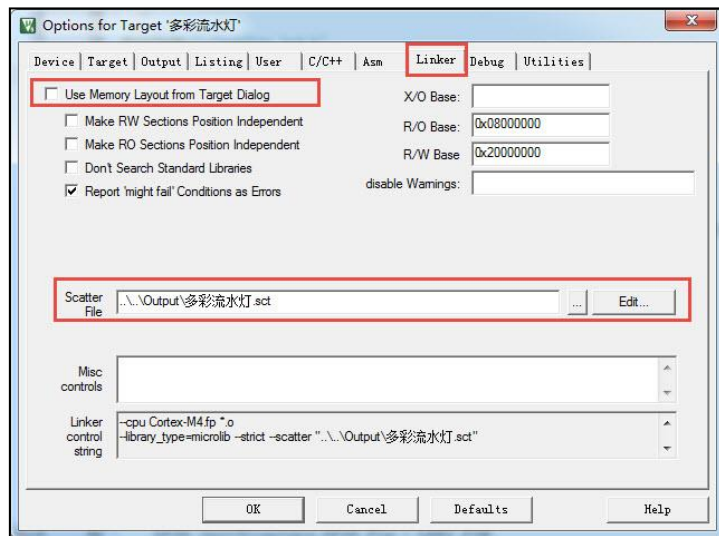


图 48-48 使用指定的 sct 文件构建工程

通过 Target 对话框控制存储器分配

若我们在 Linker 中勾选了“使用 Target 对话框的存储器布局”选项，那么“Options for Target”对话框中的存储器配置就生效了。主要配置是在 Device 标签页中选择芯片的类型，设定芯片基本的内部存储器信息以及在 Target 标签页中细化具体的存储器配置(包括外部存储器)，见图 48-49 及图 48-50。

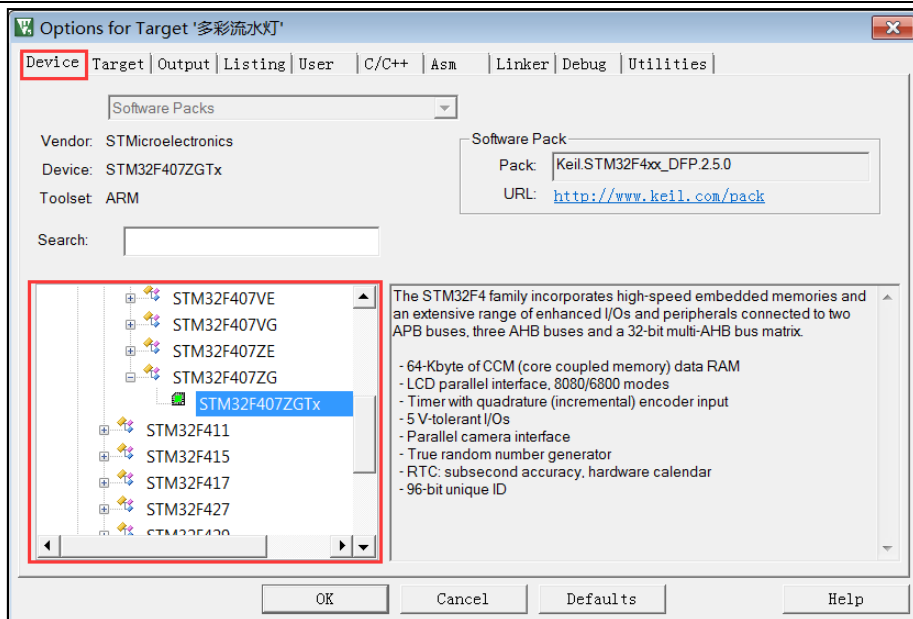


图 48-49 选择芯片类型

图中 Device 标签页中选中了芯片的型号为 STM32F429IGTx，选中后，在 Target 标签页中的存储器信息会根据芯片更新。

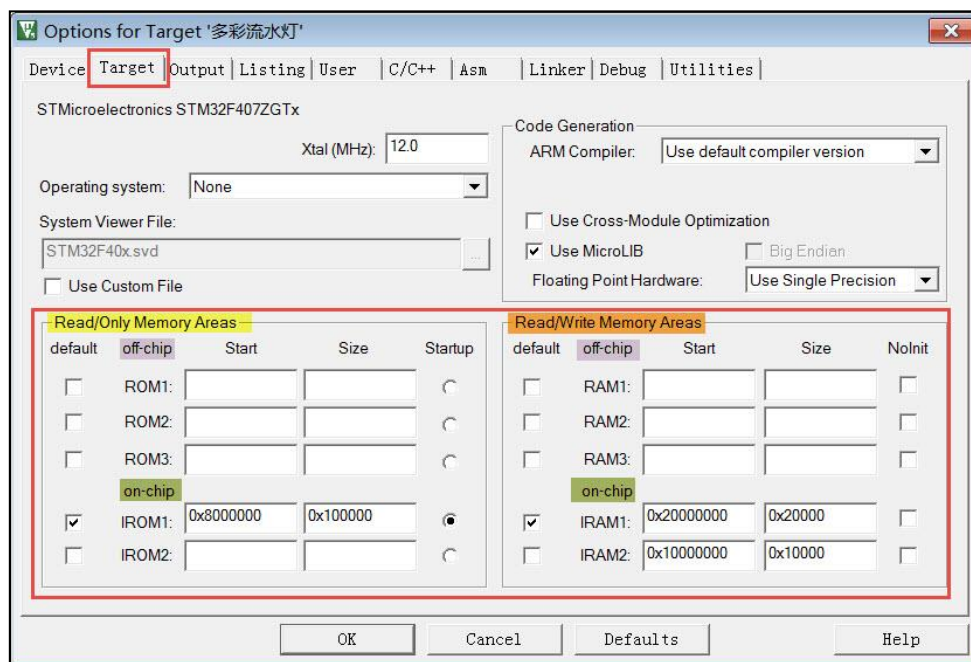


图 48-50 Target 对话框中的存储器分配

在 Target 标签页中存储器信息分成只读存储器(Read/Only Memory Areas)和可读写存储器(Read/Write Memory Areas)两类，即 ROM 和 RAM，而且它们又细分成了片外存储器(off-chip)和片内存储器(on-chip)两类。

例如，由于我们已经选定了芯片的型号，MDK 会自动根据芯片型号填充片内的 ROM 及 RAM 信息，其中的 IROM1 起始地址为 0x80000000，大小为 0x100000，正是该 STM32 型号的内部 FLASH 地址及大小；而 IRAM1 起始地址为 0x20000000，大小为 0x20000，正

是该 STM32 内部主 SRAM 的地址及大小。图中的 IROM1 及 IRAM1 前面都打上了勾，表示这个配置信息会被采用，若取消勾选，则该存储配置信息是不会被使用的。

在标签页中的 IRAM2 一栏默认也填写了配置信息，它的地址为 0x10000000，大小为 0x10000，这是 STM32F4 系列特有的内部 64KB 高速 SRAM(被称为 CCM)。当我们希望使用这部分存储空间的时候需要勾选该配置，另外要注意这部分高速 SRAM 仅支持 CPU 总线的访问，不能通过外设访问。

下面我们尝试修改 Target 标签页中的这些存储信息，例如，按照图 48-51 中的 1 配置，把 IRAM1 的基地址改为 0x20001000，然后编译工程，查看到工程的 sct 文件如代码清单 48-20 所示；当按照图 48-51 中的 2 配置时，同时使用 IRAM1 和 IRAM2，然后编译工程，可查看到工程的 sct 文件如代码清单 48-21 所示。

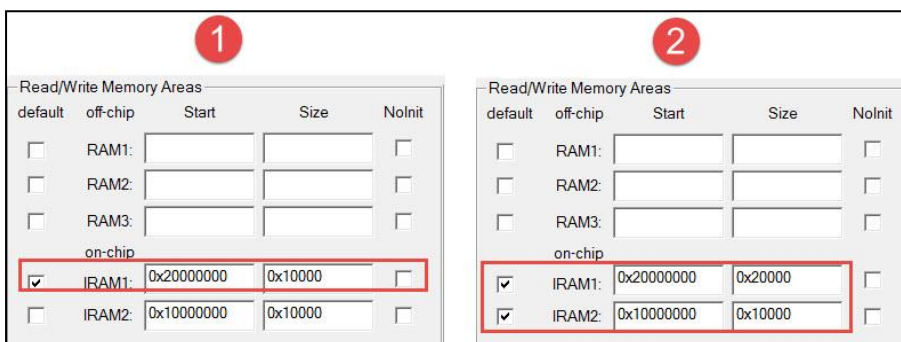


图 48-51 修改 IRAM1 的基地址及仅使用 IRAM2 的配置

代码清单 48-20 修改了 IRAM1 基地址后的 sct 文件内容

```
1 LR_IROM1 0x08000000 0x00100000 { ; load region size_region
2 ER_IROM1 0x08000000 0x00100000 { ; load address = execution address
3 *.o (RESET, +First)
4 *(InRoot$$Sections)
5 .ANY (+RO)
6 }
7 RW_IRAM1 0x20001000 0x00010000 { ; RW data
8 .ANY (+RW +ZI)
9 }
10 }
```

代码清单 48-21 仅使用 IRAM2 时的 sct 文件内容

```
1 LR_IROM1 0x08000000 0x00100000 { ; load region size_region
2 ER_IROM1 0x08000000 0x00100000 { ; load address = execution address
3 *.o (RESET, +First)
4 *(InRoot$$Sections)
5 .ANY (+RO)
6 }
7 RW_IRAM1 0x20000000 0x00020000 { ; RW data
8 .ANY (+RW +ZI)
9 }
10 RW_IRAM2 0x10000000 0x00010000 {
11 .ANY (+RW +ZI)
12 }
13 }
```

可以发现, sct 文件都根据 Target 标签页做出了相应的改变, 除了这种修改外, 在 Target 标签页上还控制同时使用 IRAM1 和 IRAM2、加入外部 RAM(如外接的 SRAM), 外部 FLASH 等。

控制文件分配到指定的存储空间

设定好存储器的信息后, 可以控制各个源文件定制到哪个部分存储器, 在 MDK 的工程文件栏中, 选中要配置的文件, 右键, 并在弹出的菜单中选择 “Options for File xxxx” 即可弹出一个文件配置对话框, 在该对话框中进行存储器定制, 见图 48-52。

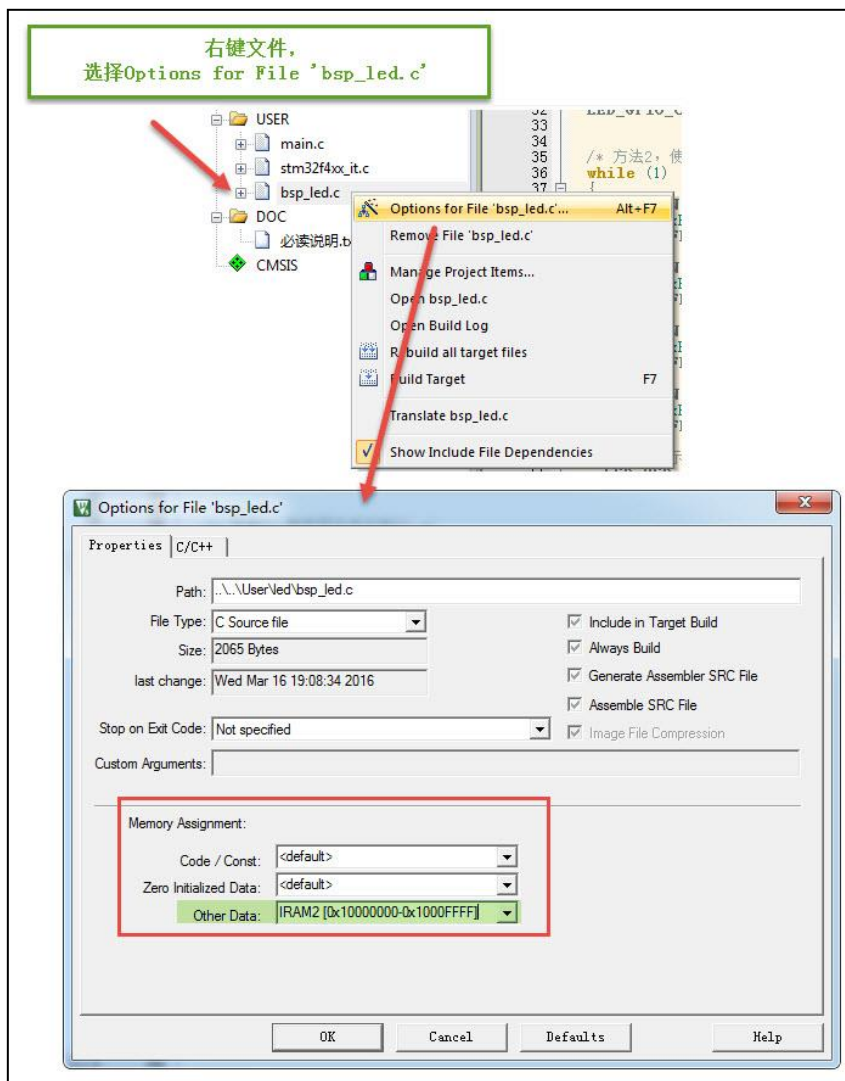


图 48-52 使用右键打开文件配置并把它的 RW 区配置成使用 IRAM2

在弹出的对话框中有一个 “Memory Assignment” 区域(存储器分配), 在该区域中可以针对文件的各种属性内容进行分配, 如 Code/Const 内容(RO)、Zero Initialized Data 内容(ZI-data)以及 Other Data 内容(RW-data), 点击下拉菜单可以找到在前面 Target 页面配置的 IROM1、IRAM1、IRAM2 等存储器。例如图中我们把这个 bsp_led.c 文件的 Other Data 属性的内容分配到了 IRAM2 存储器(在 Target 标签页中我们勾选了 IRAM1 及 IRAM2), 当在 bsp_led.c 文件定义了一些 RW-data 内容时(如初值非 0 的全局变量), 该变量将会被分配到

IRAM2 空间，配置完成后点击 OK，然后编译工程，查看到的 sct 文件内容见代码清单 48-22。

代码清单 48-22 修改 bsp_led.c 配置后的 sct 文件

```
1 LR_IROM1 0x08000000 0x00100000 { ; load region size_region
2 ER_IROM1 0x08000000 0x00100000 { ; load address = execution address
3 *.o (RESET, +First)
4 *(InRoot$$Sections)
5 .ANY (+RO)
6 }
7 RW_IRAM1 0x20000000 0x00020000 { ; RW data
8 .ANY (+RW +ZI)
9 }
10 RW_IRAM2 0x10000000 0x00010000 {
11 bsp_led.o (+RW)
12 .ANY (+RW +ZI)
13 }
14 }
```

可以看到在 sct 文件中的 RW_IRAM2 执行域中增加了一个选择 bsp_led.o 中 RW 内容的语句。

类似地，我们还可以设置某些文件的代码段被存储到特定的 ROM 中，或者设置某些文件使用的 ZI-data 或 RW-data 存储到外部 SRAM 中(控制 ZI-data 到外部 SRAM 时注意还需要修改启动文件设置堆栈对应的地址，原启动文件中的地址是指向内部 SRAM 的)。

虽然 MDK 的这些存储器配置选项很方便，但有很多高级的配置还是需要手动编写 sct 文件实现的，例如 MDK 选项中的内部 ROM 选项最多只可以填充两个选项位置，若想把内部 ROM 分成多片地址管理就无法实现了；另外 MDK 配置可控的最小粒度为文件，若想控制特定的节区也需要直接编辑 sct 文件。

接下来我们将讲解几个实验，通过编写 sct 文件定制存储空间。

48.5 实验：自动分配变量到外部 SRAM 空间

由于内存管理对应用程序非常重要，若修改 sct 文件，不使用默认配置，对工程影响非常大，容易导致出错，所以我们使用两个实验配置来讲解 sct 文件的应用细节，希望您学习后不仅知其然而且知其所以然，清楚地了解修改后对应用程序的影响，还可以举一反三根据自己的需求进行存储器定制。

在本书前面的外部 SRAM 实验中，当我们需要读写外部 SRAM 存储的内容时，需要使用指针或者 __attribute__((at(具体地址))) 来指定变量的位置，当有多个这样的变量时，就需要手动计算地址空间了，非常麻烦。在本实验中我们将修改 sct 文件，让链接器自动分配全局变量到外部 SRAM 的地址并进行管理，使得利用外部 SRAM 的空间就跟内部 SRAM 一样简单。

48.5.1 硬件设计

本小节中使用到的硬件跟“扩展外部 SRAM”实验中的一致，若不了解，请参考该章节的原理图说明。

48.5.2 软件设计

本小节中提供的例程名为“SCT 文件应用—自动分配变量到外部 SRAM”，学习时请打开该工程来理解，该工程是基于“扩展外部 SRAM”实验改写而来的。

为方便讲解，本实验直接使用手动编写的 sct 文件，所以在 MDK 的“Options for Target->Linker->Use Memory Layout from Target Dialog”选项被取消勾选，取消勾选后可直接点击“Edit”按钮编辑工程的 sct 文件，也可到工程目录下打开编辑，见图 48-53。

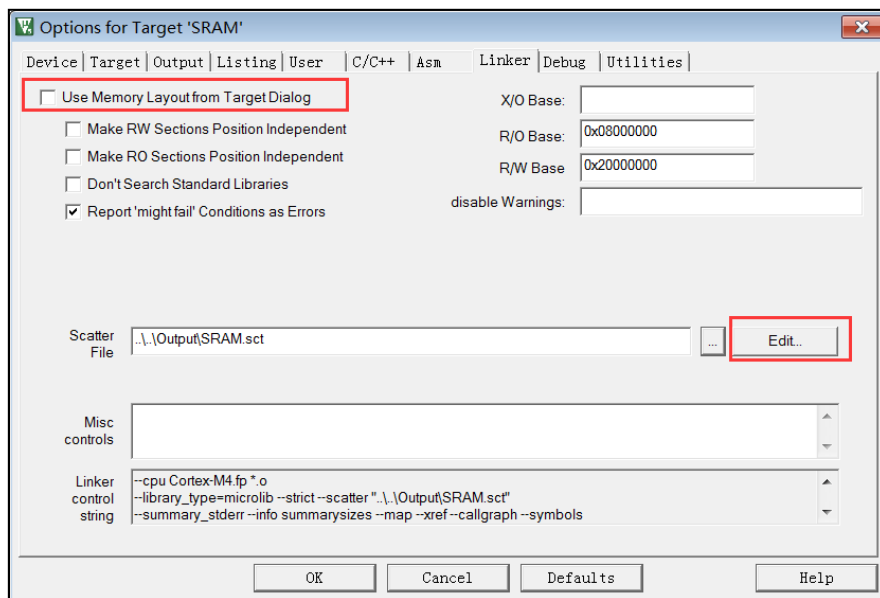


图 48-53 使用手动编写的 sct 文件

取消了这个勾选后，在 MDK 的 Target 对话框及文件配置的存储器分布选项都会失效，仅以 sct 文件中的为准，更改对话框及文件配置选项都不会影响 sct 文件的内容。

1. 编程要点

- (1) 修改启动文件，在__main 执行之前初始化外部 SRAM；
- (2) 在 sct 文件中增加外部 SRAM 空间对应的执行域；
- (3) 使用节区选择语句选择要分配到外部 SRAM 的内容；
- (4) 编写测试程序，编译正常后，查看 map 文件的空间分配情况。

2. 代码分析

在__main 之前初始化 SDRAM

在前面讲解 ELF 文件格式的小节中我们了解到，芯片启动后，会通过__main 函数调用分散加载代码__scatterload，分散加载代码会把存储在 FLASH 中的 RW-data 复制到 RAM 中，然后在 RAM 区开辟一块 ZI-data 的空间，并将其初始化为 0 值。因此，为了保证在程序中定义到外部 SRAM 中的变量能被正常初始化，我们需要在系统执行分散加载代码之前使外部 SRAM 存储器正常运转，使它能够正常保存数据。

在本来的“扩展外部 SRAM”工程中，我们使用 FSMC_SRAM_Init 函数初始化外部 SRAM，且该函数在 main 函数里才被调用，所以在外部 SRAM 正常运转之前，分散加载过程复制到外部 SRAM 中的数据都丢失了，因而需要在初始化外部 SRAM 之后，需要重新给变量赋值才能正常使用(即定义变量时的初值无效，在调用外部 SRAM_Init 函数之后的赋值才有效)。

为了解决这个问题，可修改工程的 startup_stm32f40xx.s 启动文件，见代码清单 48-23。

代码清单 48-23 修改启动文件中的 Reset_handler 函数(startup_stm32f40xx.s 文件)

```
1 ; Reset handler
2 Reset_Handler      PROC
3                     EXPORT Reset_Handler             [WEAK]
4                     IMPORT SystemInit
5                     IMPORT __main
6
7                     ;从外部文件引入声明
8                     IMPORT FSMC_SRAM_Init
9
10                    LDR      R0, =SystemInit
11                    BLX      R0
12
13                    ;在__main 之前调用 SDRAM_Init 进行初始化
14                    LDR      R0, =FSMC_SRAM_Init
15                    BLX      R0
16
17                    LDR      R0, =__main
18                    BX       R0
19                    ENDP
```

在原来的启动文件中我们增加了上述加粗表示的代码，增加的代码中使用到汇编语法的 IMPOR 引入在 bsp_sram.c 文件中定义的 FSMC_SRAM_Init 函数，接着使用 LDR 指令加载函数的代码地址到寄存器 R0，最后使用 BLX R0 指令跳转到 FSMC_SRAM_Init 的代码地址执行。

以上代码实现了 Reset_handler 在执行__main 函数前先调用了我们自定义的 FSMC_SRAM_Init 函数，从而为分散加载代码准备好正常的硬件工作环境。

sct 文件初步应用

接下来修改 sct 文件，控制使得在 C 源文件中定义的全局变量都自动由链接器分配到外部 SRAM 中，见代码清单 48-24。

代码清单 48-24 配置 sct 文件(SRAM.sct 文件)

```
1 ; *****
2 ; *** Scatter-Loading Description File generated by uVision ***
3 ; *****
4
5 LR_IROM1 0x08000000 0x00100000 {      ; 加载域
6   ER_IROM1 0x08000000 0x00100000 {      ; 加载地址 = 执行地址
7     *.o (RESET, +First)
8     *(InRoot$$Sections)
9     .ANY (+RO)
10  }
11
12
13 RW_IRAM1 0x20000000 0x00020000 {      ; 内部 SRAM
14   *.o (STACK)          ;选择 STACK 节区, 栈
```

```

15  stm32f4xx_rcc.o(+RW)           ;选择 stm32f4xx_rcc 的 RW 内容
16  .ANY (+RW +ZI)                 ;其余的 RW/ZI-data 都分配到这里
17  }
18
19  RW_ERAM1 0x6C000000 0x00100000 { ; 外部 SRAM
20
21  .ANY (+RW +ZI)                 ;其余的 RW/ZI-data 都分配到这里
22  }
23 }

```

加粗部分是本例子中增加的代码，我们从后面开始，先分析比较简单的外部 SRAM 执行域部分。

❑ RW_ERAM1 0x6C000000 0x00100000{}

RW_ERAM1 是我们配置的外部 SRAM 执行域。该执行域的名字是可以随便取的，最重要的是它的基地址及空间大小，这两个值与我们实验板配置的外部 SRAM 基地址及空间大小一致，所以该执行域会被映射到外部 SRAM 的空间。在 RW_ERAM1 执行域内部，它使用“.ANY(+RW +ZI)”语句，选择了所有的 RW/ZI 类型的数据都分配到这个外部 SRAM 区域，所以我们在工程中的 C 文件定义全局变量时，它都会被分配到这个外部 SRAM 区域。

❑ RW_IRAM1 执行域

RW_IRAM1 是 STM32 内部 SRAM 的执行域。我们在默认配置中增加了“*.o(STACK)及 stm32f4xx_rcc.o(+RW)”语句。本来上面配置外部 SRAM 执行域后已经达到使全局变量分配的目的，为何还要修改原内部 SRAM 的执行域呢？

这是由于我们在__main 之前调用的 FSMC_SRAM_Init 函数调用了很多库函数，且这些函数内部定义了一些局部变量，而函数内的局部变量是需要分配到“栈”空间 (STACK)，见图 48-54，查看静态调用图文件“SRAM.htm”可了解它使用了多少栈空间以及调用了哪些函数。

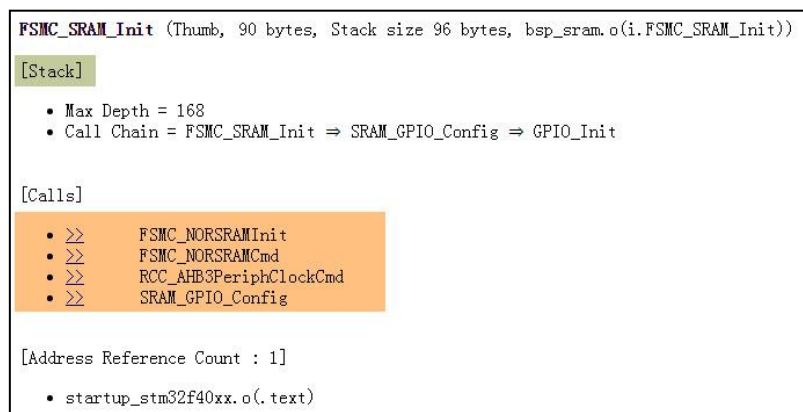


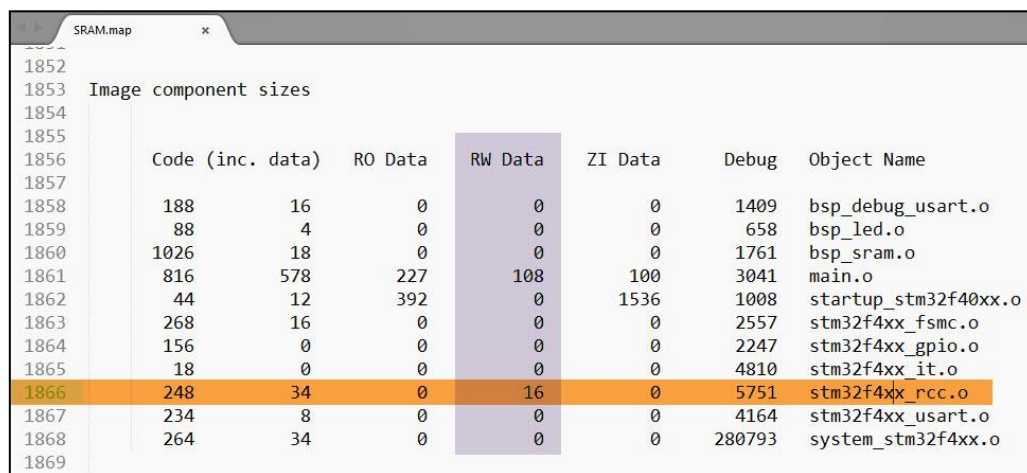
图 48-54 FSMC_SRAM_Init 的调用说明(SRAM.htm 文件)

从文件中可了解到 FSMC_SRAM_Init 使用的 STACK 的深度为 168 字节，它调用了 FMC_NORSRAMInit、FMC_NORSRAMCmd、RCC_AHB3PeriphClockCmd 及 SRAM_GPIO_Config 函数。由于它使用了栈空间，所以在 FSMC_SRAM_Init 函数执行之前，栈空间必须要被准备好，然而在 FSMC_SRAM_Init 函数执行之前，外部 SRAM 芯片却并未正常工作，这样的矛盾导致栈空间不能被分配到外部 SRAM。

虽然内部 SRAM 的执行域 RW_IRAM1 及 SDRAM 执行域 RW_ERAM1 中都使用 “.ANY(+RW +ZI)” 语句选择了所有 RW 及 ZI 属性的内容，但对于符合两个相同选择语句的内容，链接器会优先选择使用空间较大的执行域，即这种情况下只有当 SDRAM 执行域的空间使用完了，RW/ZI 属性的内容才会被分配到内部 SRAM。

所以在大部分情况下，内部 SRAM 执行域中的 “.ANY(+RW +ZI)” 语句是不起作用的，而栈节区(STACK)又属于 ZI-data 类，如果我们的内部 SRAM 执行域还是按原来的默认配置的话，栈节区会被分配到外部 SRAM，导致出错。为了避免这个问题，我们把栈节区使用 “*.o(STACK)” 语句分配到内部 SRAM 的执行域。

增加 “stm32f4xx_rcc.o(+RW)” 语句是因为 FSMC_SRAM_Init 函数调用了 stm32f4xx_rcc.c 文件中的 RCC_AHB3PeriphClockCmd 函数，而查看 map 文件后了解到 stm32f4xx_rcc.c 定义了一些 RW-data 类型的变量，见图 48-55。不管这些数据是否在 FSMC_SRAM_Init 调用过程中使用到，保险起见，我们直接把这部分内容也分配到内部 SRAM 的执行区。



	Code (inc. data)	RO Data	RW Data	ZI Data	Debug	Object Name
1852						
1853	Image component sizes					
1854						
1855						
1856						
1857						
1858	188	16	0	0	1409	bsp_debug_usart.o
1859	88	4	0	0	658	bsp_led.o
1860	1026	18	0	0	1761	bsp_sram.o
1861	816	578	227	108	3041	main.o
1862	44	12	392	0	1536	startup_stm32f40xx.o
1863	268	16	0	0	2557	stm32f4xx_fsmc.o
1864	156	0	0	0	2247	stm32f4xx_gpio.o
1865	18	0	0	0	4810	stm32f4xx_it.o
1866	248	34	0	16	5751	stm32f4xx_rcc.o
1867	234	8	0	0	4164	stm32f4xx_usart.o
1868	264	34	0	0	280793	system_stm32f4xx.o
1869						

图 48-55 SRAM.map 文件中查看到 stm32f4xx_rcc.o 文件的 RW-data 使用统计信息

变量分配测试及结果

接下来查看本工程中的 main 文件，它定义了各种变量测试空间分配，见代码清单 48-25。

代码清单 48-25 main 文件

```
1 //定义变量到外部 SRAM
2 uint32_t testValue = 7 ;
3 //定义变量到外部 SRAM
4 uint32_t testValue2 = 0;
5
6
7
8 //定义数组到外部 SRAM
9 uint8_t testGrup[100] = {0};
10 //定义数组到外部 SRAM
11 uint8_t testGrup2[100] = {1,2,3};
12
13
14 /*本实验中的 sct 配置，堆区可能工作不正常，
15 使用 malloc 无法得到正常的地址，不推荐在实际工程应用*/
16 /*另一种我们推荐的配置请参考教程中的说明*/
```

```
17
18 /*
19  * 函数名: main
20  * 描述   : 主函数
21  * 输入   : 无
22  * 输出   : 无
23  */
24 int main(void)
25 {
26     uint32_t inerTestValue =10;
27
28
29     LED_GPIO_Config();
30     LED_BLUE;
31
32     /* 配置串口1 为: 115200 8-N-1 */
33     Debug_USART_Config();
34
35     //FSMC_SRAM_Init 已经在启动文件的 Reset_handler 中调用, 进入 main 之前已经完成初始化*/
36 //    FSMC_SRAM_Init();
37
38     printf("\r\nSCT 文件应用—自动分配变量到外部 SRAM 实验\r\n");
39
40     printf("\r\n 使用" uint32_t inerTestValue =10; "语句定义的局部变量: \r\n");
41     printf("结果: 它的地址为: 0x%x,变量值为: %d\r\n",
42           (uint32_t)&inerTestValue,inerTestValue);
43
44     printf("\r\n 使用"uint32_t testValue  =7 ;"语句定义的全局变量: \r\n");
45     printf("结果: 它的地址为: 0x%x,变量值为: %d\r\n",
46           (uint32_t)&testValue,testValue);
47
48     printf("\r\n 使用"uint32_t testValue2  =0 ; "语句定义的全局变量: \r\n");
49     printf("结果: 它的地址为: 0x%x,变量值为: %d\r\n",
50           (uint32_t)&testValue2,testValue2);
51
52
53     printf("\r\n 使用"uint8_t testGrup[100]  ={0};"语句定义的全局数组: \r\n");
54     printf("结果: 它的地址为: 0x%x,变量值为: %d,%d,%d\r\n",
55           (uint32_t)&testGrup,testGrup[0],testGrup[1],testGrup[2]);
56
57     printf("\r\n 使用"uint8_t testGrup2[100]  ={1,2,3};"语句定义的全局数组: \r\n");
58     printf("结果: 它的地址为: 0x%x,变量值为: %d, %d,%d\r\n",
59           (uint32_t)&testGrup2,testGrup2[0],testGrup2[1],testGrup2[2]);
60
61
62     /*本实验中的 sct 配置, 堆区可能工作不正常,
63     使用 malloc 无法得到正常的地址, 不推荐在实际工程应用*/
64     /*另一种我们推荐的配置请参考教程中的说明*/
65
66     uint32_t * pointer = (uint32_t*)malloc(sizeof(uint32_t)*3);
67     if (pointer != NULL) {
68         *(pointer)=1;
69         *(++pointer)=2;
70         *(++pointer)=3;
71
72     printf("\r\n 使用" uint32_t *pointer = (uint32_t*)malloc(sizeof(uint32_t)*3); "动态分配的变量\r\n");
73     printf("\r\n 定义后的操作为:
74           \r\n*(pointer++)=1;\r\n*(pointer++)=2;\r\n*pointer=3;\r\n\r\n");
75     printf("结果: 操作后它的地址为: 0x%x, 查看变量值操作: \r\n",
76           (uint32_t)pointer);
77     printf("* (pointer--)=%d, \r\n",*(pointer--));
78     printf("* (pointer--)=%d, \r\n",*(pointer--));
79     printf("* (pointer)=%d, \r\n",*(pointer));
80
81     free(pointer);
```

```

80     } else {
81         printf("\r\n 使用 malloc 动态分配变量出错!! \r\n");
82     }
83
84     LED_BLUE;
85
86
87     while (1);
88 }

```

代码中定义了局部变量、初值非 0 的全局变量及数组、初值为 0 的全局变量及数组以及动态分配内存，并把它们的值和地址通过串口打印到上位机，通过这些变量，我们可以测试栈、ZI/RW-data 及堆区的变量是否能正常分配。构建工程后，首先查看工程的 map 文件观察变量的分配情况，见图 48-56 及图 48-57。

SRAM.map		地址	Number	
1741	Region\$\$Table\$\$Limit	0x08001474	0	anon\$\$obj.o(Region\$\$Table)
1742	__initial_sp	0x20000410	0	startup_stm32f40xx.o(STACK)
1743	testValue	0x6c000000	4	main.o(.data)
1744	testValue2	0x6c000004	4	main.o(.data)
1745	testGroup2	0x6c000008	100	main.o(.data)
1746	__stdout	0x6c00006c	4	stdout.o(.data)
1747	__microlib_freelist	0x6c000070	4	mvars.o(.data)
1748	__microlib_freelist_initialised	0x6c000074	4	mvars.o(.data)
1749	testGroup	0x6c000078	100	main.o(.bss)
1750	__heap_base	0x6c0000e0	0	startup_stm32f40xx.o(HEAP)
1751	__heap_limit	0x6c0002e0	0	startup_stm32f40xx.o(HEAP)
1752				

图 48-56 在 map 文件中查看工程的存储分布 1(SRAM.map 文件)

SRAM.map

✕

1831 Execution Region **RW_IRAM1** (Base: 0x20000000, Size: 0x00000410, Max: 0x00020000, ABSOLUTE)

1832

1833

1834

1835

1836

1837

1838

1839

1840 Execution Region **RW_ERAM1** (Base: 0x6c000000, Size: 0x000002e0, Max: 0x00100000, ABSOLUTE)

1841

1842

1843

1844

1845

1846

1847

1848

1849

1850

1851

1852

1853

1854

1855

1856

1857

1858

1859

1860

1861

1862

1863

1864

1865

Image component sizes

</

图 48-57 在 map 文件中查看工程的存储分布 2(SRAM.map 文件)

从 map 文件中，可看到 stm32f4xx_rcc 的 RW-data 及栈空间节区(STACK)都被分配到了 RW_IRAM1 区域，即 STM32 的内部 SRAM 空间中；而 main 文件中定义的 RW-data、

ZI-data 以及堆空间节区(HEAP)都被分配到了 RW_ERAM1 区域,即我们扩展的外部 SRAM 空间中,看起来一切都与我们的 sct 文件配置一致了。(堆空间属于 ZI-data,由于没有像控制栈节区那样指定到内部 SRAM,所以它被默认分配到外部 SRAM 空间了;在 main 文件中我们定义了一个初值为 0 的全局变量 testValue2 及初值为 0 的数组 testGrup[100],它们本应占用的是 104 字节的 ZI-data 空间,但在 map 文件中却查看到它仅使用了 100 字节的 RW-data 空间,这是因为链接器把 testValue2 分配为 RW-data 类型的变量了,这是链接器本身的特性,它对像 testGrup[100]这样的数组才优化作为 ZI-data 分配,这不是我们 sct 文件导致的空间分配错误。)

接下来把程序下载到实验板进行测试,串口打印的调试信息如图 48-58。

```
SCT文件应用——自动分配变量到外部SRAM实验

使用“uint32_t innerTestValue =10;”语句定义的局部变量:
结果: 它的地址为: 0x20000404, 变量值为: 10

使用“uint32_t testValue =7 ;”语句定义的全局变量:
结果: 它的地址为: 0x6c000000, 变量值为: 7

使用“uint32_t testValue2 =0 ;”语句定义的全局变量:
结果: 它的地址为: 0x6c000004, 变量值为: 0

使用“uint8_t testGrup[100] = {0};”语句定义的全局数组:
结果: 它的地址为: 0x6c000078, 变量值为: 0, 0, 0

使用“uint8_t testGrup2[100] = {1, 2, 3};”语句定义的全局数组:
结果: 它的地址为: 0x6c000008, 变量值为: 1, 2, 3

使用“uint32_t *pointer = (uint32_t*)malloc(sizeof(uint32_t)*3);”动态分配的变量

定义后的操作为:
*(pointer++)=1;
*(pointer++)=2;
*pointer=3;

结果: 操作后它的地址为: 0x6c0000f0, 查看变量值操作:
*(pointer--)=3,
*(pointer--)=2,
*(pointer)=1,
```

图 48-58 空间分配实验实测结果

从调试信息中可发现,变量都定义到了正确的位置,如内部变量定义在内部 SRAM 的栈区域,全局变量定义到了外部 SRAM 的区域。

经过我的测试,曾出现过堆空间分配错误的情况,该情况下即使在 sct 文件中使用“*.o(HEAP)”语句指定堆区到内部 SRAM 或外部 SRAM 区域,都无法正常使用 malloc 分配空间。另外,由于外部 SRAM 的读写速度比内部 SRAM 的速度慢,所以我们更希望默认定义的变量先使用内部 SRAM,当它的空间使用完毕后再把变量分配到外部 SRAM。

在下一小节中我们将改进 sct 的文件配置,解决这两个问题。

48.6 实验：优先使用内部 SRAM 并把堆区分配到外部 SRAM 空间

本实验使用另一种方案配置 sct 文件，使得默认情况下优先使用内部 SRAM 空间，在需要的时候使用一个关键字指定变量存储到外部 SRAM，另外，我们还把系统默认的堆空间(HEAP)映射到外部 SRAM，从而可以使用 C 语言标准库的 malloc 函数动态从外部 SRAM 中分配空间，利用标准库进行外部 SRAM 的空间内存管理。

48.6.1 硬件设计

本小节中使用到的硬件跟“扩展外部 SRAM”实验中的一致，若不了解，请参考该章节的原理图说明。

48.6.2 软件设计

本小节中提供的例程名为“SCT 文件应用—优先使用内部 SRAM 并把堆分配到外部 SRAM 空间”，学习时请打开该工程来理解，该工程从上一小节的实验改写而来的，同样地，本工程只使用手动编辑的 sct 文件配置，不使用 MDK 选项配置，在“Options for Target->linker”的选项见图 48-53。

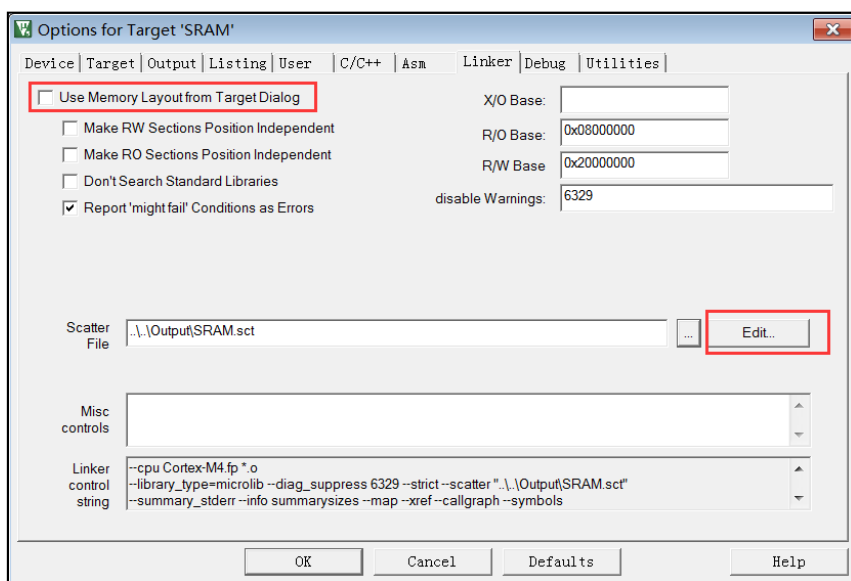


图 48-59 使用手动编写的 sct 文件

取消了这个默认的“Use Memory Layout from Target Dialog”勾选后，在 MDK 的 Target 对话框及文件配置的存储器分布选项都会失效，仅以 sct 文件中的为准，更改对话框及文件配置选项都不会影响 sct 文件的内容。

1. 编程要点

(1) 修改启动文件，在 __main 执行之前初始化外部 SRAM；

- (2) 在 sct 文件中增加外部 SRAM 空间对应的执行域；
- (3) 在外部 SRAM 中的执行域中选择一个自定义节区“EXRAM”；
- (4) 使用__attribute__关键字指定变量分配到节区“EXRAM”；
- (5) 使用宏封装__attribute__关键字，简化变量定义；
- (6) 根据需要，把堆区分配到内部 SRAM 或外部 SRAM 中；
- (7) 编写测试程序，编译正常后，查看 map 文件的空间分配情况。

2. 代码分析

在__main 之前初始化外部 SRAM

同样地，为了使定义到外部 SRAM 的变量能被正常初始化，需要修改工程 startup_stm32f40xx.s 启动文件中的 Reset_handler 函数，在__main 函数之前调用 FSMC_SRAM_Init 函数使外部 SRAM 硬件正常运转，见代码清单 48-23。

代码清单 48-26 修改启动文件中的 Reset_handler 函数(startup_stm32f40xx.s 文件)

```
1 ; Reset handler
2 Reset_Handler    PROC
3                 EXPORT Reset_Handler            [WEAK]
4                 IMPORT SystemInit
5                 IMPORT __main
6
7                 ;从外部文件引入声明
8                 IMPORT FSMC_SRAM_Init
9
10                LDR    R0, =SystemInit
11                BLX    R0
12
13                ;在__main 之前调用 SDRAM_Init 进行初始化
14                LDR    R0, = FSMC_SRAM_Init
15                BLX    R0
16
17                LDR    R0, =__main
18                BX     R0
19                ENDP
```

它与上一小节中的改动一样，当芯片上电运行 Reset_handler 函数时，在执行__main 函数前先调用了我们自定义的 FSMC_SRAM_Init 函数，从而为分散加载代码准备好正常的硬件工作环境。

sct 文件配置

接下来分析本实验中的 sct 文件配置与上一小节有什么差异，见代码清单 48-27。

代码清单 48-27 本实验的 sct 文件内容(SRAM.sct)

```
1 ; *****
2 ; *** Scatter-Loading Description File generated by uVision ***
3 ; *****
4 LR_IROM1 0x08000000 0x00100000 { ; load region size_region
5   ER_IROM1 0x08000000 0x00100000 { ; load address = execution address
6     *.o (RESET, +First)
7     *(InRoot$$Sections)
8     .ANY (+RO)
9   }
10
11
12 RW_IRAM1 0x20000000 0x00030000 { ; 内部 SRAM
```

```
13 .ANY (+RW +ZI)      ;其余的 RW/ZI-data 都分配到这里
14 }
15
16 RW_ERAM1 0x6C000000 0x00100000 { ; 外部 SRAM
17 *.o(HEAP)              ;选择堆区
18 .ANY (EXRAM)           ;选择 EXRAM 节区
19 }
20 }
```

本实验的 sct 文件中对内部 SRAM 的执行域保留了默认配置，没有作任何改动，新增了一个外部 SRAM 的执行域，并且使用了“*.o(HEAP)”语句把堆区分配到了外部 SRAM 空间，使用“.ANY(EXRAM)”语句把名为“EXRAM”的节区分配到外部 SRAM 空间。

这个“EXRAM”节区是由我们自定义的，在语法上就跟在 C 文件中定义全局变量类似，只要它跟工程中的其它原有节区名不一样即可。有了这个节区选择配置，当我们需要定义变量到外部 SRAM 时，只需要指定该变量分配到该节区，它就会被分配到外部 SRAM 空间。

本实验中的 sct 配置就是这么简单，接下来直接使用就可以了。

指定变量分配到节区

当我们需要把变量分配到外部 SRAM 时，需要使用__attribute__关键字指定节区，它的语法见代码清单 48-28。

代码清单 48-28 指定变量定义到某节区的语法

```
1 //使用 __attribute__ 关键字定义指定变量定义到某节区
2 //语法：变量定义 __attribute__((section ("节区名"))) = 变量值;
3 uint32_t testValue __attribute__((section ("EXRAM"))) = 7 ;
4
5 //使用宏封装
6 //设置变量定义到“EXRAM”节区的宏
7 #define __EXRAM __attribute__((section ("EXRAM")))
8
9 //使用该宏定义变量到外部 SRAM
10 uint32_t testValue __EXRAM = 7 ;
11
```

上述代码介绍了基本的指定节区语法：“变量定义 __attribute__((section ("节区名"))) = 变量值;”，它的主体跟普通的 C 语言变量定义语法无异，在赋值“=”号前(可以不赋初值)，加了个“__attribute__((section ("节区名")))”描述它要分配到的节区。本例中的节区名为“EXRAM”，即我们在 sct 文件中选择分配到外部 SRAM 执行域的节区，所以该变量就被分配到外部 SRAM 中了。

由于“__attribute__”关键字写起来比较繁琐，我们可以使用宏定义把它封装起来，简化代码。本例中我们把指定到“EXRAM”的描述语句“__attribute__((section ("EXRAM")))”封装成了宏“__EXRAM”，应用时只需要使用宏的名字替换原来“__attribute__”关键字的位置即可，如“uint32_t testValue __EXRAM = 7;”。有 51 单片机使用经验的读者会发现，这种变量定义方法就跟使用 keil 51 特有的关键字“xdata”定义变量到外部 RAM 空间差不多。

类似地，如果工程中还使用了其它存储器也可以用这样的方法实现变量分配，例如 STM32 的高速内部 SRAM(CCM)，可以在 sct 文件增加该高速 SRAM 的执行域，然后在执

行域中选择一个自定义节区，在工程源文件中使用“__attribute__”关键字指定变量到该节区，就可以把变量分配到高速内部 SRAM 了。

根据我们 sct 文件的配置，如果定义变量时没有指定节区，它会默认优先使用内部 SRAM，把变量定义到内部 SRAM 空间，而且由于局部变量是属于栈节区(STACK)，它不能使用“__attribute__”关键字指定节区。在本例中的栈节区被分配到内部 SRAM 空间。

变量分配测试及结果

接下来查看本工程中的 main 文件，它定义了各种变量测试空间分配，见代码清单 48-25。

代码清单 48-29 main 文件

```
1
2 //设置变量定义到“EXRAM”节区的宏
3 #define __EXRAM __attribute__((section("EXRAM")))
4
5
6 //定义变量到外部 SRAM
7 uint32_t testValue __EXRAM =7 ;
8 //上述语句等效于:
9 //uint32_t testValue __attribute__((section("EXRAM"))) =7 ;
10
11 //定义变量到 SRAM
12 uint32_t testValue2 =7 ;
13
14
15
16 //定义数组到外部 SRAM
17 uint8_t testGrup[3] __EXRAM = {1,2,3};
18 //定义数组到 SRAM
19 uint8_t testGrup2[3] = {1,2,3};
20
21
22
23
24 /**
25  * @brief 主函数
26  * @param 无
27  * @retval 无
28  */
29 int main(void)
30 {
31     uint32_t inerTestValue =10;
32     //FSMC_SRAM_Init 已经在启动文件的 Reset_handler 中调用,
33     进入 main 之前已经完成初始化*/
34     //     FSMC_SRAM_Init();
35
36     /* LED 端口初始化 */
37     LED_GPIO_Config();
38
39     /* 初始化串口 */
40     Debug_USART_Config();
41
42     printf("\r\nSCT 文件应用—自动分配变量到外部 SRAM 实验\r\n");
43
44     printf("\r\n使用“ uint32_t inerTestValue =10; ”语句定义的局部变量: \r\n");
45     printf("结果: 它的地址为: 0x%x, 变量值为: %d\r\n",
46           (uint32_t)&inerTestValue, inerTestValue);
47
48     printf("\r\n使用“uint32_t testValue __EXRAM =7 ;”语句定义的全局变量: \r\n");
```



```

49     printf("结果：它的地址为：0x%x,变量值为：%d\r\n",
50           (uint32_t)&testValue,testValue);
51
52     printf("\r\n 使用`uint32_t testValue2  =7 ;`语句定义的全局变量：\r\n");
53     printf("结果：它的地址为：0x%x,变量值为：%d\r\n",
54           (uint32_t)&testValue2,testValue2);
55
56
57     printf("\r\n 使用`uint8_t testGrup[3] __EXRAM ={1,2,3};`语句定义的全局数组：\r\n");
58     printf("结果：它的地址为：0x%x,变量值为：%d,%d,%d\r\n",
59           (uint32_t)&testGrup,testGrup[0],testGrup[1],testGrup[2]);
60
61     printf("\r\n 使用`uint8_t testGrup2[3] ={1,2,3};`语句定义的全局数组：\r\n");
62     printf("结果：它的地址为：0x%x,变量值为：%d,%d,%d\r\n",
63           (uint32_t)&testGrup2,testGrup2[0],testGrup2[1],testGrup2[2]);
64
65
66     /*使用 malloc 从外部 SRAM 中分配空间*/
67     uint32_t *pointer = (uint32_t*)malloc(sizeof(uint32_t)*3);
68
69     if (pointer != NULL) {
70         *(pointer)=1;
71         *(++pointer)=2;
72         *(++pointer)=3;
73
74     printf("\r\n 使用`uint32_t *pointer = (uint32_t*)malloc(sizeof(uint32_t)*3);`动态分配的变量\r\n");
75         printf("\r\n 定义后的操作为：\r\n*(pointer++)=1;\r\n*(pointer++)=2;\r\n*(pointer++)=3;\r\n\r\n");
76         printf("结果：操作后它的地址为：0x%x,查看变量值操作：\r\n", (uint32_t)pointer);
77         printf("(*(pointer--)=%d, \r\n", *(pointer--));
78         printf("(*(pointer--)=%d, \r\n", *(pointer--));
79         printf("(*(pointer)=%d, \r\n", *(pointer));
80
81         free(pointer);
82     } else {
83         printf("\r\n 使用 malloc 动态分配变量出错!!! \r\n");
84     }
85
86     /*蓝灯亮*/
87     LED_BLUE;
88
89
90     while (1);
91
92
93 }

```

代码中定义了普通变量、指定到 EXRAM 节区的变量并使用动态分配内存，还把它们的值和地址通过串口打印到上位机，通过这些变量，我们可以检查变量是否能正常分配。

构建工程后，查看工程的 map 文件观察变量的分配情况，见图 48-57。

SRAM.map		x		
1741	Region\$\$Table\$\$Base	0x08001430	Number	0 anon\$\$obj.o(Rregion\$\$Table)
1742	Region\$\$Table\$\$Limit	0x08001470	Number	0 anon\$\$obj.o(Rregion\$\$Table)
1743	testGrup2	0x20000010	Data	3 main.o(.data)
1744	testValue2	0x20000014	Data	4 main.o(.data)
1745	__stdout	0x20000018	Data	4 stdout.o(.data)
1746	__microlib_freelist	0x2000001c	Data	4 mvars.o(.data)
1747	__microlib_freelist_initialised	0x20000020	Data	4 mvars.o(.data)
1748	__initial_sp	0x20000428	Data	0 startup_stm32f40xx.o(STACK)
1749	testGrup	0x6c000000	Data	3 main.o(EXRAM)
1750	testValue	0x6c000004	Data	4 main.o(EXRAM)
1751	__heap_base	0x6c000008	Data	0 startup_stm32f40xx.o(HEAP)
1752	__heap_limit	0x6c000208	Data	0 startup_stm32f40xx.o(HEAP)
1753				

图 48-60 在 map 文件中查看工程的存储分布(SRAM.map 文件)

从 map 文件中可看到普通变量及栈节区都被分配到了内部 SRAM 的地址区域，而指定到 EXRAM 节区的变量及堆空间都被分配到了外部 SRAM 的地址区域，与我们的要求一致。

再把程序下载到实验板进行测试，串口打印的调试信息如图 48-61。

```
SCT文件应用——自动分配变量到外部SRAM实验

使用“uint32_t inerTestValue =10;”语句定义的局部变量:
结果: 它的地址为: 0x20000404, 变量值为: 10

使用“uint32_t testValue =7;”语句定义的全局变量:
结果: 它的地址为: 0x6c000000, 变量值为: 7

使用“uint32_t testValue2 =0;”语句定义的全局变量:
结果: 它的地址为: 0x6c000004, 变量值为: 0

使用“uint8_t testGrup[100] = {0};”语句定义的全局数组:
结果: 它的地址为: 0x6c000078, 变量值为: 0, 0, 0

使用“uint8_t testGrup2[100] = {1, 2, 3};”语句定义的全局数组:
结果: 它的地址为: 0x6c000008, 变量值为: 1, 2, 3

使用“uint32_t *pointer = (uint32_t*)malloc(sizeof(uint32_t)*3);”动态分配的变量

定义后的操作为:
*(pointer++)=1;
*(pointer++)=2;
*pointer=3;

结果: 操作后它的地址为: 0x6c0000f0, 查看变量值操作:
*(pointer--)=3,
*(pointer--)=2,
*(pointer)=1,
```

图 48-61 空间分配实验实测结果

从调试信息中可发现，实际运行结果也完全正常，本实验中的 sct 文件配置达到了优先分配变量到内部 SRAM 的目的，而且堆区也能使用 malloc 函数正常分配空间。

本实验中的 sct 文件配置方案完全可以应用到您的实际工程项目中，下面进一步强调其它应用细节。

使用 malloc 和 free 管理外部 SRAM 的空间

外部 SRAM 的内存空间非常大，为了管理这些空间，一些工程师会自己定义内存分配函数来管理外部 SRAM 空间，这些分配过程本质上就是从外部 SRAM 中动态分配内存。从本实验中可了解到我们完全可以直接使用 C 标准库的 malloc 从外部 SRAM 中分配空间，只要在前面配置的基础上修改启动文件中的堆顶地址限制即可，见代码清单 48-30。

代码清单 48-30 修改启动文件的堆顶地址(startup_stm32f40xx.s 文件)

```
1 Heap_Size      EQU      0x00000200
2
3                AREA      HEAP, NOINIT, READWRITE, ALIGN=3
4
5
6 __heap_base
7 Heap_Mem        SPACE    Heap_Size
8 __heap_limit    EQU      0x6C100000      ;设置堆空间的极限地址(外部 SRAM),
                                           ;0x6c000000+0x00100000
9
10               PRESERVE8
11               THUMB
```

C 标准库的 malloc 函数是根据 __heap_base 及 __heap_limit 地址限制分配空间的, 在以上的代码定义中, 堆基地址 __heap_base 的由链接器自动分配未使用的基地址, 而堆顶地址 __heap_limit 则被定义为外部 SRAM 的最高地址 0x6c000000+0x00100000(使用这种定义方式定义的 __heap_limit 值与 Heap_Size 定义的大小无关), 经过这样配置之后, 外部 SRAM 内除 EXRAM 节区外的空间都被分配为堆区, 所以 malloc 函数可以管理剩余的所有外部 SRAM 空间。修改后, 它生成的 map 文件信息见图 48-62。

Address	Symbol	Value	Type	Count	Object
1745	__stdout	0x20000018	Data	4	stdout.o(.data)
1746	__microlib_freelist	0x2000001c	Data	4	mvars.o(.data)
1747	__microlib_freelist_initialised	0x20000020	Data	4	mvars.o(.data)
1748	__initial_sp	0x20000428	Data	0	startup_stm32f40xx.o(STACK)
1749	testGrup	0x6c000000	Data	3	main.o(EXRAM)
1750	testValue	0x6c000004	Data	4	main.o(EXRAM)
1751	__heap_base	0x6c000008	Data	0	startup_stm32f40xx.o(HEAP)
1752	__heap_limit	0x6c100000	Number	0	startup_stm32f40xx.o ABSOLUTE

Base Addr	Size	Type	Attr	Idx	E Section Name	Object
0x6c000000	0x00000008	Data	RW	4769	EXRAM	main.o
0x6c000008	0x00000200	Zero	RW	2	HEAP	startup_stm32f40xx.o

图 48-62 使用 malloc 管理剩余 SDRAM 空间

可看到 __heap_base 的地址紧跟在 EXRAM 之后, __heap_limit 指向了外部 SRAM 的最高地址, 因此 malloc 函数就能利用所有外部 SRAM 的剩余空间了。注意图中显示的 HEAP 节区大小为 0x00000200 字节, 修改启动文件中的 Heap_Size 大小可以改变该值, 它的大小是不会影响 malloc 函数使用的, malloc 实际可用的就是 __heap_base 与 __heap_limit 之间的空间。至于如何使 Heap_Size 的值能自动根据 __heap_limit 与 __heap_base 的值自动生成, 我还没找到方法, 若您了解, 请告知。

把堆区分配到内部 SRAM 空间。

若您希望堆区(HEAP)按照默认配置, 使它还是分配到内部 SRAM 空间, 只要把 “*.o(HEAP)” 选择语句从外部 SRAM 的执行域删除掉即可, 堆节区就会默认分配到内部 SRAM, 外部 SRAM 仅选择 EXRAM 节区的内容进行分配, 见代码清单 48-31, 若您更改了启动文件中堆的默认配置, 主注意空间地址的匹配。

代码清单 48-31 按默认配置分配堆区到内部 SRAM 的 sct 文件范例

```
1 LR_IROM1 0x08000000 0x00100000 { ; load region size_region
2 ER_IROM1 0x08000000 0x00100000 { ; load address = execution address
3 *.o (RESET, +First)
4 *(InRoot$$Sections)
5 .ANY (+RO)
6 }
7 RW_IRAM1 0x20000000 0x00030000 { ; 内部 SRAM
8 .ANY (+RW +ZI) ;其余的 RW/ZI-data 都分配到这里
9 }
10
11 RW_ERAM1 0x6C000000 0x00100000 { ; 外部 SDRAM
12 .ANY (EXRAM) ;选择 EXRAM 节区
13 }
14 }
```

屏蔽链接过程的 warning

在我们的实验配置的 sct 文件中使用了 “*.o(HEAP)” 语句选择堆区, 但有时我们的工程完全没有使用堆(如整个工程都没有使用 malloc), 这时链接器会把堆占用的空间删除, 构建工程后会输出警告提示该语句仅匹配到无用节区, 见图 48-63。

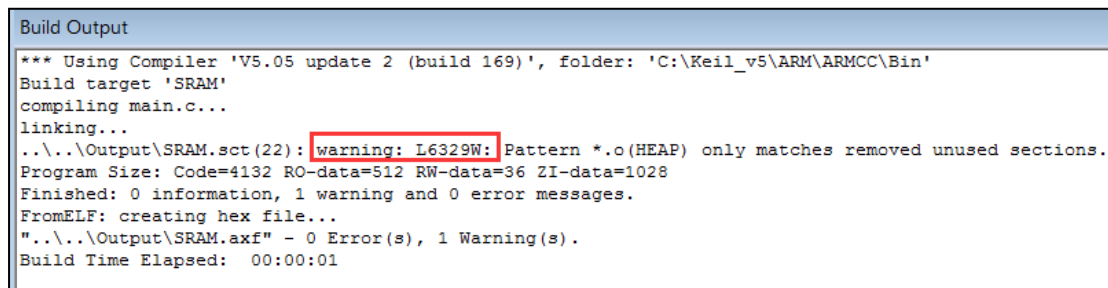


图 48-63 仅匹配到无用节区的 warning

这并无什么大碍, 但强迫症患者不希望看到 warning, 可以在 “Options for Target->Linker->disable Warnings” 中输入 warning 号屏蔽它。warning 号可在提示信息中找到, 如上图提示信息中 “warning: L6329W” 表示它的 warning 号为 6329, 把它输入到图 48-64 中的对话框中即可。

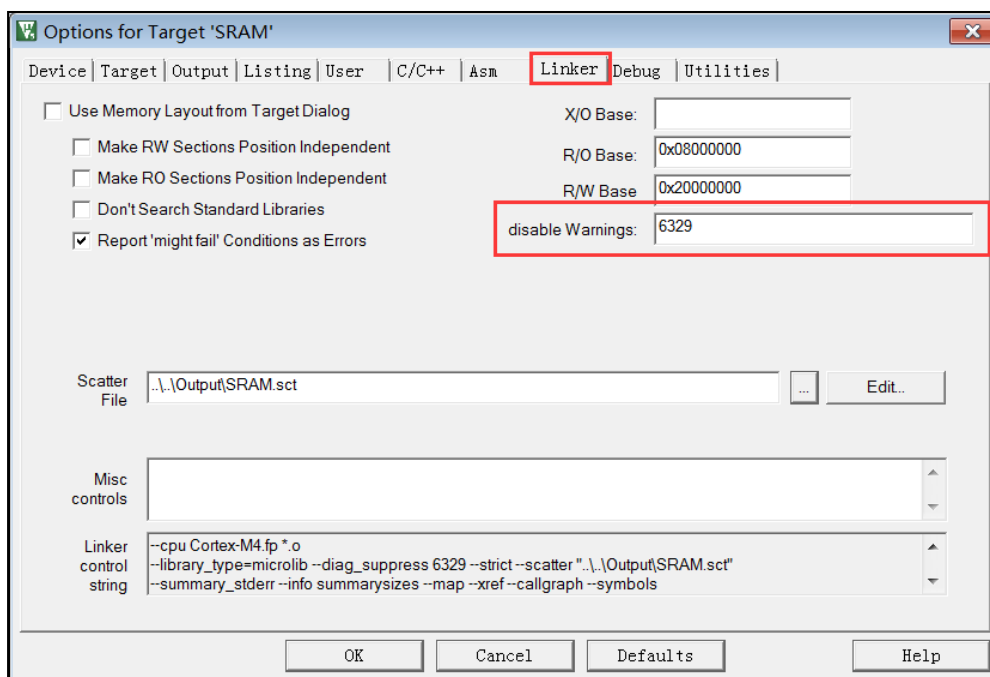


图 48-64 屏蔽链接过程的 warning

如何把栈空间也分配到外部 SRAM

前面提到因为 FSMC_SRAM_Init 初始化函数本身使用了栈空间(STACK), 而在执行 FSMC_SRAM_Init 函数之前外部 SRAM 并未正常工作, 这样的矛盾导致无法把栈分配到外部 SRAM。其实换一个思路, 只要我们的外部 SRAM 初始化过程不使用栈空间, 外部 SRAM 正常运行后栈才被分配到外部 SRAM 空间, 这样就没有问题了。

由于原来的 FSMC_SRAM_Init 实现的外部 SRAM 初始化过程使用了 STM32 标准库函数，它不可避免地使用了栈空间(定义了局部变量)，要完全不使用栈空间完成外部 SRAM 的初始化，只能使用纯粹的寄存器方式配置。在 STM32 标准库的“system_stm32f4xx.c”文件已经给出了类似的解决方案，SystemInit_ExtMemCtl 函数，见代码清单 48-32。

代码清单 48-32 SystemInit_ExtMemCtl 函数(system_stm32f4xx.c 文件)

```
#elif defined (DATA_IN_ExtSRAM)

1 /**
2  * @brief Setup the external memory controller.
3  *       Called in startup stm32f4xx.s before jump to main.
4  *       This function configures the external SRAM mounted on STM324xG_EVAL/STM324x7I boards
5  *       This SRAM will be used as program data memory (including heap and stack).
6  * @param None
7  * @retval None
8  */
9 void SystemInit_ExtMemCtl(void)
10 {
11     /*-- GPIOs Configuration -----*/
12
13     /* Enable GPIOD, GPIOE, GPIOF and GPIOG interface clock */
14     RCC->AHB1ENR |= 0x00000078;
15
16     /* Connect PDx pins to FMC Alternate function */
17     GPIOD->AFR[0] = 0x00cc00cc;
18     GPIOD->AFR[1] = 0xffffffff;
19     /* Configure PDx pins in Alternate function mode */
20     GPIOD->MODER = 0xaaaa0a0a;
21     /* Configure PDx pins speed to 100 MHz */
22     GPIOD->OSPEEDR = 0xffff0f0f;
23     /* Configure PDx pins Output type to push-pull */
24     GPIOD->OTYPER = 0x00000000;
25     /* No pull-up, pull-down for PDx pins */
26     GPIOD->PUPDR = 0x00000000;
27     /*具体配置省略*/
28
29 }
```

该函数没有使用栈空间，仅使用 register 关键字定义了两个分配到内核寄存器的变量，其余配置均通过直接操作寄存器完成。这个函数针对 ST 的一个官方评估编写的，在其它硬件平台直接使用可能有错误，若有需要可仔细分析它的代码再根据自己的硬件平台进行修改。

这个函数是使用条件编译语句“#elif defined (DATA_IN_ExtSRAM)”包装起来的，默认情况下这个函数并不会被编译，需要使用这个函数时只要定义这个宏即可。

定义了 DATA_IN_ExtSRAM 宏之后，SystemInit_ExtMemCtl 函数会被 SystemInit 函数调用，见代码清单 48-33，而我们知道 SystemInit 会在启动文件中的 Reset_handler 函数中执行。

代码清单 48-33 SystemInit 函数对 SystemInit_ExtMemCtl 的调用(system_stm32f4xx.c 文件)

```
1 /**
2  * @brief Setup the microcontroller system
3  *       Initialize the Embedded Flash Interface, the PLL and update the
4  *       SystemFrequency variable.
5  * @param None
6  * @retval None
7  */
8 void SystemInit(void)
9 {
10
```



```
11 /*****部分代码省略*****/  
12 #if defined(DATA_IN_ExtSRAM) || defined(DATA_IN_ExtSDRAM)  
13     SystemInit_ExtMemCtl();  
14 #endif /* DATA_IN_ExtSRAM || DATA_IN_ExtSDRAM */  
15 /*****部分代码省略*****/  
16 }
```

所以，如果希望把栈空间分配到外部 SRAM，可按以下步骤操作：

- ❑ 修改 sct 文件，使用 “*.o(STACK)” 语句把栈空间分配到外部 SRAM 的执行域；
- ❑ 根据自己的硬件平台，修改 SystemInit_ExtMemCtl 函数，该函数要实现外部 SRAM 的初始化过程，且该函数不能使用栈空间；
- ❑ 定义 DATA_IN_ExtSRAM 宏，从而使得 SystemInit_ExtMemCtl 函数被加进编译，并被 SystemInit 调用；
- ❑ 由于 Reset_handler 默认会调用 SystemInit 函数执行，所以不需要修改启动文件。